Bachelor Thesis

Nsak as Framework for Scenario Based Network Security

Course of study	Bachelor of Science in Computer Science
Author	Frank Gauss (gausf1) and Lukas von Allmen (vonal3)
Advisor	Wenger Hansjürg

Version 1.0 of May 15, 2026

Technik und Informatik

Abstract

One-paragraph summary of the entire study – typically no more than 250 words in length (and in many cases it is well shorter than that), the Abstract provides an overview of the study.

Contents

1. Introduction

According to the World Economic Forum's Global Risks Report 2026, cyber insecurity is listed among the top ten global risks and is projected to rise significantly from position 30 to position 5 in the long term, highlighting the growing adverse threat of AI technologies [1].

Complementing this global perspective, the CrowdStrike Global Threat Report highlights a significant rise in AI-enabled adversaries [2].

The absence of, or understaffing of, security experts, paired with the emerging complexity of systems and business models cite Figueredo2024RCVaR, highlights the increasing need for automated tools and frameworks capable of identifying and mitigating emerging cyber threats.

To address this, the *Network Swiss Army Knife (NSAK)*, a modular, open-source framework, is designed to orchestrate security scenarios consisting of reusable attack drills. The framework aims to support automated security testing and adversary emulation in controlled network environments.

The development of the NSAK framework builds upon previous project work, in which the core architecture, simplified configuration management, and a basic scenario execution model were developed. [3].

This thesis further extends the framework by enhancing configuration management for operating the NSAK device, establishing a reproducible test environment, and investigating how agentic AI can support automated red-team and blue-team scenarios.

The thesis is structured as follows: **Chapter 2** reviews related work and analyzes how selected security frameworks integrate AI into their products. **Chapter 3** introduces the fundamental concepts and outlines the key design decisions that form the basis of this work. **Chapter 4** describes the research methodology, including the approach used for evaluation and data collection. **Chapter 5** presents the implementation of the proposed solution, followed by **Chapter 6**, which evaluates its effectiveness in a virtual and physical test environments. Finally, **Chapter 7** discusses the results and their implications, while **Chapter 8** concludes the thesis and highlights potential directions for future work.

Match the structure overview to the actual structure

sharpen research question and contribution

1.1. Motivation

Hypothesis: AI can enhance red and blue team simulations in a network environment by generating and executing scenarios within the Network Swiss Army Knife (NSAK) framework and assisting in the evaluation of detection results. AI can enhance red and blue team simulations in a network environment by generating and executing scenarios within the NSAK framework and assisting in the evaluation of detection results.

1.2. Predecessor Project: Project 2

This section summarizes the predecessor project that led to the bachelor's thesis. The NSAK is a cybersecurity framework that operates with the following resources and concepts: *Devices*, which are physical or virtual machines capable of running the NSAK. The security operation is executed in a network environment and can involve one or more attacks or defense scenarios. The smallest unit in the framework is a drill, which can be orchestrated in a scenario and chained with other drills.

While the modularity and proof of concept were confirmed, several aspects like enhanced configuration management, Graphical User Interface (GUI) or a Representational State Transfer Application Programming Interface (REST API) are mentioned for future work.

The goals of this thesis directly address these open points and are defined in Section 1.3.

Proof Reading

1.3. Project Goals

For a goal to be considered **S.M.A.R.T.**, it must fulfill the following criteria:

- **Specific**: What do we want to achieve?
- **Measurable**: What are the success criteria?
- **Achievable**: How do we plan to achieve this goal?
- **Relevant**: Why is it important to achieve this goal?
- **Time-bound**: Goals categorized as *must* have to be achieved within the thesis, while *should* or *could* goals are considered optional or nice to have.

1.3.1. Research Goals

Compare Frameworks (must)

To identify the USP (unique selling point) of the NSAK framework, we will analyze and compare existing security emulation frameworks. First, relevant frameworks such as MITRE Caldera and Atomic Red Team will be identified and reviewed. Based on this analysis, the identified concepts will be compared with the architecture of the NSAK framework. The comparison will focus on how scenarios and drills are defined, how configuration and parameter handling are structured, how reusable components are organized, how automation is achieved, and which AI capabilities they provide.

The analysis and comparison will be based on the following properties:

- Concepts
- Modularity
- Configurability
- Automation
- AI Capabilities

Success criteria:

- At least two security emulation frameworks are identified and reviewed
- A matrix comparing the specified properties of the selected frameworks is created
- The USP of the NSAK framework is identified

AI Integration Research (must)

The goal of this research is to investigate how AI (artificial intelligence) can support automated security testing and adversary emulation. The research evaluates potential integration points where AI could enhance the scenario execution, automation, decision-making, and analysis capabilities of the NSAK framework.

Success criteria:

- How is AI currently used in cybersecurity automation?
- Which tasks in adversary emulation can be supported or automated by AI?

- How could AI interact with the NSAK scenarios and drills (e.g., Agents, MCP)?
- What architectural changes would be required to integrate AI into NSAK?

1.3.2. Framework

Configuration Management (must)

The goal is to extend the NSAK framework, so that scenarios and drills can expose configurable parameters to increase the flexibility of scenarios and reusability of drills. Parameters can be provided via command-line arguments or YAML Ain't Markup Language (YAML) files and are resolved by the framework into a unified run configuration used for scenario execution.

Success criteria:

- Scenarios and drills can expose configurable parameters
- The available parameters can be listed by the Command Line Interface (CLI) with the `--help` option
- Parameters can be provided via CLI arguments or YAML files
- The framework resolves the inputs into a run configuration (internal data structure).

REST API (could)

Adding a REST Application Programming Interface (API) to the NSAK framework was already proposed in the predecessor project, as it greatly enhances interoperability with other tools. The priority of this goal depends heavily on the outcome of the goal 1.3.1 and on whether we want to fulfill the goal 1.3.2, which is prioritized as "could". If the evaluated AI integration requires a REST API (e.g., MCP), this goal needs to be prioritized as "must"; it will be prioritized as "could".

Success criteria:

- A REST API specification with feature parity to the CLI is created.
- The REST API is implemented according to the specification.
- The API is documented (OpenAPI) and usable by external clients.

- The REST API remains optional and does not replace the CLI-based interaction.
- Authentication and Authorization are explicitly out of scope.

MCP Server (could)

The MCP Protocol allows the AI to interact with applications in a standardized way - and could therefore be necessary for implementing AI-based scenarios. If the outcome of the research task 1.3.1 highlights the necessity to specify/expose an MCP-Server to interact seamlessly with the NSAK framework, the implementation is a (must), otherwise a (could).

- A MCP-Server is specified and implemented
- Documentation how to connect to the MCP is provided.
- A form of authentication is provided.

Web GUI (could)

The goal is to design and implement a web-based graphical user interface that allows users to interact with the NSAK framework in a user-friendly way. The Web GUI will enable management of scenarios and drills, triggering scenario execution, and visualization of execution results and system status.

Success criteria:

- A simple Web GUI that interacts with the REST API has been implemented.
- The Web GUI displays a list of available scenarios and their states.
- The scenario's results can be viewed on the Web GUI
- Scenarios can be configured and executed via the Web GUI
- Authentication and authorization are explicitly out of scope.

1.3.3. Scenarios

AI Scenarios (must)

The goal is to implement artificial intelligence according to the result of 1.3.1 and to explore the use of AI to support red, blue, and purple team exercises in the NSAK framework. The AI will be evaluated for its potential to assist in executing attacks in a scenario and analyzing detection results. The findings will help assess how AI could enhance and coordinate red and blue team activities across a network via the NSAK device.

Success criteria:

- The AI is integrated into NSAK as a scenario or as part of the framework, depending on the result of 1.3.1.
- The AI can execute CLI or Python tools to directly or indirectly interact with the attached network.
- An operator can interact with or stop the running AI.
- The result artifacts are stored and can be presented or reused for further scenarios.

Reproducible Test Environment (must)

The goal is to design and implement a reproducible and safe test environment for the NSAK framework that enables consistent execution and evaluation of scenarios and drills. The environment should provide a stable basis for testing and comparison of manual and AI-assisted scenarios.

Success criteria:

- The test environment can be set up reproducibly using a defined configuration or setup procedure with Infrastructure as code (e.g., Ansible)
- NSAK scenarios and drills can be executed consistently within the environment.
- The environment supports both manual and AI-assisted scenarios.
- Scenario results can be collected and analyzed for evaluation purposes.

Network Discovery – Red Team (must)

The goal is to implement a reference red team scenario for network discovery within the NSAK framework. The scenario should simulate reconnaissance activities to identify hosts, services, and network characteristics, and the results should be comparable to the AI-Agent approach.

Success criteria:

- A red team network discovery scenario is implemented in the NSAK framework.
- Discovered network information is collected as artifacts.
- The scenario can be executed reproducibly within the test environment.
- The results can be compared with those generated by the AI-assisted approach.

Intrusion Detection – Blue Team (should)

The goal is to define and implement a blue team reference scenario for intrusion detection within the NSAK framework. The scenario should allow observation and evaluation of suspicious network activity to assess detection capabilities in the test environment and compare the results with those of the AI approach.

Success criteria:

- A blue-team intrusion-detection scenario is implemented in the NSAK framework.
- Detection events or alerts can be observed and documented during scenario execution.
- Detection results can be compared with results from AI-assisted scenario execution.

1.3.4. Evaluation Goals

AI Scenario Evaluation (must)

The objective is to determine the effectiveness of the AI-based scenario introduced by the goal 1.3.3 within the network environment defined by the goal 1.3.3 The evaluation will assess whether the AI scenario successfully executed the expected red and blue team activities for the provided prompts and the quality of the results.

Success criteria:

- A methodical approach for the assessment is evaluated and documented.
- The results of the AI scenario for red and blue team activity are assessed.
- There is a conclusion on the effectiveness of the AI scenario for red and blue team activities.

AI vs Engineered Scenarios (must/should)

The goal of this evaluation is to compare the quality of AI-based red (network reconnaissance) (must) and blue (network intrusion detection) (should) team scenarios with that of manually engineered scenarios.

The comparison focuses on criteria such as execution reliability, reproducibility, and coverage. The results will help determine the potential benefits and limitations of AI-supported scenarios and provide insights into how AI could enhance future versions of the NSAK framework.

Success criteria:

- A methodical approach for the comparison is evaluated and documented.
- The results of the AI scenario are compared against the respective results of the red team scenario (must)
- The results of the AI scenario are compared against the respective results of the blue team scenario (should)
- There is a conclusion on the effectiveness of the AI scenario compared to manually engineered scenarios.

1.3.5. Out of Scope / Optional Feature

At the project management meeting on 05.03.26 (see Table A.17 in Appendix A.7), the initial project scope was discussed. During planning, the team proposed using AI-based scenarios and comparing them with the implemented versions, which required reassessing certain milestones.

Running AI-based scenarios does not require a REST API or GUI. A reproducible test setup and further research are needed to properly evaluate this approach.

2. Related Research

2.1. Framework Comparison

Previous research, such as 'SOK: A Survey of Open-Source Threat Emulators', compares multiple types of threat emulation frameworks using both quantitative and qualitative metrics [4]. Among the evaluated frameworks, Atomic Red Team, Mitre Caldera, and Metasploit received high scores in categories as environments and operator, environment configuration and scenario execution.

n the following section, the selected frameworks are briefly introduced, and the unique selling point of the NSAK is highlighted. As a point of reference, the cybersecurity community increasingly relies on the concepts of MITRE ATT&CK, which has established itself as a standard framework in the field since its introduction in 2014 [5, 6].

The MITRE ATT&CK is a globally accessible knowledge base of adversary tactics and techniques based on real-world observations [5]. The knowledge base is structured into TTP (Tactics, Techniques, and Procedures), which are organized into 14 attack behavior groups. The TTP is getting continuously updated and extended, which helps to defend against the capabilities of threat actors.

The evaluation of the proposed frameworks is based on information obtained from official documentation [7–9] and the findings of Zilberman et al. [4].

Characteristic	Metasploit Framework	Atomic Red Team	MITRE Caldera
Primary Purpose	Exploitation and penetration testing framework	Security testing through small atomic ATT&CK tests	Automated adversary emulation platform
Relation to MITRE ATT&CK	Partial mapping via modules	Direct mapping to ATT&CK techniques	Built around ATT&CK adversary emulation
Automation Level	Medium (scripts and modules)	Low (manual execution of tests)	High (automated attack chains)
Main Functionality	Exploit development, payload delivery, post-exploitation	Execute small ATT&CK technique tests	Simulate adversary campaigns across hosts
Architecture	Modular framework with exploits, payloads, and auxiliary modules	Collection of scripts/tests organized by ATT&CK techniques	Client-server platform
Typical Use Case	Penetration testing and vulnerability validation	Detection validation and security control testing	Red team automation and purple team exercises
Difficulty Level	Medium–High	Low–Medium	Medium–High

Table 2.1.: Comparison of Metasploit, Atomic Red Team, and MITRE Caldera ([4, 7–9])

Table formatting

As shown in Table 2.1, the key characteristics of the frameworks are differ greatly. **Atomic Red Team** provides libraries of attack scripts that implement individual Tactics, Techniques, and Procedures of MITRE ATT&CK. Each attack can be executed independently or chained together into a larger attack scenario. **Mitre Caldera** is a complex, agent based adversary simulation tool, whereas **Metasploit** is a framework that provides exploits, payloads and auxiliary modules. Further details of the frameworks are presented in the following sections. 2.1.12.1.22.1.3

2.1.1. Atomic Red Team

The Atomic Red Team repository provides documentation and configuration files for each technique. For every TTP, a YAML configuration file defines the execution parameters, while an enhanced Markdown file contains a human-readable description of the attack. This documentation includes a short description of the technique, execution instructions, and additional contextual information [8].

The framework can be executed directly from the command line and provides

features such as logging, detailed error messages, and automatic prerequisite checks. If a technique cannot be executed, for example due to operating system constraints, the framework reports the issue and can optionally attempt to install the required prerequisites [8].

Zilberman et al. recommends deeper cybersecurity knowledge to use Atomic Red Team framework effectively [4]. The Open Source mentality, expandability and easily usable scripts can be a great asset for an independent framework such as the NSAK.

2.1.2. Caldera

Mitre Caldera is also built on the MITRE ATT&CK knowledge base and functions as a command-and-control center. A central server controls agents running on the target system, executes attacks called abilities, and sends the operation results back to the server. Every Ability is a single attack step built on the Tactics, Techniques, and Procedures. The framework ships with a graphical user interface and is primarily used for red team attack simulation, purple teaming, and detection engineering. The additional features, like a training plugin for educational purposes, include facts that can be configured as dynamic safe values, which can be stored and parsed in later steps [7].

With its provided interface and functionality, Caldera is most suitable as a threat emulator, offering a wide range of built-in features, including lateral movement and Command and Control (CNC) communication [4].

2.1.3. Metasploit

Metasploit is a widely used penetration testing framework for testing or attacking systems and exploiting known vulnerabilities. Metasploit follows a sequence of steps to identify a vulnerability in a target system, select a vulnerability for the attack, select an exploit that uses this vulnerability, and deliver a payload that runs on the target system. The configuration via the CLI requires knowledge of the target network and system and requires partial configuration [9, 10].

According to Zilberman et al., an operator requires extensive cybersecurity knowledge to effectively use the framework. To support the operator, the GUI “Armitage”, which is not further evaluated in this work, can be used [4].

2.1.4. NSAK

The NSAK framework, as an independent universal Network Swiss Army Knife, is not confined and can leverage, if needed, different sets of attack frameworks.

An approach could be to use an AI-Enhanced Network Discovery Scenario, which provides the necessary information to configure a Metasploit attack that leads to an attack operation in Caldera and an atomic red team detection script. Hardware-based isolation and network position provide flexibility and should help reduce the operational costs of red and blue teams by reducing simple configuration overhead [3].

2.2. AI-Research

2.2.1. AI and Language Models

Language Model (LM) describes the technique for advancing language intelligence in machines and its development history can be divided into several stages. The first stage, Statistical Language Model (SLM) emerged around the 1990s, and the basic idea is to assume, that such a model can predict the next word. But the fact that an exponential number of transition probabilities are needed, caused a lack of accuracy and led limited success.

To overcome these limitations, Neural Language Model (NLM) was introduced to learn a distributed representation of words and capture more complex patterns, leading to a major breakthrough. Building on this, the advancement of pre-trained language models Pre-trained Language model (PLM), which are first trained on large datasets and then fine-tuned for specific tasks, represented another significant development.

Building on this development, Large Language Model (LLM)s further increases the scale of PLMs by expanding parameter count and computational power [11].

In large language models, parameters are the trainable numerical values that define how the model processes input and generates output. During training, these parameters are continually adjusted to enable the model to learn patterns and language context [12]. The models most of us use today, such as ChatGPT, Claude and Gemini are pre-trained LLMs used for language inference.

These advances led to MCP, a protocol that allows tools and services to expose an interface specifically for AI. Further, the introduction of AI-agents is enabling a new level of autonomy for achieving concrete goals with LLMs. The combination of these technologies is now resulting in systems, where LLMs empower autonomous agents capable of decision-making to execute complex tasks.

2.2.2. MCP

Anthropic, a US software company focused on AI research, introduced the protocol in 2024. The MCP was introduced as an open source standard that enables AI assistants to connect to external systems, including databases, business tools, and development environments [13]. Dynamic tool discovery and schema negotiation is helping clients and agents to access these functions in a standardized manner and allows autonomous interaction. Additionally, MCP allows information to flow in both directions, so the model can send requests, and tools can send updates or events.

This architectural approach shifts the AI integration from static toward an interoperable ecosystem of AI-enabled services. On the other hand, the flexible design and open source approach of MCP unlocks multiple novel and systematic security risks that are in need of enhanced security and governance [14].

2.2.3. Agentic-AI

Agentic Artificial Intelligence (Agentic-AI) can be understood as an advanced machine intelligence that is capable of perceiving, reasoning, and acting with a certain level of autonomy. Traditional systems and agents are more limited to executing predefined commands, whereas Agentic-AI is not restricted in this way. Due to their ability to adapt in real time, they can continuously refine their knowledge, techniques, and data. The ability to access multiple inputs, from natural language processing to sensor data, allows Artificial Intelligence Agent (AI Agent)s to develop a good understanding of their environment [15].

Multi Agent System (MAS) defines a way in which multiple specialist agent coordinate and cooperate with each other to solve a task, that would not be solvable to one agent alone.

Therefore, the agents need a form of interaction that allows them to act reactively, often through a shared mental model, which helps create a common understanding of the complex tasks to be solved. To implement a MAS efficiently, modern design principles such as encapsulation and modularity are important, which are helping to balance resource usage of the system and LLM [16]

Huang et al. provide evidence that Agentic-AI and MAS are capable of solving complex tasks across various sectors, including security and banking. However, these systems also introduce new potential vulnerabilities, both within AI Agent systems themselves and through their interactions with external environments.

The identified vulnerabilities can be categorized into two types: accidental and deliberate. Accidental vulnerabilities include software bugs, logical errors introduced by agents, hardware malfunctions, and poor data quality. Deliberate

attacks, on the other hand, encompass adversarial attacks targeting LLMs, data poisoning, and model theft.

To mitigate these risks, a comprehensive framework consisting of governance mechanisms, proactive monitoring, regulatory compliance, and rigorous testing is required to ensure the secure and reliable operation of agentic systems [17].

Fazit MCP and Agentic AI

In the cybersecurity podcast Risky Business, it is mentioned that MCP is "dead". The open-source strategy of Anthropic (2.2.2) has led to the development of multiple MCP implementations that were not designed with a strong focus on security, but rather on rapid adoption of a novel technique.

This provocative claim is based on the observation that, in order to handle the excessive MCP tooling, AI Agents may instead rely on direct access to a root shell, as it provides a more efficient way of interaction [18].

How and why is this relevant for our thesis? how could we implement it? To which findings should we pay attention? What can we ignore for now?

Evaluated Frameworks and MCP

Caldera uses a MCP plugin as a bridge between the model and the REST API, allowing the agent to plan and issue commands in a manner similar to a human operator.

Caldera introduces three main research areas. Firstly, a planner determines the order and selection of abilities and can dynamically construct new adversaries based on user requirements through prompt-based interaction.

Secondly, an adversary factory model is used to generate adversaries tailored to the operator's specific use cases.

Thirdly, a forward-deployed agent is enhanced with Retrieval Augmented Generation (RAG) for Cyber Threat Intelligence (CTI), leveraging Structured Threat Information Expression (STIX)-based data to create reproducible and testable adversaries [19].

The MCP-Server, provided by **Metasploit**, enables a LLM to interact with the Metasploit Framework dynamically. The Interface covers module information (a Metasploit module is a known exploit), exploitation workflows, payload generation, session management and handler management to help manage an attack. The documentation includes a security warning that emphasizes the importance of reviewing every prompt, using a test environment, and being aware of post-exploitation commands. **Metasploit** is a powerful exploitation framework, and the simplicity that AI could bring can also harm. [20]

Ellaborate on this, because we will actually not use MCP

I think we are to focused on the term MCP now. We should speak of tool-calling, MCP is

The **Atomic Red Team** MCP Server provides an interface that allows AI systems to access and use tests from Atomic Red Team. It enables searching, validating, and optionally executing attack techniques (e.g., based on MITRE ATT&CK). [21]

What about the other frameworks? What about AI Agents? What can we learn from their approach? How does it affect our strategy?

Outcome Comparison

The framework comparison highlights the difference between the NSAK and the established frameworks. Each compared framework performs a specific task in a complex operation, and the NSAK device can leverage the strengths of multiple aspects of a different set of tools. Building upon these insights, the NSAK is designed to extend and integrate these capabilities into a unified and adaptable system and the change of focus to AI gets further elaborated in this thesis.

3. Concepts

3.1. Framework Resources

The NSAK framework is built around four resource types that were originally defined in "Project 2" 1.2 [3]. Each resource type has a distinct role in the framework and is stored as a YAML file in the resource library.

3.1.1. Device

A **Device** resource describes a physical or virtual machine that is capable of running the NSAK framework. It serves as the execution host for scenarios and drills and is typically deployed at a strategic network position to give it the access required for the operations it performs.

A device exposes its network configuration to the framework by declaring its interfaces and their Internet Protocol addresses in its resource YAML. As an alternative, a special device with the id "Unknown" was added, which auto discovers the hosts network configuration. This allows scenarios and drills to resolve interface names and target addresses without hardcoding network details, making the same scenario portable across different devices and network environments.

3.1.2. Environment

An **Environment** represents a specific network topology, including its infrastructure components, hosts, and services [3]. It describes the target network context in which a scenario is intended to operate. For example, a simple client–server setup over a switch, a Wireless LAN access point with connected clients, or a segmented business network.

Environments are referenced by scenarios to make explicit which network context they are designed for, helping operators select and validate the right configuration before executing an operation.

3.1.3. Drill

A **Drill** is a self-contained, reusable unit of execution with a specific, well-defined goal [3]. Its goal can be an active or passive attack step, network discovery, traffic capture, a configuration change, or any other discrete action relevant to a red or blue team activity.

Examples of drills include Address Resolution Protocol spoofing, host discovery via Address Resolution Protocol scan, transparent Transmission Control Protocol proxying, traffic capture with T-Shark, and network configuration tasks such as enabling IP forwarding or Network Address Translation (NAT).

Each drill declares the system and Python packages it requires as well as a typed `interface` that lists its input arguments and return type. The return value of one drill can be consumed as the input of a subsequent drill within a scenario, enabling data to flow through a chain of execution steps.

3.1.4. Scenario

A **Scenario** describes a concrete red or blue team use case and orchestrates a sequence of drills targeted at one or more environments [3]. Where a drill is a single technical reusable and modular building block, a scenario is an implementation of a specific attack or defense operation.

Like drills, scenarios declare a typed `interface` with input arguments, allowing operators to parameterise a scenario at execution time via the CLI without modifying the scenario definition itself. This separation keeps scenarios reusable across different network configurations while still allowing runtime flexibility.

3.2. Configuration Management

The NSAK framework distinguishes three configuration layers, each with a different scope and lifetime. Keeping them separate avoids coupling concerns that change at different rates and makes it easier to reason about where a given piece of configuration comes from.

3.2.1. Static Configuration

The static configuration layer involves values which are specific for a deployment and are very unlikely to change after the initial provision of the NSAK device. It mainly establishes the filesystem layout that all other parts of the framework

depend on. For example, where the resource library is located, where runtime data is written, and what the base directory of the installation is.

3.2.2. Runtime Configuration

The runtime configuration layer holds mutable state, which might be changed by an operator via the CLI. It is stored as a YAML file, which is currently loaded at the start of every invocation. If no configuration file exists yet, a default is created automatically on first access, so no explicit initialization step is required from the user.

3.2.3. Resource Configuration

The resource configuration layer is provided by the resource library rather than the framework core. Each resource declares its own configuration inside a YAML file stored alongside its implementation. The framework reads these files to discover and load resources at runtime.

Typed Interface Arguments

Drills and scenarios declare a typed `interface` in their resource configuration YAML file, that lists the arguments with an explicit type annotation and an optional default value. The framework reads these annotations at runtime and automatically generates the correct CLI options for each resource. This means that adding a new argument to a drill or scenario immediately exposes it as a typed CLI option without requiring any changes to the framework or the CLI code.

Mergeable YAML Structure

All resource YAML files follow a common structure in which resources are nested under a resource-type key (e.g. `drills:`, `scenarios:`). This structure ensures that YAML files from different parts of the library could be merged into a single in-memory data structure without key conflicts, which is a prerequisite for supporting multiple library paths and composite library configurations in the future.

4. Methodology

5. Implementation

5.1. Configuration Management Implementation

The configuration management in NSAK is organised into three distinct layers, each serving a different scope and lifetime.

The **static configuration** layer resolves fixed filesystem paths at import time. It derives all relevant paths from a single base directory, which can be overridden via an environment variable to support different deployment contexts such as Docker containers or automated test environments.

The **runtime configuration** layer holds mutable state that persists across CLI invocations. It is loaded from a YAML file on every command execution and written back whenever state changes, for example, when a different device is loaded. This layer serves as the single source of truth for any information that must be shared between separate CLI calls.

The **resource configuration** layer is defined by the resource library itself. Each resource type –devices, drills, scenarios, and environments –declares its configuration inside a YAML file stored in the library. For devices, this includes network interface assignments, while drills and scenarios additionally declare their typed argument interfaces, which the CLI uses to generate the correct command options at runtime.

This section is repeating what's already stated in the concepts chapter...

5.1.1. NSAK Configuration Management

The NSAK configuration is split into two distinct layers: a static layer that resolves fixed paths at import time, and a mutable runtime layer that is persisted to disk and loaded on every invocation.

Static Configuration

The static configuration is defined in `src/nsak/core/settings.py` and shown in listing 1. It derives all relevant paths from a single `BASE_PATH`, which is either supplied via the `NSAK_BASE_PATH` environment variable or resolved automatically to the repository root at import time. `RUN_PATH` points to the `run/` directory where runtime data such as the persisted config is stored, and `LIBRARY_PATHS` holds the set of directories that are searched for resource libraries. Overriding `NSAK_BASE_PATH` is the primary mechanism to run NSAK from a different base directory, e.g. inside a Docker container or during automated testing.

```python
import logging
import os
from pathlib import Path

from dotenv import load_dotenv

load_dotenv(os.getenv("NSAK_ENV_FILE", None))

logger = logging.getLogger()


def get_base_path() -> Path:
    """
    Returns the default base path.

    :return:
    """
    base_path = os.getenv("NSAK_BASE_PATH", None)

    if base_path is not None:
        return Path(base_path).absolute()

    return Path(__file__).resolve().parents[3].absolute()


def get_run_path() -> Path:
    """
    Returns the run path.

    :return:
    """
    run_path = os.getenv("NSAK_RUN_PATH", None)

    if run_path is not None:
        return Path(run_path).absolute()

    return (get_base_path() / "run").absolute()


def get_library_paths() -> set[Path]:
    """
    Returns a list of library paths.

    :return:
    """
    library_paths = set()

    default_path = (BASE_PATH / "lib").absolute()
    if default_path.exists():
        library_paths.add(default_path)

    library_path = os.getenv("NSAK_LIBRARY_PATH", None)
    if library_path is not None:
        library_path = Path(library_path).absolute()
```

Runtime Configuration

The runtime configuration is defined as the "Config" dataclass in `src/nsak/core/configuration`, and holds all state that can change during normal operation, currently the `debug` flag and the active "Device". It is persisted as a YAML file at `run/config.yaml` and loaded on every CLI invocation.

To avoid loading the configuration on import, the module-level `config` object is wrapped in a `lazy_object_proxy.Proxy`. The proxy defers the actual `Config.load()` call until the object is first accessed. Since the root CLI group imports `config` on every command invocation (listing 3), this ensures the configuration is always initialised before any command logic runs. If `run/config.yaml` does not yet exist, `Config.init()` is called automatically to create a default configuration and write it to disk, eliminating the need for manual setup on first run.

```python
from __future__ import annotations

import dataclasses
import logging

from nsak.core.configuration.ai_configuration import AiConfiguration
from nsak.core.configuration.container_info import ContainerInfo
from nsak.core.configuration.email_configuration import EmailConfiguration
from nsak.core.configuration.loki_configuration import LokiConfiguration
from nsak.core.device import Device

logger = logging.getLogger(__name__)


@dataclasses.dataclass(kw_only=True)
class Configuration:
    """
    Class for loading and saving the configuration.
    """

    from ..device.device_manager import DeviceManager

    debug: bool = dataclasses.field(default=True)
    device: Device = dataclasses.field(
        default_factory=DeviceManager.get_default,
    )
    container: ContainerInfo = dataclasses.field(
        default_factory=ContainerInfo.load,
    )
    email: EmailConfiguration | None = dataclasses.field(
        default=None,
    )
    ai: AiConfiguration | None = dataclasses.field(default=None)
    loki: LokiConfiguration | None = dataclasses.field(default=None)

    def save(self) -> None:
        """
        Shorthand proxy for ConfigManager save.
        """
        from .configuration_manager import ConfigurationManager
                                25
        ConfigurationManager.save(self)
```

Listing 2: NSAK Runtime Configuration

```python
import click

from .ai_agent import ai_agent_group
from .config import config_group
from .device import device_group
from .drill import drill_group
from .environment import environment_group
from .scenario import kill_all_scenarios, scenario_group


@click.group()
def cli() -> None:
    """
    CLI root.
    """
    # Load the configuration for initialization
    from nsak.core import config  # noqa


cli.add_command(scenario_group)
cli.add_command(drill_group)
cli.add_command(device_group)
cli.add_command(environment_group)
cli.add_command(ai_agent_group)
cli.add_command(config_group)
cli.add_command(kill_all_scenarios, name="killswitch")
```

Listing 3: NSAK CLI Initialisation

Mutations to the runtime configuration always follow the same pattern: a manager method (e.g. `DeviceManager.load()`) updates the relevant field on the `config` object and calls `config.save()` to persist the change. Custom YAML representers are registered for `IPInterface` and `PosixPath` so that these types are serialised as plain strings rather than Python-specific tags. Listing 4 shows an example of the persisted `run/config.yaml` after loading the `bananapi_r4` device.

```yaml
debug: true
device:
  id: bananapi_r4
  name: BananaPI R4
  path: <base-path>/lib/devices/bananapi_r4
  author: vonal3@bfh.ch
  repository: <repository-url>
  configuration:
    network:
      ethernets:
        lan1@eth0:
          addresses:
            10.10.10.30/24:
              is_target: true
              is_management: false
```

Listing 4: Persisted Runtime Configuration (run/config.yaml)

5.1.2. Device Configuration Management

In the context of "Project 2"1.2, this resource type was already defined conceptually, and the BananaPI R4 was added as an example in the library under `lib/devices/bananapi_r`
But the resource type was not actually added to the NSAK core, and to manage device configurations, we need to implement the Device resource type first.

Create New Device Resource Type

Similar to the other resource types, a YAML specification for defining devices in the library and multiple classes for representing and managing devices in the core are required. Because the existing resource YAML specifications do not allow merging YAML files, this implementation proposes a new base structure that fulfills this trait. Section 5.1.2 describes the necessary changes to the existing YAML specifications. The listing 5 shows the initial specification for a device resource, which will get extended later on with a section for configuration management.

```yaml
devices:
    <str:resource-id>:
        id: <str:device-id>
        name: <str:device-name>
        description: <str:device-description>
        author: <str:author-email>
        repository: <str:resource-repository>
```

Listing 5: Device Resource YAML Specification

In the table 5.1 below, the required Python classes are listed and described. The classes will be located under `src/nsak/core/device` within their respective files.

Class	File Name	Description
Device	device.py	Dataclass representing a device resource at runtime, closely reflecting the YAML specification.
DeviceLoader	device_loader.py	Loads and returns Device instances from the resource library.
DeviceManager	device_manager.py	Implements business logic and exposes an API for use by the CLI.

Table 5.1.: Device Resource Classes

In "Project 2"1.2, it became already clear that the described classes share a lot of boilerplate code between each other. For this reason, the base classes described in 5.1 were added to eliminate duplicated logic, thereby improving compliance with the Don't Repeat Yourself principles. The classes will be located under `src/nsak/core/resource` within their respective files and the adoption of the existing classes is described in section 5.1.2.

Class	File Name	Description
Resource	resource.py	Base class for all resources: Scenario, Drill, Environment and Device.
ResourceLoader	resource_loader.py	Base class which handles searching and loading resources from the filesystem.
ResourceManager	resource_manager.py	Base class which defines common methods for all resource manager classes.

Table 5.2.: Resource Python Classes

To complete the integration of the device resource, the new functionality must be exposed via CLI. The available bash commands are shown in the following listing 6.

```
# Show usage, options and subcommands for the device resource
nsak device --list

# List all devices found in the resource library
nsak device list

# Output:
❯ nsak device list
ID           NAME         PATH
bananapi_r4  BananaPI R4  <repository>/lib/devices/bananapi_r4
```

Listing 6: NSAK CLI: List Devices

Device Configuration Management Implementation

To allow a device to expose its configuration options, the YAML specification needs to be extended accordingly. For now, the focus is only on network configuration, as this is the most important information for the scenarios and drills. The specification was designed so that other sections can be added later. The following excerpt 7 shows the specification for the configuration section of the device resource YAML.

```
devices:
    <str:resource-id>:
        # ...
        configuration:
          network:
            ethernets:
              <str:intreface-name>:
                addresses:
                  <str:cidr>:
                    is_target: <boolean>
                    is_management: <boolean>
```

Listing 7: Device Resource Configuration YAML Specification

The following code snippet, located under `src/nsak/core/device/device_loader.py` 8

shows how the YAML configuration gets parsed to the respective datastructures defined in `src/nsak/core/network/configuration.py` and `src/nsak/core/device/device.p`

```
class DeviceLoader(ResourceLoader[Device]):
    """
    Finds and loads devices from the library paths.
    """

    ResourceClass = Device

    @classmethod
    def _parse_device_configuration(
        cls, data: dict[str, Any]
    ) -> DeviceConfigration | None:
        if "configuration" not in data:
            return None
        network = data["configuration"].get("network") or {}
        if network == "auto":
            ethernets = {}
            for interface in get_network_interfaces():
                ethernets[interface.name] = {
                    "addresses": {
                        ip: {
                            "is_target": True,
                            "is_management": False,
                        }
                        for ips in interface.ips.values()
                        for ip in ips
                    }
                }
        else:
            ethernets = network.get("ethernets") or {}

        return DeviceConfigration(
            raw=data["configuration"],
            network=NetworkConfiguration(
                ethernets={
                    interface: EthernetConfiguration(
                        name=interface,
                        addresses={
                            ip: IPConfiguration(
                                ip=ip_interface(ip),
                                is_target=address.get("is_target",
                                False),
                                is_management=address.get("is_management",
                                False),
                            )
                            for ip, address in
                            ethernet.get("addresses",
                            {}).items()
                        },
                    )
                    for interface, ethernet in ethernets.items()
                }
            )
```

```python
@dataclasses.dataclass(frozen=True, kw_only=True)
class IPConfiguration:
    """
    Represents an ip configuration on an interface.
    """

    ip: IPInterface
    is_target: bool
    is_management: bool


@dataclasses.dataclass(frozen=True, kw_only=True)
class EthernetConfiguration:
    """
    Represents an ethernet node.
    """

    name: str
    addresses: dict[str, IPConfiguration]
```

```python
@dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
class DeviceConfigration:
    """
    Represents the device configuration.
    """

    raw: object
    network: NetworkConfiguration


@dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
class Device(Resource):
    """
    Represents a device.
    """

    NAME = "device"
    KEY = "devices"

    id: str
    name: str
    description: str
    path: Path
    author: str
    repository: str
    configuration: DeviceConfigration | None = None
```

Listing 9: Device Configuration Datastructures

The following listing shows the newly available CLI commands for managing device configurations 10:

```
# List all subcommands for the Device resource
nsak device --help

# List all available devices
nsak device list

# Show the device details and configuration
nsak device show <str:device-id>

# Load a device into in the NSAK configuration
nsak device load <str:device-id>

# Show the currently loaded device stored in the NSAK
↪  configuration
nsak device loaded

# Reset the loaded device
nsak device unload
```

Listing 10: Device Configuration CLI Management

Refactor Existing Resources

The implementation of the device resource 5.1.2 introduced an improved base structure for the resource YAML specifications and three base classes which are abstracting the common structure and functionality. This section describes the resulting changes in the existing code and resource YAML specifications.

As already stated in section 5.1.2 all resource type specific classes were refactored to inherit from their respective base classes as described in table 5.1. With the example of the `EnvironmentLoader`, the diff below 5.1 illustrates how the abstraction of common properties and methods allows for the removal of a lot of boilerplate code.

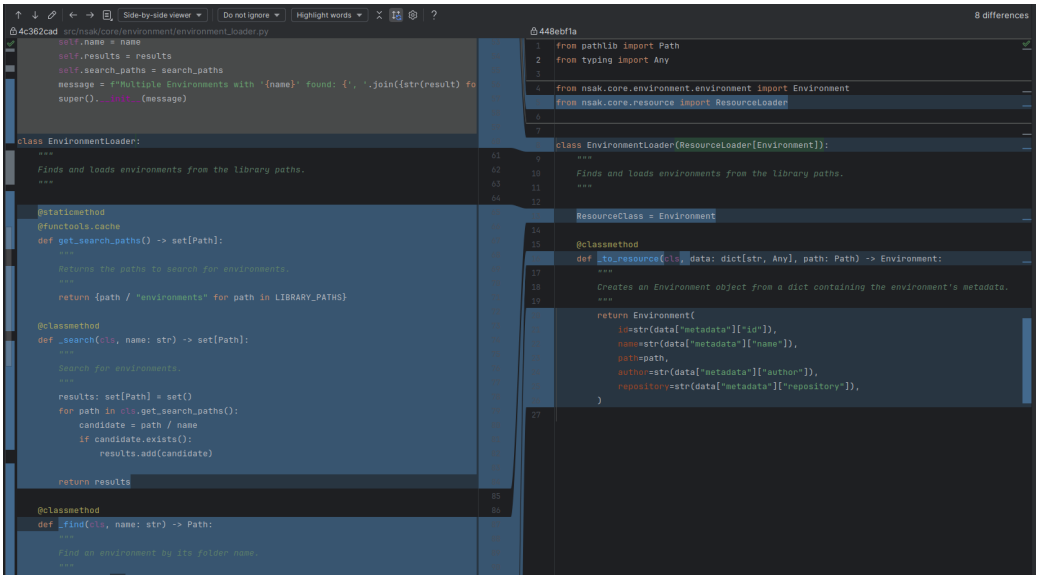


Figure 5.1.: EnvironmentLoader Base Class Refactoring Diff

The structure of all resources was aligned by nesting the resource under their respective resource type key (`scenarios`, `drills`, `environments`, and `devices`), so that we could merge all YAML resource files into a single library data structure without key conflicts. In the YAML specification 11 below, the new base structure derived from the device resource 5.1.2 is illustrated. All existing resource YAML files in the library were adjusted accordingly, and thanks to the introduction of the `ResourceLoader` base class, the required adjustments in the code were minimal.

```yaml
# Resource type version was removed, as it was not used anyway

# scenarios|drills|environments|devices
<str:resource-type-key>:
    # unique resource identifier
    <str:resource-id>:
        # The "metadata" key is removed and the children are now
        ↪  nested directly under the resource
        name: <str:name>
        description: <str:description>
        author: <str:email>
        repository: <str:link>
        # ... resource type specific properties, which
        ↪  previously were at the top level
```

Listing 11: Mergeable Resource YAML Specification

5.1.3. Scenario & Drill Configuration Management

Both the scenario and drill resource types were already fully implemented in "Project 2"1.2. Their YAML specifications already contained an `interface` section describing argument and return types, but this information was purely documentary and had no effect at runtime.

As part of this project, the `interface.arguments` section was extended 12 so that each argument carries an explicit `type` annotation and an optional `default` value.

```yaml
<str:resource-type-key>:
    <str:resource-id>:
        # Unchanged properties...

        interface:
            arguments:
                <str:argument-name>:
                    type: <str:type-annotation>
                    default: <any:default-value>
            return_type: <str:type-annotation>
```

Listing 12: Updated Interface Specification

The CLI reads these annotations at runtime to generate typed command options

for each drill and scenario automatically, without requiring any additional hand-written code. The `discover_hosts` drill in listing 13 illustrates this: its single `interface` argument of type `str` is translated directly into a `--interface` option on the generated CLI command.

```yaml
drills:
  discover_hosts:
    name: Discover Hosts
    author: vonal3@bfh.ch
    repository:
     ↪  https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffe

    dependencies:
      system:
        - arp-scan
        - nmap
      python: [ ]

    interface:
      arguments:
        interface:
          type: str
        subnet:
          type: str
          default: null
      return_type: NetworkDiscoveryResultMap
```

Listing 13: discover_hosts Drill YAML

Scenarios follow the same `interface` convention and additionally declare the drills and environments they depend on. The `mitm` scenario in listing 14 chains the `discover_hosts`, `arp_spoof`, and `transparent_tcp_proxy` drills together and exposes a single typed `interface` argument, which the CLI surfaces in the same way as for individual drills.

```yaml
scenarios:
  mitm:
    id: mitm
    name: MITM Scenario
    author: vonal3@bfh.ch
    repository:
     ↪  https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffe

    drills:
      - discover_hosts
      - arp_spoof
      - transparent_tcp_proxy

    environments:
      - simple_tcp_client_server

    dependencies:
      system:
        - arp-scan
        - dsniff
        - iptables
      python:
        - scapy

    interface:
      arguments:
        interface:
          type: str
      return_type: None
```

Listing 14: mitm Scenario YAML

5.2. Reproducible Test Environment

Unless stated otherwise, all referenced implementation artifacts in this section originate from the NSAK repository [3].

The scope of this thesis is confined to the AI based reconnaissance and optionally, the intrusion detection scenarios A.3.1, including the design and implementation of corresponding drills.

To ensure a controlled and safe evaluation of the implementation, a simulated test environment has to be established. This approach prevents unintended interference with production systems and ensures reproducibility during the development and testing phases.

For this exact purpose the "Environment"3.1.2 resource type was implemented into the NSAK framework. In the predecessor project 1.2, a simplified network topology comprising a client (Alice) and a server (Bob) is used to emulate and analyze a Man In The Mittle (MITM) scenario. This environment is described in a Docker Compose file, located under: `lib/environments/simple_tcp_client_server/compose.yaml`.

The whole setup can be simulated with the following command: `nsak environment simulate simple_tcp_client_server mitm`.

...

While Docker or Podman provides a general-purpose containerization platform, they abstract away most of the lower levels of the network stack and thus lack the support for modeling network topologies. In the existing implementation this was partially addressed, by using the macvlan network driver, to enable level 2 networking. In contrast, Containerlab builds upon Docker Compose and optimize its functionalities for networking purposes.

Maybe it would make sense to describe this in the project 2 section instead

The primary advantages of Containerlab are:

- It's ability to create a reproducible containerized network topology based on a simple configuration file
- Support multiple interfaces nodes which closely resemble real-world network devices (in comparison Docker typically operates with a single network interface).
- The rapid creation, extension and teardown of complete network environments, ensure that each experiment starts from a clean and controlled state.

In summary, while OCI containers serves as the underlying container engine, Containerlab provides a higher-level network simulation.

The following listing 15 shows an excerpt of a Containerlab topology.yaml file.

Basically the nodes are defined and with virtual Ethernet links connected. The bridge represents a multi port switch.

```
topology:
 nodes:
  node1:
  kind: linux
  image: alpine:3.19
  cap-add:
   - NET_ADMIN
  sysctls:
   net.ipv4.ipforward: 1

  node2:
   kind: linux

 links:
  - endpoints: [ "node1:eth1", "bridge:nd1" ]
```

Listing 15: Excerpt of the Containerlab topology definition

5.2.1. Network Topology

The network topology shown in Figure 5.2 illustrates the IP addressing scheme and the interconnections between the individual components.

The `external` node represents an external attacker interacting with the network from the Wide Area Networks (WAN).

The `nsak` container can be deployed in different network segments, enabling the simulation of various attacker positions and reconnaissance scenarios.

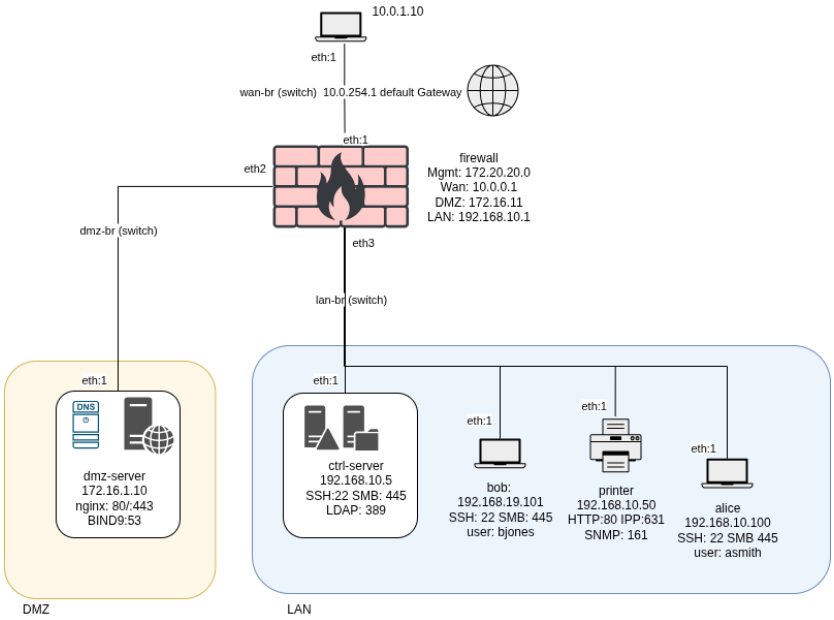


Figure 5.2.: topology-test-environment

5.2.2. DMZ-Server

To evaluate reconnaissance scenarios, common services must run on endpoints.

Berkley Internet Name Domain (BIND) is a complete implementation of the DNS protocol that can be configured through the `named.conf` file as an authoritative name server and resolver.

In the test lab illustrated in 5.2, the functionality of the web server and DNS server was combined for simplicity.

During the De-Militarized Zone (DMZ)-server container build process, a setup shell script is executed.

In the first part, the authoritative BIND `named.conf` file, DNS options, and master zones for forward and reverse lookups are created.

In the forward zone, domain names map to IP addresses, while in the reverse zone, IP addresses map to domain names 56.

add filepath and combine with other sections

```
; <<>> DiG 9.18.44 <<>> @172.16.1.10 vlab.local AXFR
; (1 server found)
;; global options: +cmd
vlab.local.             300     IN      SOA     ns1.vlab.local. hostmaster.vlab.local.vlab.local. 2024031501 3600 900 604800 300
vlab.local.             300     IN      NS      ns1.vlab.local.
alice.vlab.local.       300     IN      A       192.168.10.100
bob.vlab.local.         300     IN      A       192.168.10.101
ctrl.vlab.local.        300     IN      A       192.168.10.5
file.vlab.local.        300     IN      A       192.168.10.5
ns1.vlab.local.         300     IN      A       172.16.1.10
printer.vlab.local.     300     IN      A       192.168.10.50
web.vlab.local.         300     IN      A       172.16.1.10
vlab.local.             300     IN      SOA     ns1.vlab.local. hostmaster.vlab.local.vlab.local. 2024031501 3600 900 604800 300
;; Query time: 0 msec
;; SERVER: 172.16.1.10#53(172.16.1.10) (TCP)
;; WHEN: Sat Apr 04 13:34:18 UTC 2026
;; XFR size: 10 records (messages 1, bytes 333)
```

Figure 5.3.: Authoritative Zone Transfer Snippet

The output of a full zone transfer illustrated in figure 5.3 displays the resolved IP addresses and domain names. In our test environment, the restriction of Authoritative Zone Transfer (Full Zone Transfer) requests to only trustworthy IPs is intentionally omitted; the devices are deliberately left unhardened.

In the second part, a minimal `??` web server with HTTP and HTTPS support is created. Like in real setups, HTTP traffic on port 80 is redirected to HTTPS on port 443. To provide an artifact for reconnaissance purposes, a file named `{secret.txt} containing sensitive data is placed on the server.`

The LAN zone of the network, consists of 4 devices.

5.2.3. Domain Control and File Server

The internal server, reachable at 192.168.10.5, combines the role of domain controller and file server into a single node.

It exposes Secure Shell (SSH) on port 22, Server Message Block (SMB) on ports 139 and 445, and Lightweight Directory Access Protocol (LDAP) on port 389.

The LDAP service is intentionally misconfigured to permit anonymous read access to the entire `dc=lab,dc=local` directory tree, exposing user accounts and group memberships without authentication.

The SMB service provides three shares: `public` is readable without credentials, while `finance` and `it` require valid user accounts.

Sensitive files such as payroll exports and budget summaries are placed in the restricted shares as intentional post-exploitation artifacts 57.

5.2.4. Workstations

Two workstations reside on the LAN segment: Alice at 192.168.10.100, assigned to the Finance department, and Bob at 192.168.10.101, assigned to the IT department.

Both nodes expose SSH on port 22 and SMB on port 445, and both SSH banners intentionally disclose the organization name.

On Alice, the home directory of user `asmith` contains a confidential finance report and a drive mapping file that reveals the path to the central file share on the domain server.

On Bob, the home directory of user `bjones` contains a scripts directory with a host inventory that lists all internal IP addresses and their respective roles 58.

5.2.5. Printer

The printer service, reachable at 192.168.10.50, provides two different interfaces. The first web interface is administrative and is available on port 80. It shows internal printer metadata, such as device name and contact information. The printer version is intentionally disclosed in this admin view. The second interface, a REST API, listens on port 631 at the `{/jobs}` endpoint, where it displays a predefined list of print jobs. All printer requests are logged in `{/var/logs/printer_sim_logs}` inside the container 59.

5.2.6. Introduced Vulnerabilities

For reconnaissance purposes, several vulnerabilities and flaws were introduced, which are partly mentioned in the previous sections and are presented in detail in Listing 61. These flaws are necessary to compare the results of the larger and smaller models as well as the implemented vs the AI-enhanced scenarios.

5.3. AI –Scenarios

This section describes the design and implementation of the milestone AI Scenarios A.3.1, which is based on the user story map workshop AI Purple Team A.11.7. In the workshop we defined the features which such a scenario may provide and categorized them in “Must”, “Could” and “Should”. Because of time constraints, we set the focused only on the user stories categorized as “Must”. The main goal is to implement an AI reconnaissance scenario, which is described in the 5.3.8. But on the way it made sense to also create an intermediary scenario which can run custom prompts 5.3.7. Further, we already explored the AI based intrusion detection scenario 5.3.9 which should be seen as proof of concept, because the corresponding milestone and goals are categorized as “Should”.

5.3.1. AI Agent

An AI agent is an autonomous system that combines a large language model (LLM) with a set of tools it can invoke to accomplish a goal. Rather than just generating text, the agent follows a reasoning loop: it plans which tool to call, executes it, observes the result, and continues until the task is complete [**?**].

The project team chose LangChain as the agentic framework without comparing it with alternative solutions, for the following reasons:

- Free and open-source
- Python package with native integration into the project’s toolchain
- Suggested by various sources and has a good reputation in the community
- Extensive documentation and active community support

For the integration into the NSAK framework we decided to implement an `AiAgent` class as the central abstraction, which wraps a `BaseChatModel` together with a set of LangChain tools and delegates the reasoning loop to LangChain’s built-in `create_agent()`.

AI Configuration

Before any AI scenario can run, the runtime configuration must contain an `AiConfiguration` entry specifying which model and backend to use. Listing 16 shows the dataclass, which holds two fields: `model` and `base_url`. For authentication, the basic auth can be provided in the `base_url` field otherwise a provider specific environment variable like “ANTHROPIC_AUTH_TOKEN” must be used.

Fix authentication in the framework: It should have its own provider-agnostic config field

```python
from __future__ import annotations

import dataclasses


@dataclasses.dataclass(kw_only=True)
class AiConfiguration:
    """
    Configuration for the AI agent backend.
    """

    model: str
    base_url: str
```

Listing 16: AI Configuration Dataclass

The `model` field also encodes the model provider and follows the format `<provider>:<model-tag>` for example:

- `ollama:qwen2.5:7b-instruct` - a local Ollama instance
- `anthropic:claude-opus-4-5` - Anthropicś API
- `openai:gpt-4o` - OpenAIś API

Where currently only “ollama”, “anthropic” and “openai” are supported.

The corresponding CLI commands to configure the AI backend are:

- `nsak config set ai\.model "ollama:qwen2.5:7b-instruct"`
- `nsak config set ai\.base\_url "http://localhost:11434"`

Provider Abstraction

The `AiAgent.create_model()` method parses the provider prefix and dispatches to the appropriate LangChain chat model class via `PROVIDER_MAP`, as shown in listing 17. This means any supported provider can be swapped out by changing a single configuration value, without touching scenario code. All providers receive `temperature=0` to make agent behaviour as deterministic as possible.

```
PROVIDER_MAP: dict[str, Callable[[str, str, Any], BaseChatModel]] = {
    "ollama": lambda model, base_url, kwargs: ChatOllama(
        model=model, base_url=base_url, **kwargs
    ),
    "openai": lambda model, base_url, kwargs: ChatOpenAI(
        model=model, base_url=base_url, **kwargs
    ),
    "openwebui": lambda model, base_url, kwargs: ChatOpenAI(
        model=model, base_url=base_url, **kwargs
    ),
    "anthropic": lambda model, base_url, kwargs: ChatAnthropic(
        model_name=model, base_url=base_url, **kwargs
    ),
}

credentials = base64.b64encode(b"nsak:fec515d7-07bb-4a0a-b089-a0588465ccaf").decode()

mcp_client = MultiServerMCPClient(
    {
        "drawio": {
            "transport": "streamable_http",
            "url": "https://drawio.hiube.ch/mcp",
            "headers": {
                "Authorization": f"Basic {credentials}",
            },
```

Listing 17: AI Provider Map and MCP Client Initialisation

5.3.2. LangChain Concepts

LangChain provides a framework for developing applications powered by large language models (LLMs) and agent-based systems. It enables developers to combine modular components and external integrations, simplifying AI application design while maintaining flexibility as underlying technologies evolve [22].

The primary LangChain concepts used in the NSAK framework are:

- `BaseChatModel` - provider-agnostic interface for the underlying LLM.
- `BaseTool` - interface for callable tools that the agent can invoke.

- `create_agent()` - builds a LangGraph-based ReAct agent graph that wires the model and tools together.
- `messages` - a list of conversation messages (user, AI, tool) maintained across the agent's reasoning loop.

```python
import asyncio
import base64
import logging
from pprint import pformat
from typing import Any, AsyncGenerator, Callable, Sequence

from langchain.agents import create_agent
from langchain_anthropic.chat_models import ChatAnthropic
from langchain_core.language_models import BaseChatModel
from langchain_core.messages import AIMessage
from langchain_core.tools import BaseTool
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain_ollama.chat_models import ChatOllama
from langchain_openai.chat_models import ChatOpenAI

from nsak.core.ai.tools import (
    cli_tool,
    generate_drill_tools,
    host_configuration,
    human_interaction_hook,
    send_email,
)
from nsak.core.configuration import config

logger = logging.getLogger(__name__)

PROVIDER_MAP: dict[str, Callable[[str, str, Any], BaseChatModel]] = {
    "ollama": lambda model, base_url, kwargs: ChatOllama(
        model=model, base_url=base_url, **kwargs
    ),
    "openai": lambda model, base_url, kwargs: ChatOpenAI(
        model=model, base_url=base_url, **kwargs
    ),
    "openwebui": lambda model, base_url, kwargs: ChatOpenAI(
        model=model, base_url=base_url, **kwargs
    ),
    "anthropic": lambda model, base_url, kwargs: ChatAnthropic(
        model_name=model, base_url=base_url, **kwargs
    ),
}

credentials = base64.b64encode(b"nsak:fec515d7-07bb-4a0a-b089-a0588465ccaf").decode()

mcp_client = MultiServerMCPClient(
    {
        "drawio": {
            "transport": "streamable_http",
            "url": "https://drawio.hiube.ch/mcp",
            "headers": {
                "Authorization": f"Basic {credentials}",
            },
        }
```

The `AiAgent.__init__()` method calls `create_model()` to instantiate the provider-specific chat model and then passes it to `create_agent()` together with the tool list and a fixed system prompt that establishes the cybersecurity simulation context. The `run()` method invokes the agent once and streams all resulting messages (user, AI, tool calls, tool results) as formatted strings. The `run_interactive()` method extends this into an interactive loop, where after each agent response the operator is prompted for the next instruction, and the full conversation history is retained in the `messages` list until the operator types an empty line or `exit`.

Tool Calling Concepts

Tool calling is the mechanism by which the LLM declares that it wants to invoke an external function. The framework intercepts the model's output, parses the tool call, executes the corresponding Python function, MCP, or Skill, and appends the result to the message history before asking the model to continue. This loop repeats until the model produces a final text response with no tool calls.

In NSAK, tools can be provided through several complementary mechanisms:

- **LangChain Tools** - native Python functions decorated with `@tool`, which LangChain automatically wraps with type-safe JSON schemas for the model. This is the primary mechanism used for NSAK-specific tools (host configuration, CLI access, e-mail, human interaction hook). Further, all NSAK drills are wrapped as LangChain tools via `generate_drill_tools()`, giving the agent type-safe access to all drill capabilities.
- **MCP** - the Model Context Protocol allows external tool servers to expose additional capabilities; used here for the draw.io integration (see 5.3.2).
- **CLI Access** - the `cli_tool` gives the agent direct shell access, allowing it to run any installed binary (nmap, tshark, etc.).

Tool Calling Models

Considering its limited size of 7 billion parameters, we were quite happy with our first test model "qwen2.5:7b-coder". But as it turns out, not all models are capable of calling tools. Often, multiple variants of the same model are available, and they don't only differ in size, but also for different strengths or capabilities such as tool calling. The tool calling variants are often have the suffix "instruct". Luckily, "qwen2.5:7b-coder" has such variant called "qwen2.5:7b-instruct".

The `create_ai_agent()` factory function shown in listing 19 is the central assembly point used by all three AI scenarios. It reads the `ai` configuration block, assembles the tool list (always including `cli_tool`, `host_configuration`, and

`send_email`), optionally adds the human interaction hook or drill tools, loads MCP tools with graceful fallback, and returns a configured `AiAgent` instance.

```
                break

            messages.append({"role": "user", "content":
            ↪  next_instruction})


async def create_ai_agent(
    interactive: bool = False,
    enable_drill_tools: bool = False,
) -> AiAgent:
    """
    Creates an AI-agent from the runtime configuration.
    """
    if config.ai is None:
        message = "ai is not configured. Run: nsak config set
        ↪  ai.model <provider:model:tag> and nsak config set
        ↪  ai.base_url <url>"
        raise RuntimeError(message)

    tools: list[BaseTool | Callable[..., Any] | dict[str, Any]]
    ↪  = [
        cli_tool,
        host_configuration,
        send_email,
    ]

    if enable_drill_tools:
        tools.extend(generate_drill_tools())

    if interactive:
        tools.append(human_interaction_hook)

    # try:
    #     mcp_tools = await mcp_client.get_tools()
    #     tools.extend(mcp_tools)
    # except Exception as e:
    #     logger.warning("Failed to load MCP tools; continuing
    ↪  without them.", exc_info=e)

```

Listing 19: create_ai_agent() Factory Function

The `enable_drill_tools` flag, when set to `True`, calls `generate_drill_tools()`, which dynamically wraps every NSAK drill as a type-safe LangChain tool. This gives the agent access to structured capabilities such as `discover_hosts` and `port_scan` without requiring CLI invocation.

MCP Integration

The MCP [13] is an open protocol that standardises how AI models connect to external tool servers. Rather than implementing tool calling ad-hoc, MCP servers expose a well-defined interface that any compatible LLM framework can consume [**?**]. LangChain Agents must be invoked in an event loop to properly support MCP, this forced us to refactor the implementation "ai_agent.py" to be async. Further we need to add support for async drills and scenarios, as otherwise we could not run the AI agent inn those contexts.

NSAK integrates an external draw.io MCP server to allow the AI agent to create and manipulate diagrams during a scenario run. The client is initialised at module level (listing 17, lines 41–51 using `MultiServerMCPClient` from `langchain-mcp-adapters`, pointing to `https://drawio.hiube.ch/mcp` with Basic authentication.

MCP tools are loaded asynchronously inside `create_ai_agent()` and appended to the tool list. If the server is unavailable, the exception is caught and logged as a warning so the scenarios can still run without diagram support.

LangChain Tool: Host Configuration

The `host_configuration` tool shown in listing 20 gives the agent a structured view of the host's current configuration. It returns a dictionary containing the loaded device configuration with interface names, Internet Protocol addresses, and the `is_target` / `is_management` flags loaded from the device YAML. This bootstraps the agent's situational awareness without requiring it to run `ip addr` or parse raw network output - the agent knows from the start which interfaces are part of the attack surface and which are for management traffic.

```python
@tool  # type: ignore[misc]
def host_configuration() -> dict[str, Any]:
    """
    Returns the host network configuration as a dictionary.

    Use this tool first to understand the local network setup before running any scans or attacks.

    The returned dictionary has the following structure:
    {
        "debug": bool,
        "device": {
            "id": str,
            "name": str,
            "description": str,
            "configuration": {
                "network": {
                    "ethernets": {
                        "<interface_name>": {
                            "name": str,
                            "is_up": bool,        # False means the interface is DOWN, skip it
                            "is_target": bool,    # True means this interface is used for attacks/scans
                            "is_management": bool # True means this interface is used for device access
                            "addresses": {
                                "<cidr>": {
                                    "ip": str,        # e.g. "10.10.10.5/24" - use this as nmap source IP
                                    "is_target": bool,
                                    "is_management": bool,
                                }
                            },
                        }
                    }
                }
            }
        }
    }

    Guidance:
    - Use `is_target=True` interfaces/IPs when running nmap scans or attacks
    - Skip interfaces where `is_up=False`
    - Use `is_management=True` interfaces for device access or data extraction
    - `configuration` may be None if the device has not been configured yet
    """
```

LangChain Tool: CLI Tool

The `cli_tool` shown in listing 21 allows the AI agent to execute arbitrary shell commands with a configurable timeout (default 120 s). It returns a tuple of `(exit_code, stdout, stderr)`, giving the agent full feedback to diagnose failures and adapt its next action. The scope is intentionally broad: the agent can invoke nmap, tshark, apt, or any other binary available in the execution environment. Besides the execution of drills, this flexibility is what makes the AI scenarios powerful, but it is also why the kill switch described in 5.3.6 is a necessary safety measure.

```python
@tool  # type: ignore[misc]
def cli_tool(command: str, timeout: int = 120) -> tuple[int, str, str]:
    """
    Run an arbitrary CLI command and return its output.

    Use nmap for network scanning, e.g.:
    - "nmap -sV 10.10.10.1" (service version detection)
    - "nmap -sC -sV -oN output.txt 10.10.10.1" (default scripts + versions)
    - "nmap -p 1-1000 10.10.10.1" (port range)

    Use `apt install` if a cli tool is missing.

    Args:
        command: Full CLI command string to execute.
        timeout: Max seconds to wait (default 120, increase for large scans).

    Returns
    -------
        Tuple of (exit_code, stdout, stderr).
        Exit code 0 means success. Check stderr on failure.
    """
    try:
        completed_process = subprocess.run(  # noqa: S603
            shlex.split(command),
            capture_output=True,
            text=True,
            check=True,
            timeout=timeout,
        )
        return (
            completed_process.returncode,
            completed_process.stdout,
            completed_process.stderr,
        )
    except subprocess.TimeoutExpired as e:
        returncode = -1
        stdout = str(e.stdout) or ""
        stderr = f"Command timed out after {timeout}s"
        return (
            # Special return code
            returncode,
            stdout,
            stderr,
        )
    except subprocess.CalledProcessError as e:
        return (
            e.returncode,
            e.stdout,
            e.stderr,
        )
    except Exception as e:
```

5.3.3. Human Interaction Hook

LangChain Tool: Human Interaction Hook

Even tough LangChain has its own concept, called Human-in-the-loop, for now a simple python based tool was implemented as shown in 22, to achieve the same goal. When enabled via the `interactive` flag in `create_ai_agent()`, the agent can pause its execution and ask the operator a question, then incorporate the answer into its next reasoning step.

```python
@tool  # type: ignore[misc]
def human_interaction_hook(question: str) -> str:
    """
    Ask the human operator a question and return their answer.

    Use this tool when you need information, confirmation, or guidance that
    cannot be determined from the available tools or the current context.

    Args:
        question: The question or request to present to the human operator.

    Returns
    -------
        The human operator's response as a string.
    """
    return input(f"[Human Input Required]\n{question}\n> ")
```

Listing 22: LangChain Tool: Human Interaction Hook

5.3.4. Logging

Remote Logging System: Grafana / Loki

Uses the package python-logging-loki. We went for this instead of collecting the logs from stdout, because we can provide more metadata in form of “tags”.

A.12.10

```python
@dataclasses.dataclass(kw_only=True)
class LokiConfiguration:
    """
    Configuration for logging to loki.
    """

    url: str
    username: str
    password: str
```

Listing 23: Loki Logging Configuration

Add a figure

5.3.5. E-Mail Alerting

The python standard library already includes packages for creating and sending E-Mails via Simple Mail Transfer Protocol (SMTP). The "EmailBackend" implemented to the NSAK framework is an opinionated wrapper around those packages and leverages the previously introduced 5.1 to set up necessary configuration. If no SMTP configuration is provided as specified in 24, the system will write the E-Mails as logs instead. The working tool can be seen in actions in the test run of the AI Reconnaissance scenario 5.3.8, where the operator instructs the agent to send the results via E-Mail.

```python
@dataclasses.dataclass(kw_only=True)
class EmailConfiguration:
    """
    SMTP Configuration for sending emails.
    """

    host: str
    port: int
    encryption: EmailEncryptionType = EmailEncryptionType.NONE
    username: str
    password: str
    sender: str
    receiver: str
```

Listing 24: E-Mail Alerting Configuration

```
class EmailBackend:
    """
    Email Backend used for sending mails via SMTP.
    """

    backend: smtplib.SMTP | LoggerEmailBackend

    def __init__(self, email_config: EmailConfiguration | None)
    ↪   -> None:
        """
        Set up the SMTP client with the correct configuration
        ↪  for encryption.
        """
        self.config = email_config
        self._init_backend()
```

Listing 25: E-Mail Alerting Implementation

LangChain Tool: E-Mail Alerting

The `send_email` tool shown in listing 26 exposes the `EmailBackend` to the AI agent. During a scenario run the agent can autonomously send alert e-mails to operators - for example, when the AI intrusion detection scenario identifies a suspicious host. Subject lines are automatically prefixed with `[NSAK]` so that operators can filter them in their inbox. Optional `cc_recipients` and `attachments` parameters allow the agent to include additional stakeholders or attach captured evidence files.

```python
@tool  # type: ignore[misc]
def send_email(
    subject: str,
    message: str,
    cc_recipients: list[str] | None = None,
    attachments: list[str] | None = None,
) -> None:
    """
    Send an email notification to the configured receiver.

    Use this tool to alert the human operator about findings, incidents, or
    completed tasks that require their attention outside the current session.

    Args:
        subject: A short, descriptive subject line for the email, which is added to the subject prefix `NSAK - AI Agent:`.
        message: The full body of the email.
        cc_recipients: Optional comma seperated list of emails, which should receive a carbon copy of the email.
        attachments: Optional list of absolute file paths to attach to the email (e.g. scan reports, captured files).
    """
    subject = f"NSAK - AI Agent: {subject}"
    email_backend.send(subject, message, cc_recipients, attachments)
```

Listing 26: LangChain Tool: E-Mail Alerting

5.3.6. CLI Kill Switch

It's possible that an AI-Agent starts doing unexpected, unwanted or even destructive actions. This was also identified as a possible risk in the risk assessment A.7.8. This kind of misbehavior, combined with elevated permissions and tool calling capabilities via direct CLI access, could lead to a wide range of issues:

- The AI-Agent is stuck in a loop while using most system resources.
- The AI-Agent misconfigures the system, rendering it inaccessible or even

unusable.

- The AI-Agent burns too many API credits because of a bug, an inefficient request, or a huge task.
- The AI-Agent starts an attack against a production network.

For these reasons we want to be able to kill a specific or all scenarios as fast as possible. As a first measure, the project team decided to add this functionality as simple CLI commands:

- `nsak scenario kill <scenario-id>`: Kill a specific scenario (sends `SIGTERM` to the container)
- `nsak scenario killswitch`: Kill all running scenarios
- `nsak killswitch`: Alias for `nsak scenario killswitch`

5.3.7. Scenario: AI Agent

The AI Agent scenario is the most generic of the three: it accepts a free-form `prompt` argument and runs the AI agent in an interactive loop, maintaining the full conversation history across turns. This makes it the primary entry point for ad-hoc AI-assisted tasks where no dedicated scenario exists yet. The operator can guide the agent step by step, observe its reasoning, and issue follow-up instructions until they type an empty line or `exit`. Listing 27 shows the scenario entrypoint and listing 28 shows its YAML interface declaration.

```python
"""
Scenario entrypoint for AI based network reconnaissance.
"""
import logging

from nsak.core import create_ai_agent

logger = logging.getLogger(__name__)

async def run(prompt: str, interactive: bool = True) -> None:
    """
    Scenario, which runs an AI agent with a custom prompt in an interactive loop.

    After each agent response the operator is prompted for the next instruction.
    Enter an empty line or 'exit' to stop the session.

    :return: None
    """

    logger.info("Starting scenario: AI Agent")

    ai_agent = await create_ai_agent(interactive)
    result = ai_agent.run_interactive(prompt)

    async for line in result:
        logger.warning(line)
```

Listing 27: Scenario: AI Agent - scenario.py

```yaml
scenarios:
  ai_agent:
    id: ai_agent
    name: AI Agent (custom prompt)
    author: vonal3@bfh.ch
    repository:
     ↪  https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffe

    drills:

    environments:

    dependencies:
      system:
      python:

    interface:
      arguments:
        prompt:
          type: str
        interactive:
          type: bool
          default: True
      return_type: None
```

Listing 28: Scenario: AI Agent - scenario.yaml

Scenario: AI Agent - Test Run

```
nsak scenario execute ai_agent --prompt "Scan the target network and
report all findings" --interactive True
```

Describe and add a reference to the result of the first testrun

5.3.8. Scenario: AI Reconnaissance

The AI Reconnaissance scenario automates host and service discovery on a specified network interface. It encodes a structured five-step plan directly into the initial prompt: the agent is instructed to call `host_configuration` to determine the Internet Protocol addresses and subnets on the interface, then use the `cli` tool to invoke nmap for host scanning, then again for service enumeration, and finally to report all findings including vulnerability indicators. If the `interactive`

flag is set, the last step invites the operator to issue follow-up instructions via the `human_interaction_hook`.

This prompt-driven approach means the scenario does not contain any network scanning logic itself - the agent decides which nmap flags to use, how to interpret results, and what to scan next. Listing 29 shows the scenario entrypoint and listing 30 shows its YAML interface declaration.

```python
"""
Scenario entrypoint for AI based network reconnaissance.
"""
import logging

from nsak.core import create_ai_agent

logger = logging.getLogger(__name__)

async def run(interface: str, interactive: bool = False) -> None:
    """
    Scenario, which conducts AI-based network reconnaissance.

    :return: None
    """

    logger.info("Starting scenario: AI Reconnaissance")

    # Initial prompt
    prompt = f"""
Goal:
Discover all subnets, hosts and services on the following interface: {interface}

Steps:
1. Use the the `host_configuration` tool to get the IPs and subnets on the interface.
2. Use the `cli` tool to invoke commands like nmap to scan the subnets for available hosts.
3. Use the `cli` tool to invoke commands like nmap to scan the host for exposed services.
4. List all subnets, hosts and services and state if they are vulnerable.
5. Use the `human_interaction_hook` tool to allow the operator to execute followup steps.
    """

    # Run agent
    ai_agent = await create_ai_agent(interactive)
    result = ai_agent.run(prompt)

    # Logging
    async for line in result:
        logger.warning(line)
```


Listing 29: Scenario: AI Reconnaissance - scenario.py

```yaml
scenarios:
  ai_reconnaissance:
    id: ai_reconnaissance
    name: AI Reconnaissance
    author: vonal3@bfh.ch
    repository:
     ↪  https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffe

    drills:

    environments:
      - simple_tcp_client_server

    dependencies:
      system:
      python:

    interface:
      arguments:
        interface:
          type: str
        interactive:
          type: bool
          default: False
      return_type: None
```

Listing 30: Scenario: AI Reconnaissance - scenario.yaml

Scenario: AI Reconnaissance - Test Run

```
nsak scenario execute ai_reconnaissance --interface eth0
```

Describe and add a reference to the result of the first testrun

5.3.9. Scenario: AI Intrusion Detection

The AI Intrusion Detection scenario is a proof of concept that should be seen as a “Should” feature. It demonstrates a two-phase approach: first it performs host discovery using `DrillManager.execute("discover_hosts")`, then it captures live network traffic using `tshark` (a system dependency declared in the scenario YAML). Up to 200 packets are captured over a 60-second window and extracted as ?? fields (frame number, source and destination Media Access Control, source

and destination Internet Protocol, Transmission Control Protocol/User Datagram Protocol (UDP) destination port, protocol stack, and frame length).

Both the host discovery table and the packet ?? are injected into a structured `PROMPT_TEMPLATE` that instructs the AI to identify at most five suspicious events, each with source/destination Internet Protocol and Media Access Control, protocol, and the reason for suspicion, and to identify any malicious hosts. By constraining the number of events the prompt keeps the agent's output concise and actionable. Listing 31 shows the full scenario entrypoint and listing 32 shows its YAML interface declaration.

```
"""
Scenario entrypoint for AI based intrusion detection.
"""
import logging
import subprocess

from nsak.core import create_ai_agent, DrillManager
from nsak.core.network import NetworkDiscoveryResultMap

logger = logging.getLogger()

PACKET_COUNT = 200
CAPTURE_DURATION = 60  # seconds
PROMPT_TEMPLATE = """
You are analyzing network traffic for intrusion detection on interface %(interface)s.

Network Discovery Result Map:
%(host_discovery_result_map)s

Captured packets (CSV):
%(csv_summary)s

Identify at most 5 suspicious events if any. For each, output:
- src/dst IP and MAC
- protocol
- reason it is suspicious

Identify malicious hosts (MAC and/or IP) if any.

Be concise. Do not repeat packet data.
"""

async def run(interface: str, interactive: bool = False) -> None:
    """
    Scenario, which listens to network traffic and tries to detect intruders.

    :return: None
    """
    logger.info("Starting scenario: AI Intrusion Detection")

    host_discovery_result_map: NetworkDiscoveryResultMap = DrillManager.execute("discover_hosts", interface=interface)

    # Capture traffic and extract fields including MAC addresses
    result = subprocess.run(
        [
            "tshark",
            "-i", interface,
            "-c", str(PACKET_COUNT),
            "-a", f"duration:{CAPTURE_DURATION}"
```

```yaml
scenarios:
  ai_intrusion_detection:
    id: ai_intrusion_detection
    name: AI Intrusion Detection
    author: vonal3@bfh.ch
    repository:
     ↪  https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffe

    drills:

    environments:
      - simple_tcp_client_server

    dependencies:
      system: [ tshark ]
      python:

    interface:
      arguments:
        interface:
          type: str
        interactive:
          type: bool
          default: False
      return_type: None
```

Listing 32: Scenario: AI Intrusion Detection - scenario.yaml

Scenario: AI Intrusion Detection - Test Run

```
nsak scenario execute ai_intrusion_detection --interface eth0
```

Describe and add a reference to the result of the first testrun

5.3.10. LangChain Agent Chat UI

We did not add the LangChain Agent Chat UI because we made some implicit architecture decisions which prevent easy integration.

If we decide to implement it regardless, we would need to redesign and refactor the AI Agent implementation, for example by switching to a callback-based output model. A useful reference for such a migration is available at this LangChain UI tutorial.

5.4. Red Team: Reconnaissance Scenario

The reconnaissance scenario is an engineered red team scenario that systematically maps an unknown network. It chains four drills in sequence: host discovery, port scanning, and service enumeration. If no interface is specified by the operator, the scenario first discovers all active physical interfaces and, if any lacks an IP address, requests one via Dynamic Host Configuration Protocol (DHCP) before scanning. The scenario results serve as the reproducible baseline for comparison with the AI-driven reconnaissance approach. The full scenario implementation is provided in Listing **??**.

5.4.1. Goal

The goal of the scenario is to identify all reachable hosts, their open ports and running services in a target network without prior knowledge of the topology. The drill chain is designed to work generically across different network environments: the operator can either provide a specific interface and subnet or let the scenario discover the available interfaces automatically.

5.4.2. Discover Hosts Drill

The *Discover Hosts* drill is the first active step of the scenario. For local network segments, it runs `arp-scan --localnet` on the given interface, which sends Address Resolution Protocol requests and collects the Internet Protocol address, Media Access Control address, and vendor of each responding host. If an explicit subnet is provided, the drill falls back to `nmap -sn` instead, which uses Internet Control Message Protocol (ICMP) and Transmission Control Protocol probes and therefore works across Layer 3 boundaries. Media Access Control addresses are only available for hosts on the same Layer 2 segment; remote hosts are recorded without a Media Access Control. The full implementation is provided in Listing **??**.

Drill Interface

The drill exposes two arguments: a required `interface` and an optional `subnet`. If `subnet` is omitted, the drill defaults to a local ARP scan; if provided, it switches to the nmap-based subnet scan. The resource configuration is shown in Listing **??**.

5.4.3. Enumeration

After host discovery, the scenario runs two further drills that progressively deepen the collected information.

Port Scan

The *Port Scan* drill receives the `NetworkDiscoveryResultMap` produced by host discovery and runs `nmap -sV --open` against each discovered Internet Protocol. For every open port, the drill records the port number, transport protocol (Transmission Control Protocol/UDP), service name, and detected version string and appends a `NetworkService` entry to the existing result map. The full implementation is provided in Listing **??**.

Service Enumeration

The *Enumerate Services* drill takes the enriched result map and runs targeted Nmap Scripting Engine (NSE) scripts for each discovered service. Well-known services such as HTTP, DNS, FTP, SMTP, SMB, and LDAP are mapped to a specific set of scripts (e.g. `http-title`, `dns-zone-transfer`, `smb-security-mode`). Unknown services fall back to the generic `banner` script, which captures the initial response of any Transmission Control Protocol service. The drill returns a mapping of `ip:port` keys to lists of finding strings extracted from the NSE output. The full implementation is provided in Listing **??**.

5.4.4. Output Artifacts

The primary output of the scenario is the `NetworkDiscoveryResultMap`, which aggregates all results per interface. Each entry maps an interface name to a `NetworkDiscoveryResult` containing the list of discovered `NetworkService` objects with their endpoints (IP, MAC, port, protocol, version). After port scanning, the scenario prints the result as a human-readable table. The service enumeration findings are printed separately as a structured list grouped by `ip:port`.

6. Evaluation

Aspect	Traditional Recon	AI Recon
File	lib/scenarios/reconnaissance/scenario.py	lib/scenarios/ai_reconnaissance/scenario.py
Return value	NetworkDiscoveryResultMap (structured)	None (LangChain stream only)
Tools	arp-scan, nmap (Drills)	LangChain tool_calls (cli, host_configuration)
Async	sync	async
Determinism	deterministic (fixed drill order in code)	non-deterministic (LLM selects tools autonomously)
Error handling	explicit in code (e.g. DHCP fallback, interface discovery)	implicit at model level (may skip steps or hallucinate)

Table 6.1.: Comparison between Traditional Recon and AI Recon

Limitation: Traditional Recon returns a structured `NetworkDiscoveryResultMap`, whereas AI Recon returns `None` (log stream only). A direct comparison of results therefore requires either parsing the AI output or documenting this asymmetry as a limitation.

Qualitative Criteria

- **Speed:** Time from scenario start to complete output of all discovered hosts and services
- **Correctness:** Share of correctly identified hosts and services compared to the ground truth
- **Initial knowledge (pre-conditions):** Which assumptions does the scenario require (interface, subnet)?
- **Creativity:** Does AI Recon identify relationships or suggest follow-up steps not covered by Traditional Recon?
- **Token usage:** Number of tokens consumed per run (AI Recon only)

- **Hallucination:** Hosts or services in the output that do not exist in the environment
- **Task completion:** Do both scenarios complete the task fully?

Quantitative Criteria

- **Consistency:** How much does the output vary across $n = 10$ runs with an identical prompt, per model and environment?
- **Prompt sensitivity:** How does the model respond to minimally altered prompts (e.g. a different interface name)?

With 3 models $\times$ 3 environments $\times$ $n = 10$ runs, this yields 90 measurements for AI Recon.

Environments for Data Collection

1. Virtual environment (containerlab)
2. Physical environment
3. BFH lab

Models Under Comparison

1. Self-hosted local LLM (Ollama)
2. BFH LLM
3. Frontier model (Anthropic Claude)

7. Discussion

8. Conclusion

Bibliography

[1] World Economic Forum. The global risks report 2026, 2026.

[2] CrowdStrike. Global threat report 2025, 2025.

[3] Frank Gauss and Lukas von Allmen. Nsak (network swiss army knife). https://github.com/nsak-team/nsak, 2026. Accessed: 2026-01-10.

[4] Polina Zilberman, Rami Puzis, Sunders Bruskin, Shai Shwarz, and Yuval Elovici. Sok: A survey of open-source threat emulators. arXiv preprint, 2020.

[5] Blake E. Strom, Andy Applebaum, Doug Miller, Adam Nickels, Cody Pennington, and Chris Thomas. Mitre att&ck: Design and philosophy. Technical report, MITRE Corporation, 2020.

[6] Bader Al-Sada, Alireza Sadighian, and Gabriele Oligeri. Mitre att&ck: State of the art and way forward. *ACM Computing Surveys*, 2023.

[7] MITRE Corporation. Mitre caldera documentation, 2024.

[8] Red Canary. Atomic red team, 2024.

[9] Rapid7. Metasploit framework documentation, 2024.

[10] David Kennedy, Jim O'Gorman, Devon Kearns, and Mati Aharoni. *Metasploit: The Penetration Tester's Guide*. No Starch Press, 2011.

[11] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2):1–124, 2023.

[12] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.

[13] Anthropic. Model context protocol. https://www.anthropic.com/news/model-context-protocol, 2024. Accessed: 2026-03-18.

[14] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers, 2025.

[15] K Huang. Define ai agent. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.

[16] J Huang K Huang. Define ai agent. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.

[17] K Jackson C Hughes J Huang, K Huang. Ai agent safety and security considerations. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.

[18] Risky Business Feature Podcast. ...mcp is dead. https://risky.biz/, 2026. Podcast episode , Accessed: 2026-03-18.

[19] Ethan Michalak and Mark Perry. Connecting llms to caldera with the mcp plugin. https://medium.com/@mitrecaldera/connecting-llms-to-caldera-with-the-mcp-plugin-da1450254827, 2025. MITRE Caldera Blog, Accessed: 2026-03-16.

[20] GH05TCREW. Metasploit mcp server. https://github.com/GH05TCREW/MetasploitMCP, 2025. Accessed: 2026-03-16.

[21] Hare Sudhan. Atomic red team mcp server. https://github.com/cyberbuff/atomic-red-team-mcp, 2025. Accessed: 2026-03-16.

[22] LangChain AI. Langchain: The agent engineering platform. https://github.com/langchain-ai/langchain, 2026. Accessed: 2026-05-02.

[23] Advisera. Iso 27001 risk assessment, treatment, and management: The complete guide, 2025. Accessed: 2026-03-04.

[24] Simplr. General recommended vram guidelines for llms, 2025. Online; accessed 10 March 2026.

List of Figures

List of Tables

List of Listings

Glossary

This document is incomplete. The external file associated with the glossary 'main' (which should be called `thesis.gls`) hasn't been created.

Check the contents of the file `thesis.glo`. If it's empty, that means you haven't indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can't be generated. If the file isn't empty, the document build process hasn't been completed.

If you don't want this glossary, add `nomain` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[nomain]{glossaries-extra}
```

Try one of the following:

- Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:

  ```
  \usepackage[automake]{glossaries-extra}
  ```

- Run the external (Lua) application:

  ```
  makeglossaries-lite.lua "thesis"
  ```

- Run the external (Perl) application:

  ```
  makeglossaries "thesis"
  ```

Then rerun LaTeX on this document.

This message will be removed once the problem has been fixed.

A. Project Management

A.1. Stakeholders

Hj. Wenger: Instructor and Network Specialist
Urs Keller: Expert and Entrepreneur

A.2. Agile Management Process

The project followed an interactive development approach using Gitlab for issue tracking and planning. We defined our project management in a Scrum-like manner, but we individualized the process according to the projects scope and our needs. Work items where organized into epics, milestones and issues. Each iteration lasts two weeks. We decided to hold the following Scrum ceremonies after each sprint:

Sprint Planning	**Sprint Review**	**Sprint Retro**	**Checkpoint**
Select milestone with sprint goal	Close open tickets	Reflect on sprint	Present sprint results
Check issues for DoR and obstacles	Gather team feedback	Identify improvements	Get stakeholder feedback
Ensure issues are clear	Validate delivered features	Discuss what went well	Define next tasks
Create sprint backlog	Review progress	Discuss issues/problems	Discuss issues/problems

Table A.1.: Key Scrum Meetings

The workflow used for ticket tracking consisted of the following stages:

A.2.1. Issue Lifecycle:

Buckets	Description
Backlog	List of all tasks and features planned for the project.
Todo	Tasks selected for the current sprint that are ready to be worked on.
Q&A	Tasks waiting for clarification, review, or testing.
Done	Completed tasks that meet the defined acceptance criteria.

Table A.2.: Issue Board

A.2.2. Definition of Ready (DoR)

Category	Criteria
Clarity	▶ Clear and precise title ▶ Objective or problem described in two to three sentences ▶ Scope clearly defined
Scope	▶ Affected module specified (drill, scenario, core, container, frontend) ▶ External dependencies identified
Validation	▶ Testable acceptance criteria defined ▶ Expected behavior clearly specified

Table A.3.: Definition of Ready Criteria for Tickets

A.2.3. Definition of Done (DoD)

Category	Criteria
Implementation	▶ All acceptance criteria fulfilled ▶ Container execution verified ▶ Pre-commit hooks pass ▶ Logging and error handling consistent
Testing	▶ Feature or bug fix tested ▶ Acceptance criteria verifiable (automated or manual) ▶ Relevant edge cases considered ▶ Execution reproducible
Documentation	▶ README updated if required ▶ Architecture changes documented ▶ Thesis relevance briefly noted ▶ Limitations documented ▶ Required documentation stored for thesis or appendix

Table A.4.: Definition of Done Criteria for Tickets

A.3. Project Structure

This section describes the projects structure and list the goals as milestones linked to their respective page in GitLab.

A.3.1. Milestones

A milestone represent crucial steps toward the overall project goal. Per our definition a milestone has at least one epic and the mandatory milestones are listed in TableA.5

GitLab Milestones
Project Management
Research AI Integration
Research: Compare Frameworks
NSAK Framework: Configuration Management
NSAK Scenario: Network Discovery - Red Team
NSAK Scenario: AI Scenarios
Reproducible Test Environment

Table A.5.: Project Milestone Overview in GitLab

Project Planning

19.02.2026 – 10.03.2026

This is the initial milestone which is used to plan the whole project and bachelor thesis.

- ✓ The bachelor thesis is planned as an agile (scrum-like) project
- ✓ A risk analysis for the project was conducted and the identified risks are assessed and addressed
- ✓ All meetings, deadlines, scrum ceremonies and sprints are added to the Outlook calendar
- ✓ The goals of the thesis are defined and categorized as “must”, “should” and “could”
- ✓ The goals are created as milestones and divided into epics, categorized by the labels: Project Management, Research, Framework, Implementations, Documentation
- ✓ We know how we coordinate the project with the stakeholders (instructor and expert)

Research: AI Integration

10.03.2026 – 22.03.2026

The goal of this research milestone was to investigate how AI can support automated security testing and adversary emulation. The research evaluated potential integration points where AI could enhance scenario execution, automation,

decision-making, and analysis capabilities of the NSAK framework. The milestone was extended by three days following input from the expert and instructor to clarify the intended interaction between LLMs and NSAK.

- ☐ How is AI currently used in cybersecurity automation?
- ☐ Which tasks in adversary emulation can be supported or automated by AI?
- ☐ How could AI interact with the NSAK scenarios and drills (e.g. Agents, MCP)?
- ☐ What architectural changes would be required to integrate AI into NSAK?

Research: Compare Frameworks

10.03.2026 – 19.03.2026

This milestone focused on the research and comparison of existing security automation frameworks that could potentially support or influence the NSAK scenario and drill execution model. The evaluation included frameworks such as MITRE CALDERA and Atomic Red Team, examining their execution models, configuration and parameter handling, extensibility mechanisms, and the structure of reusable test definitions.

The expected outcomes were:

- ☐ A short comparison of selected frameworks
- ☐ A summary of relevant architectural concepts
- ☐ An initial assessment of how these frameworks relate to NSAK

NSAK Framework: Configuration Management

10.03.2026 – 19.03.2026

This milestone defined the configuration model and execution framework for NSAK scenarios. It introduced a structured approach for configuring scenarios and drills, including typed argument declarations, and the generation of a resolved run configuration. The goal was to provide a flexible and reproducible configuration system that allows scenarios and drills to expose parameters and execute in a defined order.

- ✓ Drills and scenarios can expose runtime parameters as configuration objects
- ✓ The configuration objects can be set via CLI arguments
- ✓ A configuration management system is implemented in the NSAK framework

NSAK Scenario: Network Discovery – Red Team

02.04.2026 – 16.04.2026

Network discovery is one of the most essential Red Team activities in the reconnaissance phase. This milestone covered the implementation of a scenario that identifies network participants and services, producing a structured network map that can be consumed by subsequent drills such as Address Resolution Protocol spoofing.

Goals (Must):

- ☐ The scenario identifies the available ports and interfaces on the device
- ☐ The scenario discovers the available subnets and obtains valid addresses
- ☐ The scenario discovers addresses, addresses and services (port scan)
- ☐ The scenario supports at least a fast full-search mode
- ☐ The scenario returns a network map as a structured data type

Goals (Could):

- ☐ A human interaction hook allows the operator to select the discovery mode
- ☐ Additional modes: slow full search, distinct search

NSAK Scenario: AI Scenarios

19.03.2026 – 16.04.2026

This milestone explored what happens when an AI agent is given full access to the NSAK framework for both Red and Blue Team activities. The scenario is intentionally open-ended, reflecting the research nature of the milestone.

Goals (Must):

- ☐ The operator can provide a custom prompt or select a predefined Red or Blue Team prompt
- ☐ The scenario runs an AI agent with a remote connection
- ☐ An operator can interact with the agent via CLI at all times
- ☐ The scenario logs all relevant actions and network traffic locally
- ☐ The AI can trigger an e-mail alert to notify an operator
- ☐ The scenario has a kill switch to stop execution at any time

Goals (Should):

- ☐ Human interaction hook via Web GUI
- ☐ Remote logging and sniffing
- ☐ Kill switch with full device shutoff

Reproducible Test Environment

19.03.2026 – 02.04.2026

The goal of this milestone was to design and implement a reproducible and safe test environment for the NSAK framework, providing a stable basis for consistent execution and evaluation of both manual and AI-assisted scenarios.

- ☐ The test environment can be set up reproducibly using Infrastructure as Code (e.g. Ansible).
- ☐ NSAK scenarios and drills can be executed consistently within the environment.
- ☐ The environment supports both manual and AI-assisted scenarios.
- ☐ Scenario results can be collected and analyzed for evaluation purposes.

A.3.2. Epics

consist of multiple stories and contribute to fulfilling milestones.

A.3.3. Issues

Each Story is assigned a weight, sprint, milestone and epic and are usually represented by an “Issue” in gitlab.

A.4. Sprints / Iterations

A.4.1. Planning

“Task” and “Issue”, the terms are often used interchangeably with “Stories”. Stories must meet the A.2.2 before implementation and A.2.3 before closing.

A.5. Planning and Estimation

The following estimation strategy was used:

- 1 hour corresponds roughly to 1 weight point
- Iteration length: 2 weeks
- Weights per iteration: 40h per person = 80 weight per iteration
- Sprint planning duration: 1.5 hours per iteration
- Sprint review duration: 1 h per iteration
- Sprint retro duration: 1 h per iteration

A.6. Project Roadmap

To keep track of the overall progress in the project, the project team uses the "Roadmap" feature in GitLab. After each sprint planning meeting we create a new screenshot of the roadmap and add it to the according sprint section in A.7. The first sprint planning is an exception, because at this point the diagram would have been empty.

A.7. Sprint Ceremonies

A.7.1. Sprint 1: 19.02 –10.03.2026

Sprint Planning 1: 19.02.2026

For the planning of sprint 1 we derived all issues from the milestone project management A.5 and had an overall weight from 89.

Sprint Review 1: 10.03.2026

The milestone was successfully reached, and the board was cleared in preparation for the next sprint. All issues were reviewed and moved to the “Done” column.

Milestone Review: Project Management As shown in burndown and burnup charts A.1 of the Project Planning A.3.1 milestone, all deliverables were fulfilled. However, the deadlines of the milestone had to be moved to the 10.03.2026, because the project planning required more time than initially expected. It should also be noted that the charts abruptly go down or up respectively, as we closed alot of the tickets after we reviewd them together just before we held this meeting.

Figure A.1.: Checklist and Burndown Sprint1

Sprint Retro 1: 10.03.2026

This sprint retro was held in Microsoft Whiteboard as it provides simple templates, the result is illustrated in A.2.

Key Takeaways:

- Maintain clear and consistent communication with the instructor.
- Be aware of maintaining a healthy work–life balance during the project.
- Apply lean project management practices to keep the process efficient and focused.

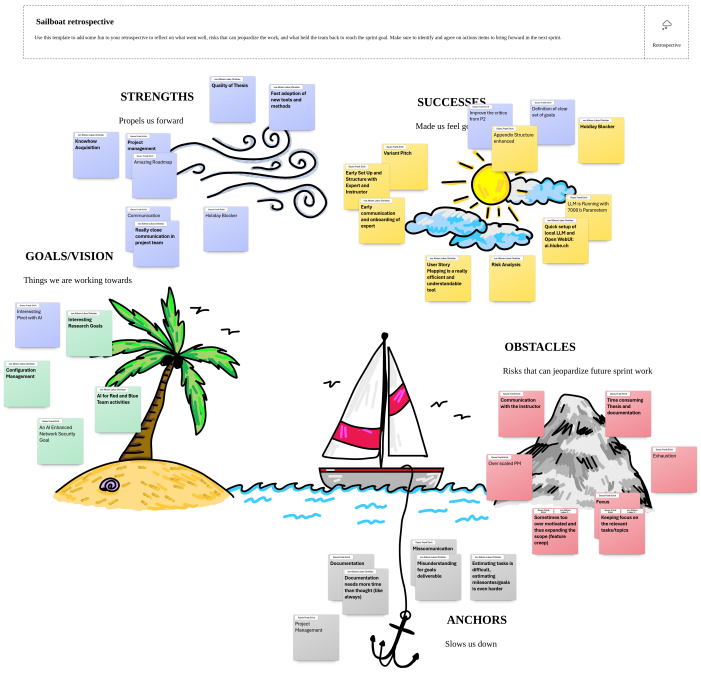


Figure A.2.: Sprint Retro 1

A.7.2. Sprint 2: 05.03 –18.03.2026

Sprint Planning 2: 05.03.2026

The current state of the project roadmap is shown in the following screenshot A.3.

Week	Responsible	Task
Week 1	Lukas	Configuration management implementation
Week 1	Frank	Research and comparison of frameworks
Week 1	Team	AI workshop on Thursday and writing of the related issues
Week 2	Team	AI research and evaluation
Week 2	Team	Preparation and pitch of the project deliverables

Table A.6.: Sprint Planning 2

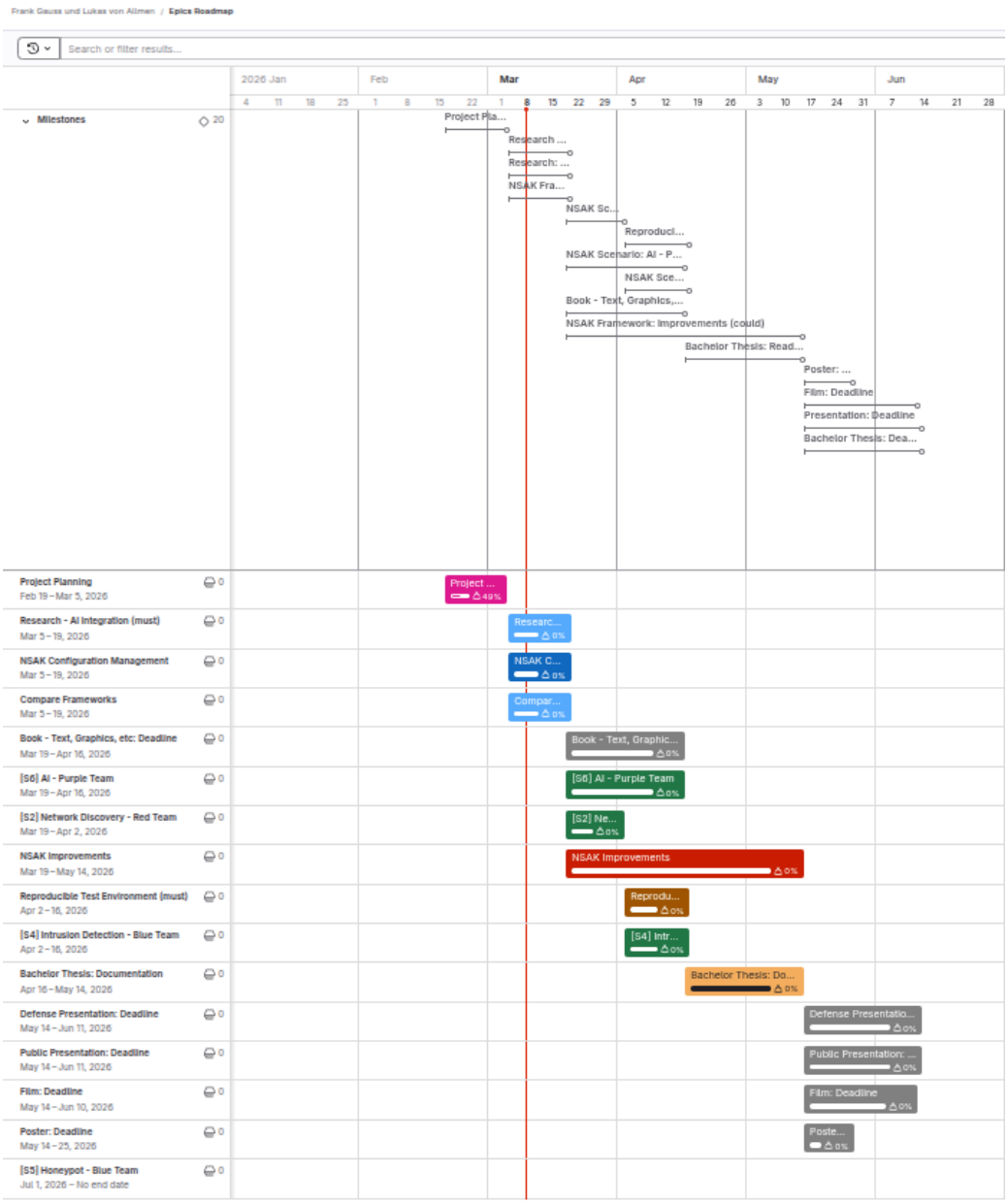


Figure A.3.: Roadmap Sprint Planning 2

Sprint Review 2: 19.03.2026

We got some helpful inputs related to both milestones from the tutor and expert in the checkpoint meeting A.8.3 earlier this day. We decided to still consider the milestone to be successfully reached as the planned work package and deliverables were finalized and presented. The resulting follow-up tasks are planned for

the next week and as a consequence we will hold the next sprint planning A.7.3 a week later than planned. All issues were reviewed and moved to the “Done” column in preparation for the next sprint. It must be noted, that because we usually review and close the tickets at the end of the sprint, the charts are not representative in most cases.

Milestone Review: Compare Frameworks

The Compare Frameworks A.3.1 milestone is completed as illustrated in A.4.

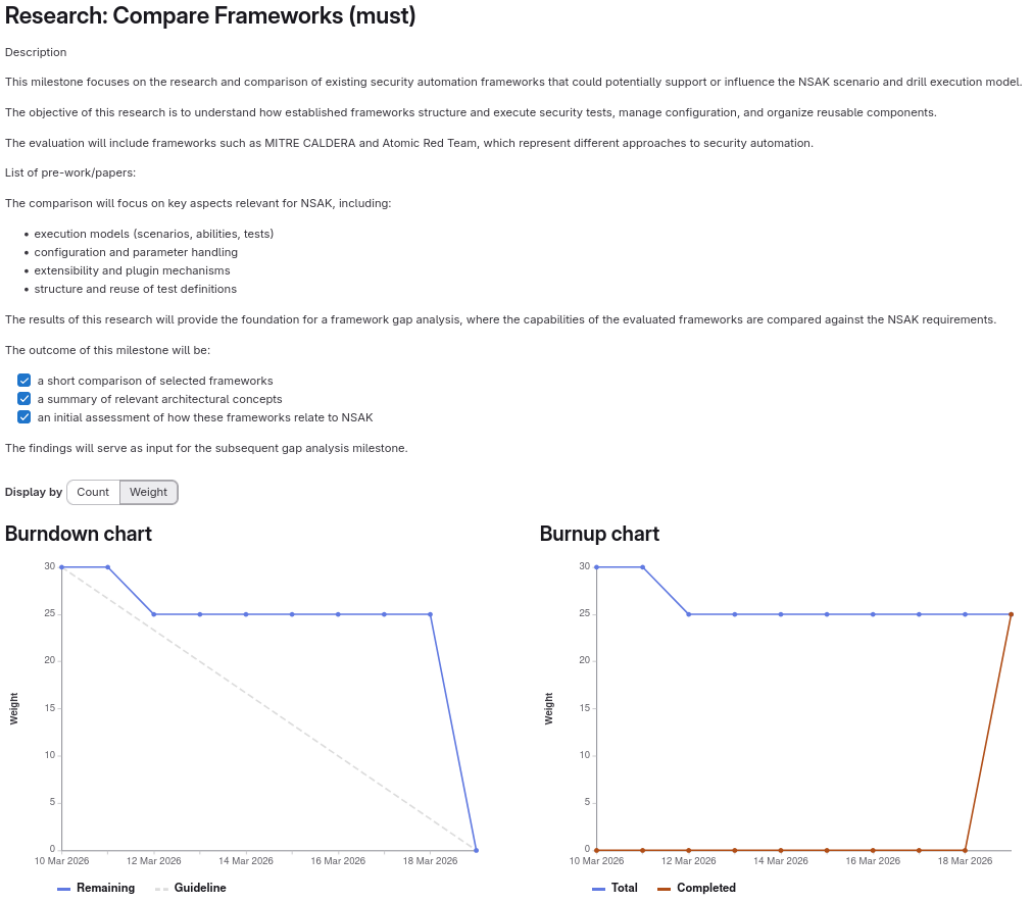


Figure A.4.: Milestone Description and Charts: Compare Frameworks

Milestone Review: Research AI-Integration

As shown in A.5, the milestone Research AI-Integration A.3.1 is completed.

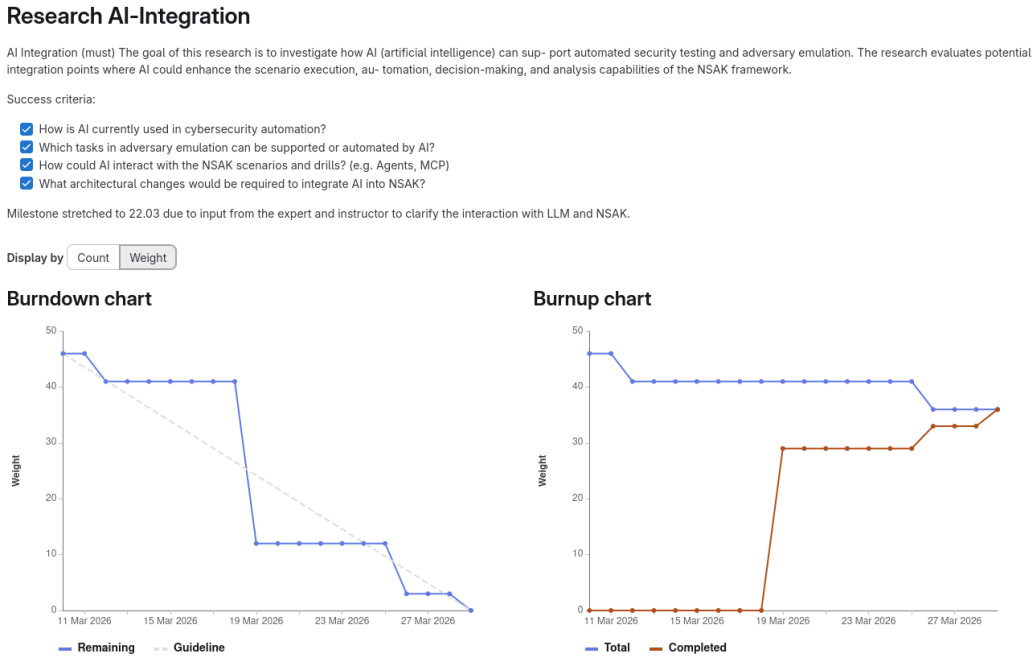


Figure A.5.: Milestone Description and Charts: Research AI-Integration

Milestone Review: Configuration Management The burnup and burndown charts below A.6 are showing that the milestone Configuration Management A.3.1 is implemented as planned.

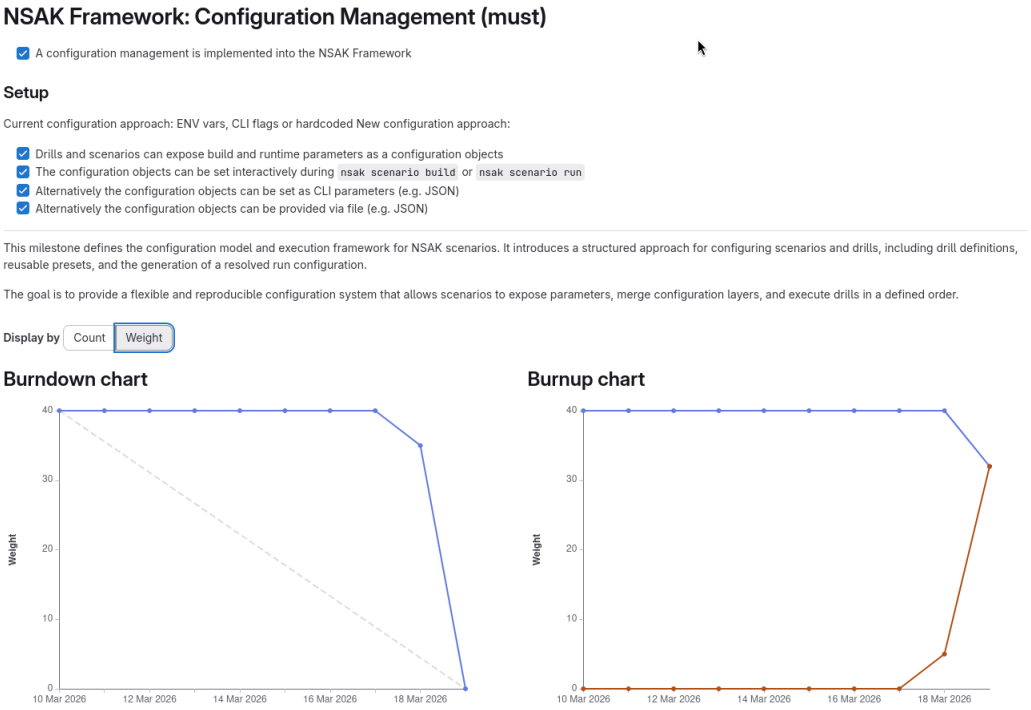


Figure A.6.: Milestone Description and Charts: Configuration Management

Sprint Retro 2: 19.03.2026

This sprint retro was held in Microsoft Whiteboard as it provides simple templates, the result is illustrated in A.7.

Key Takeaways:

- The long project planning phase greatly helped us with the projects overview and minimizes the planning needed for each sprint.
- The “focus weeks” really boosted the projects progress, and we are looking forward to the next “focus weeks”.
- Because we did not review tasks (Quality Assurance (QA)) on a regular basis, all this work needed to be done at the end of an iteration.
- Reviewing on a more regular basis would also lead to more accurate burnup and burndown charts.
- We decided to not include the charts in the future, because it makes no sense to adjust our process only for the sake of good-looking charts.

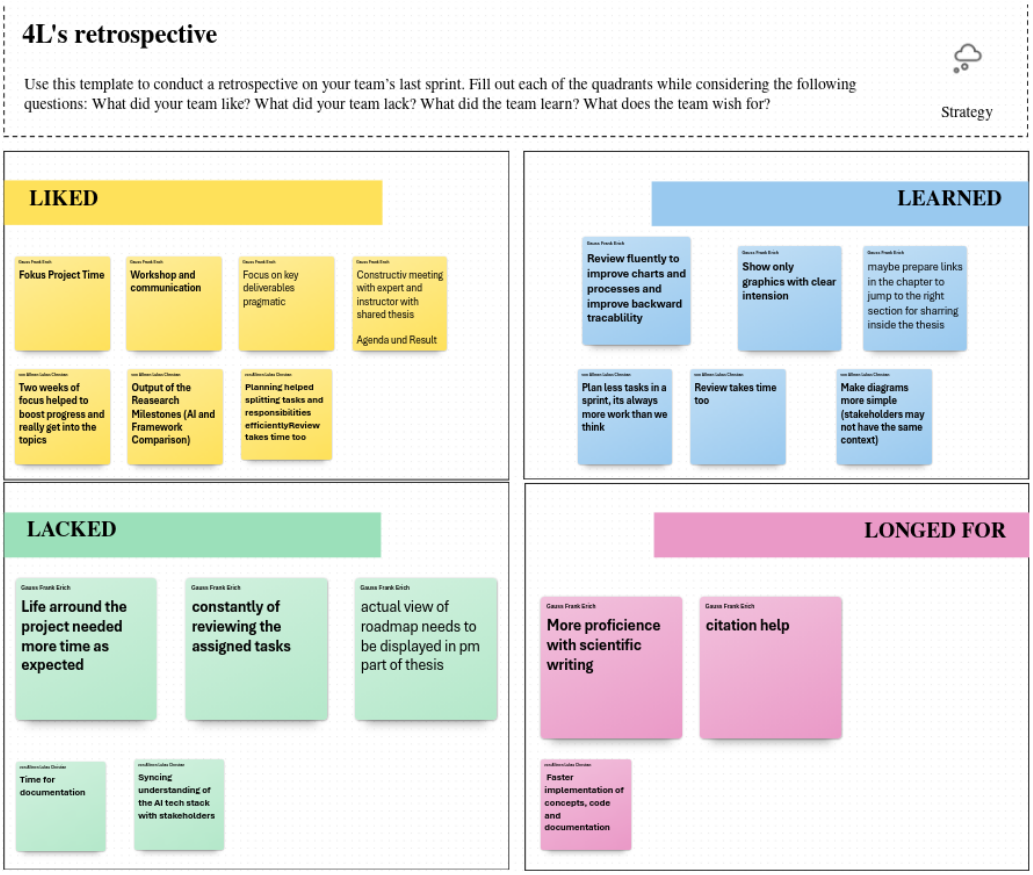


Figure A.7.: Sprint Retro 2

A.7.3. Sprint 3: 26.03 –09.04.2026

Sprint Planning 3: 26.03.2026

Because we got some important input regarding variant decision for AI-integration, framework comparison, and configuration management A.18 which led to additional work. As consequence of this, we decided to shorten the third sprint to one week, which is reflected in the following table A.7 and postponed the sprint planning from the 19.03 to the 26.03. To account for the delayed start in the overall planning we adjusted the roadmap as shown in the screenshot below A.8.

Week	Responsible	Task
Week 1 & 2	Frank	Reproducible Test Environment implementation
Week 1 & 2	Lukas	NSAK Scenario: AI Scenarios (first part): AI-Agent implementation

Table A.7.: Sprint Planning 3

Sprint Review 3: 09.04.2026

The completion of the following two milestones in this sprint are marking a very important point in our bachelor thesis, as we are now able to run the AI based reconnaissance scenario against the reproducible test environment. This enables us to reproducibly test and analyze the actual capabilities of the AI-agent in a realistic scenario. Further we can now demonstrate the full interplay of the technologies to the projects stakeholders and possibly other interested persons.

Milestone Review: Reproducible Test Environment The success criteria of the Reproducible Test Environment A.3.1 milestone are completed as illustrated in A.9.

short description about the results of the milestone

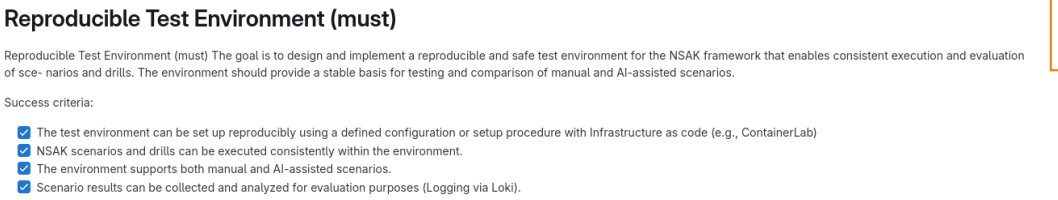


Figure A.9.: Milestone Description: Reproducible Test Environment

Milestone Review: Implement AI Scenarios (first part): AI-Agent implementation As shown in A.10, all "must" goals of the milestone Implement AI Scenarios A.3.1

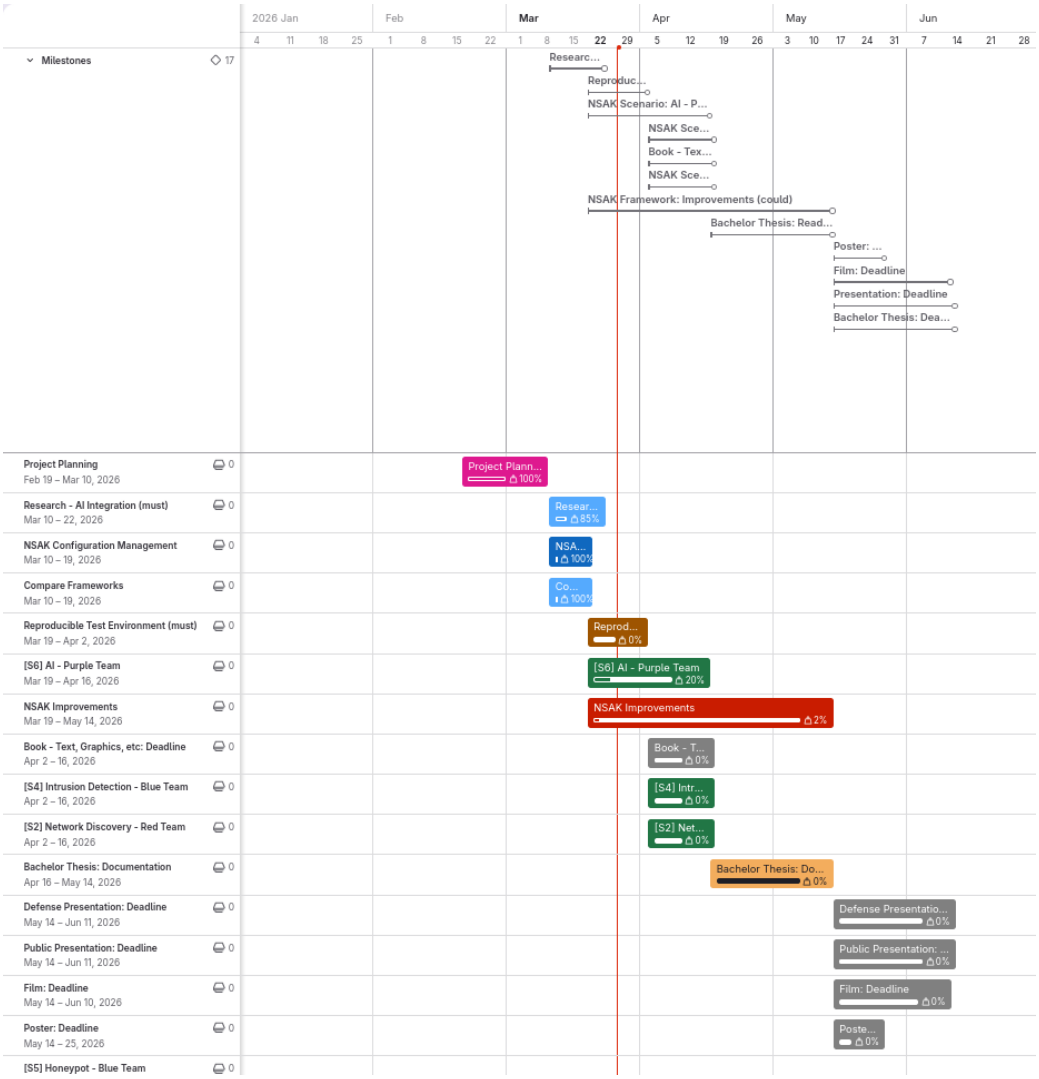
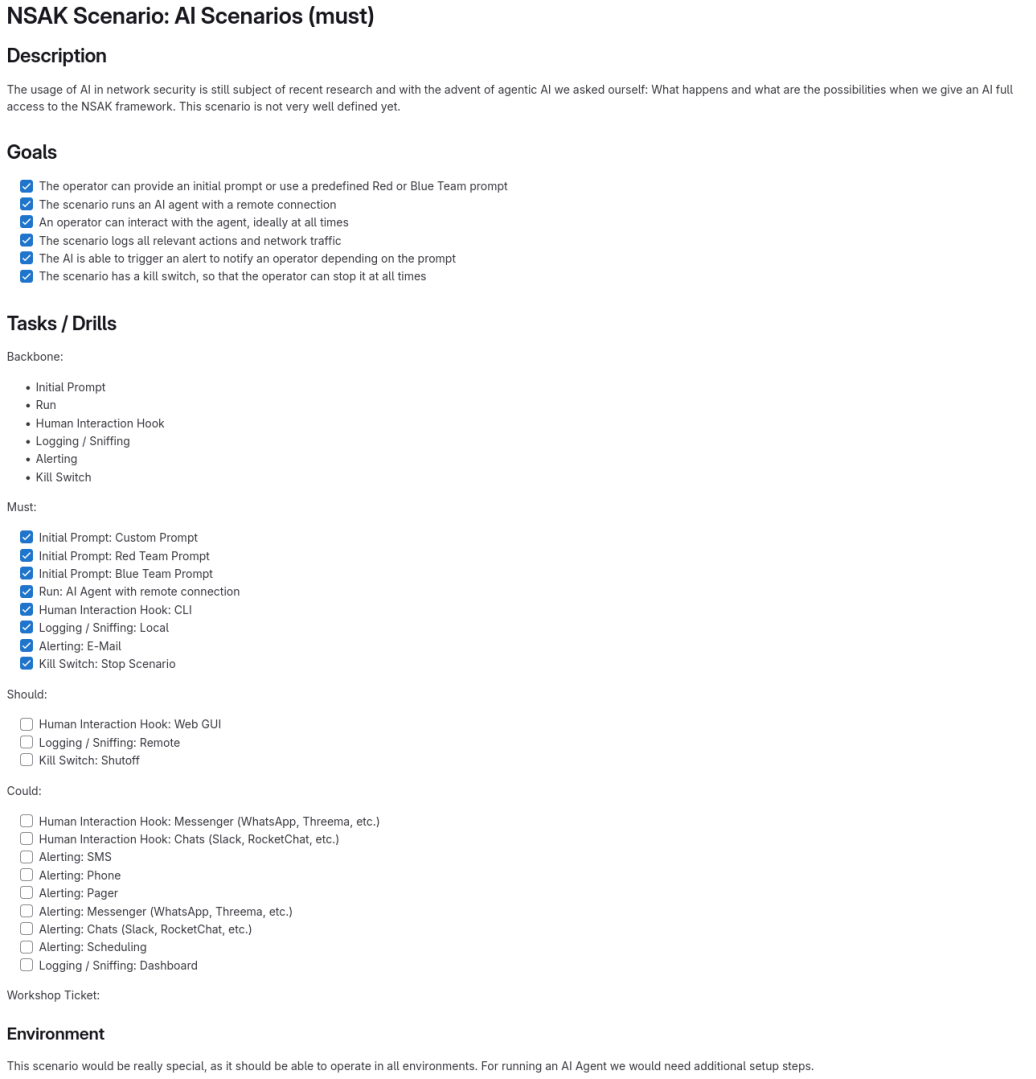


Figure A.8.: Roadmap Sprint Planning 3

where completed. But there is still some work todo, to document the implementation and the learnings we made during the development. Further we could improve the implementation in several aspects such as tool-calling and external logging, where we can better analyze the AI-agents behavior. Because this was expected, we planned two sprints for the realization of the AI Scenarios.



NSAK Scenario: AI Scenarios (must)

Description

The usage of AI in network security is still subject of recent research and with the advent of agentic AI we asked ourself: What happens and what are the possibilities when we give an AI full access to the NSAK framework. This scenario is not very well defined yet.

Goals

- [x] The operator can provide an initial prompt or use a predefined Red or Blue Team prompt
- [x] The scenario runs an AI agent with a remote connection
- [x] An operator can interact with the agent, ideally at all times
- [x] The scenario logs all relevant actions and network traffic
- [x] The AI is able to trigger an alert to notify an operator depending on the prompt
- [x] The scenario has a kill switch, so that the operator can stop it at all times

Tasks / Drills

Backbone:

- Initial Prompt
- Run
- Human Interaction Hook
- Logging / Sniffing
- Alerting
- Kill Switch

Must:

- [x] Initial Prompt: Custom Prompt
- [x] Initial Prompt: Red Team Prompt
- [x] Initial Prompt: Blue Team Prompt
- [x] Run: AI Agent with remote connection
- [x] Human Interaction Hook: CLI
- [x] Logging / Sniffing: Local
- [x] Alerting: E-Mail
- [x] Kill Switch: Stop Scenario

Should:

- [ ] Human Interaction Hook: Web GUI
- [ ] Logging / Sniffing: Remote
- [ ] Kill Switch: Shutoff

Could:

- [ ] Human Interaction Hook: Messenger (WhatsApp, Threema, etc.)
- [ ] Human Interaction Hook: Chats (Slack, RocketChat, etc.)
- [ ] Alerting: SMS
- [ ] Alerting: Phone
- [ ] Alerting: Pager
- [ ] Alerting: Messenger (WhatsApp, Threema, etc.)
- [ ] Alerting: Chats (Slack, RocketChat, etc.)
- [ ] Alerting: Scheduling
- [ ] Logging / Sniffing: Dashboard

Workshop Ticket:

Environment

This scenario would be really special, as it should be able to operate in all environments. For running an AI Agent we would need additional setup steps.

Figure A.10.: Milestone Description and Charts: Research AI-Integration

Sprint Retro 3: 09.04.2026

The Key Take Away of A.11 are to estimate the milestones more realistically. We spend a lot of time with documentation which hinders the development. The

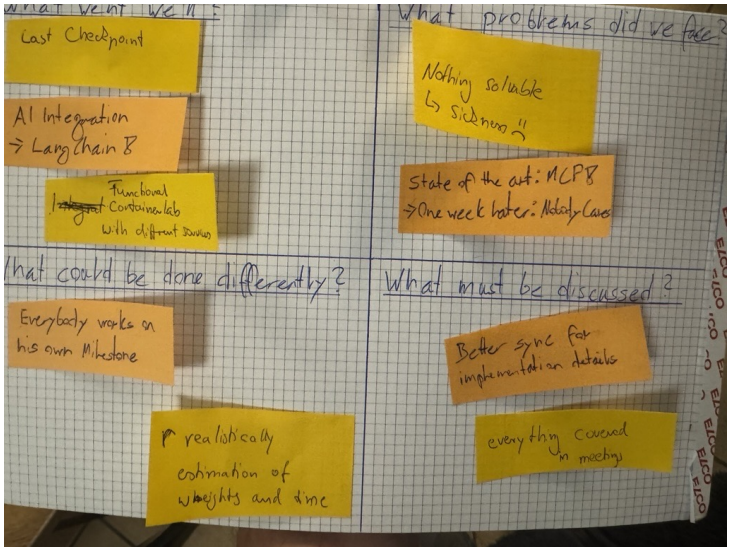


Figure A.11.: Retro Sprint 3

upside of our continuous approach is a natural growing thesis which helps in the long run. Overall, we are quite happy with the outcome of sprint 3.

A.7.4. Sprint 4: 10.04 –02.05.2026

Sprint Planning 4: 09.04.2026

Week	Responsible	Task
Week 1	Frank & Lukas	Bring together and test reproducible test environment and AI Reconnaissance
Week 2 & 3	Frank	Network Discovery - Red Team
Week 2	Lukas	NSAK Scenario: AI Scenarios (second part): Improve tool calling and documentation
Week 3	Lukas	External Logging with Grafana / Loki

Table A.8.: Sprint Planning 4

No photograph of the sprint planning roadmap was taken during this sprint, so no visual reference is available for this planning session.

Sprint Review 4: 02.05.2026

In this sprint could finally bring together the reproducible network environment and the AI scenario, we realized the last scenario categorized as “Must” goal, extended the logging capabilities and could refine the AI agent. Now we can focus on refining the documentation and finalizing the last preparations before moving to the last phase of our thesis, the evaluation and discussion.

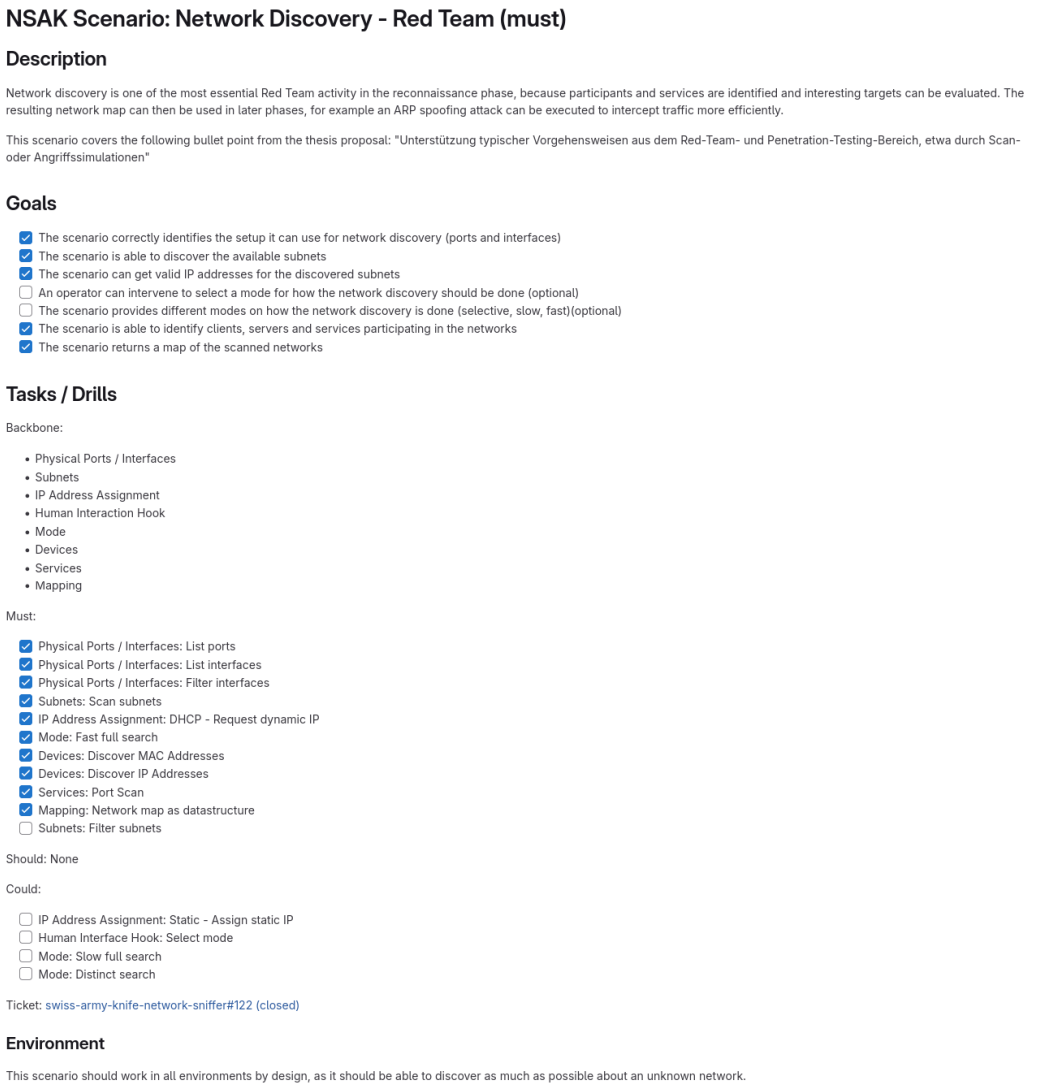
Test run of the AI scenario in the reproducible network environment After finishing the previous sprint we were excited to test the AI scenario in the reproducible network environment, as it’s marking a big milestone of our bachelor thesis. We spent more or less a whole day using different models and make fine adjustments to the setup, to see what’s possible. In the end we could instruct the AI agent to conduct network discovery, write a report in markdown, create a network diagram of the findings and send the artifacts via E-Mail. The results of this test run where added to the implementation section of the thesis 5.3.8.

Milestone Review: Network Discovery - Red Team In A.12, not all “must” goals of the milestone where completed. During planning, a human interaction mode was discussed in which an operator could choose how fast or noisy the reconnaissance should be executed. This feature was not implemented, as it does not contribute to answering the thesis question. The scenario is able to discover MAC and IP addresses using arp-scan on Layer 2, and protocols and services using

nmap within the local subnet. All results are stored in a `NetworkDiscoveryResultMap` and displayed to the operator. Service enumeration using targeted NSE scripts was implemented as a dedicated drill, which runs service-specific scripts such as `http-title`, `dns-zone-transfer`, and `smb-security-mode` on all discovered ports.

The scenario could be extended with an optional static IP address, which would be useful in networks where DHCP is not available. Host discovery in remote networks is currently limited to IP addresses only, as Media Access Control addresses are not available across Layer 3 boundaries.

Maybe move the content to the implementation section and reference it



NSAK Scenario: Network Discovery - Red Team (must)

Description

Network discovery is one of the most essential Red Team activity in the reconnaissance phase, because participants and services are identified and interesting targets can be evaluated. The resulting network map can then be used in later phases, for example an ARP spoofing attack can be executed to intercept traffic more efficiently.

This scenario covers the following bullet point from the thesis proposal: "Unterstützung typischer Vorgehensweisen aus dem Red-Team- und Penetration-Testing-Bereich, etwa durch Scan- oder Angriffssimulationen"

Goals

- [x] The scenario correctly identifies the setup it can use for network discovery (ports and interfaces)
- [x] The scenario is able to discover the available subnets
- [x] The scenario can get valid IP addresses for the discovered subnets
- [ ] An operator can intervene to select a mode for how the network discovery should be done (optional)
- [ ] The scenario provides different modes on how the network discovery is done (selective, slow, fast)(optional)
- [x] The scenario is able to identify clients, servers and services participating in the networks
- [x] The scenario returns a map of the scanned networks

Tasks / Drills

Backbone:

- Physical Ports / Interfaces
- Subnets
- IP Address Assignment
- Human Interaction Hook
- Mode
- Devices
- Services
- Mapping

Must:

- [x] Physical Ports / Interfaces: List ports
- [x] Physical Ports / Interfaces: List interfaces
- [x] Physical Ports / Interfaces: Filter interfaces
- [x] Subnets: Scan subnets
- [x] IP Address Assignment: DHCP - Request dynamic IP
- [x] Mode: Fast full search
- [x] Devices: Discover MAC Addresses
- [x] Devices: Discover IP Addresses
- [x] Services: Port Scan
- [x] Mapping: Network map as datastructure
- [ ] Subnets: Filter subnets

Should: None

Could:

- [ ] IP Address Assignment: Static - Assign static IP
- [ ] Human Interface Hook: Select mode
- [ ] Mode: Slow full search
- [ ] Mode: Distinct search

Ticket: swiss-army-knife-network-sniffer#122 (closed)

Environment

This scenario should work in all environments by design, as it should be able to discover as much as possible about an unknown network.

Figure A.12.: Milestone Description: Reproducible Test Environment

Figure A.13.: Retro Sprint 4

Milestone Review: Implement AI Scenarios (second part): Improve tool calling and documentation As already shown in A.10 from the previous sprint review A.7.3, all "must" goals of the milestone Implement AI Scenarios A.3.1 where completed. In this second part the AI scenarios where tested, tool calling was improved and the documentation was added to the implementation section of the thesis 5.3. Further we could already improve the AI scenarios with the findings of the extensive test run described above A.7.4.

External Logging with Grafana / Loki In the fifth checkpoint meeting A.8.5 which as in this sprint, we received the feedback that we should improve the logging for the AI-agent and especially which tools it is calling. We decided to set up a self-hosted logging system with Grafana and Loki, where we can collect all logs of the runs and create custom queries to make it easier to analyze the behavior of the AI-agent. The usage is described in the implementation section of the AI Scenarios 5.3.4. The self-hosted setup is documented in the addendum A.12.10.

Sprint Retro 4: 02.05.2026

Key Takeaways:

- Overall good mood and good progress
- Use 80/20 rule for documentation during the implementation phase, so that the base structure is already there.
- Work more continuously on the Bachelor Thesis during the Week (1-2h every day)

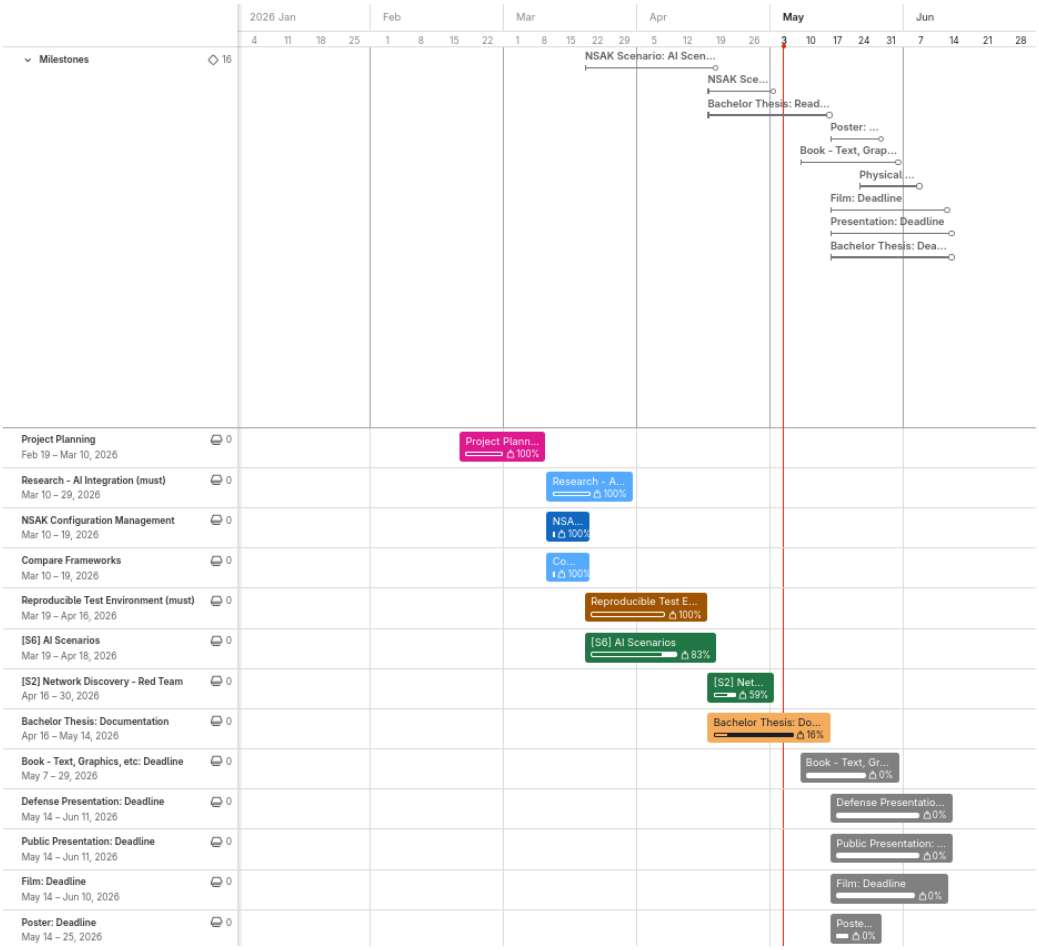


Figure A.14.: Roadmap Sprint Planning 5

A.7.5. Sprint 5: 03.05 –15.05.2026

Week	Responsible	Task
Week 1	Frank & Lukas	Focus on thesis document and prepare evaluation
Week 2	Lukas	Build physical test environment
Week 2	Lukas & Frank	Evaluate the scenarios in containerlab and test if the drills are sufficient to test in bfh lab

Table A.9.: Sprint Planning 5

Sprint Planning 5: 30.04.2026

Sprint Review 5: 15.05.2026

Sprint Retro 5: 15.05.2026

A.7.6. Sprint 6: 15.05 –28.05.2026

Sprint Planning 6: 14.05.2026

Sprint Review 6: 28.05.2026

Sprint Retro 6: 28.05.2026

A.7.7. Sprint 7: 29.05 –11.06.2026

Sprint Planning 7: 28.05.2026

Sprint Review 7: 11.06.2026

Sprint Retro 7: 11.06.2026

Rewrite in past tense

A.7.8. Risk Assessment

To methodically conduct the risk assessment the section “Risk assessment” from an online guide published by Advisera [23], which describes the risk assessment process according to ISO 27001, was used as a reference.

Risk Assessment Process:

1. Risk categories and criteria: Define which dimensions and taxonomies are important for the assessment.
2. Risk identification: Collect possible risks in a brainstorming session.
3. Risk analysis: Assess the risks criteria on a risk matrix.
4. Risk evaluation: Create a list of risks ordered by the resulting risk level and evaluate if they are acceptable or need treatment.
5. Risk treatment: Describe the strategy and how to treat the risks.

Risk Categories & Criteria:

The table A.10 shows four commonly used categories, which where used to group the risks.

Category	Description
Strategic Risk	Risks related to business decisions (e.g., wrong prioritization).
Operational Risk	Risks related to internal processes (e.g., Scrum, QA process).
Financial Risk	Risks related to costs and expenses (e.g., hardware costs).
External Risk	Risks related to uncontrollable sources (e.g., force majeure).

Table A.10.: Risk categories used for the project risk assessment

The criteria for analysing the risks are summarized in the table A.11

Criterion	Description
Probability	Likelihood that the risk will occur.
Impact	Severity of consequences if the risk materializes.
Level	Risk level calculated as Probability + Impact.

Table A.11.: Risk evaluation criteria

The levels of the criteria and their respective numerical weights are listed in the tables A.12 and A.13.

Level	Weight	Description
Low	1	Unlikely
Medium	2	Possible
High	3	Likely

Table A.12.: Risk probability scale

Level	Weight	Description
Low	1	Minor inconveniences
Medium	2	Delays milestones or reduces scope
High	3	Threat to the thesis

Table A.13.: Risk impact scale

The following table A.14 defines the calculation method and the resulting risk levels, used for the assessment.

Risk Level	Weight	Calculation
Very Low	2	Low + Low
Low	3	Low + Medium
Medium	4	Medium + Medium / Low + High
High	5	Medium + High
Critical	6	High + High

Table A.14.: Risk level classification (probability and impact are interchangeable)

Risk Identification

The identification process was conducted with the help of a digital brainstorming, where the risks are already grouped by category. The identified risks are represented by the cards shown in A.15. The respective risk categories are visualized by the color coding of the cards and described in the legend.

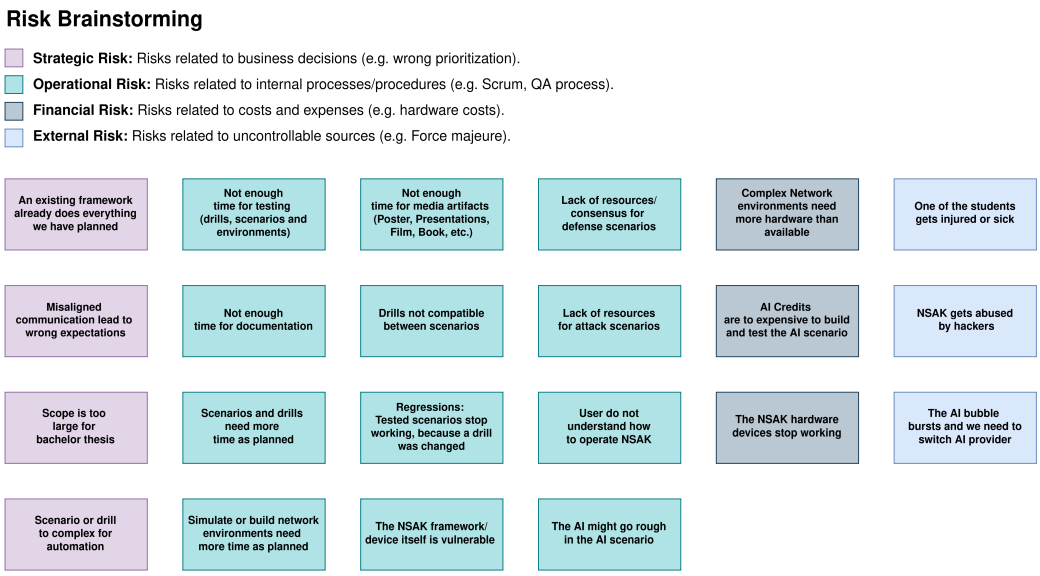


Figure A.15.: Risks Brainstorming

Risk Analysis

For analysing the risks a matrix was prepared, where the criteria “Impact” and “Probability” are representing the x- and y-axis respectively. The project team then assessed the criteria for each individual risk and placed them on the matrix accordingly. In a second step, the risks where moved if they did not fit relatively to each other. The resulting “Risk Analysis Matrix” is illustrated in the following figure A.16. The background color of the cells is representing the resulting risk level of the tasks.

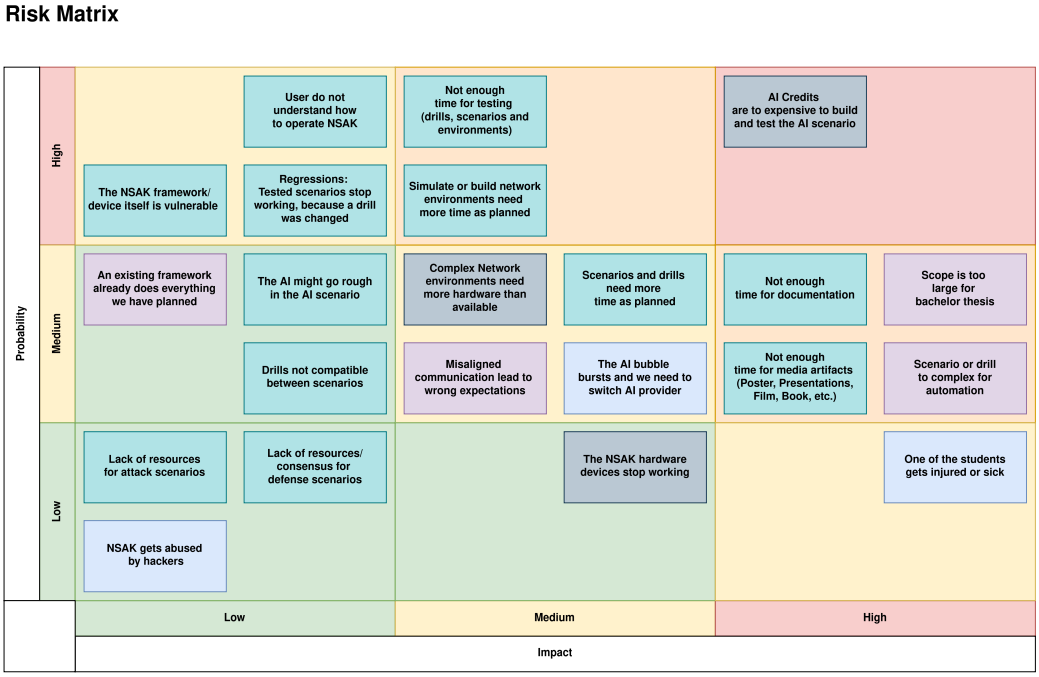


Figure A.16.: Risks Brainstorming

Risk Evaluation & Treatment

In a first step a table with the required columns was created. Then the risks and weights for the criteria where added with an ID in the format “R-000”, to easier reference them in the thesis later on. After sorting the table by the calculated risk level, the risk evaluation was conducted by deciding which treatment strategy described in A.15 we want to apply.

Strategy	Description
Mitigate	Reduce the likelihood or impact of a risk.
Transfer	Move the risk to a third party.
Accept	Acknowledge the risk and take no further action.
Avoid	Eliminate or circumvent the cause of the risk.

Table A.15.: Risk Treatment Strategies

Finally, the project team went through the risks and discussed how to treat them effectively. The resulting “Risk Assessment” is illustrated in the following table A.17.

Risk Assessment

Criteria	Level
1 = Very Low	2 = Low
3 = Medium	4 = Medium
6 = High	9 = Critical

Category
Strategic
Operational
Financial
External

Risk ID	Description	Risk Category	Probability	Impact	Level	Strategy	Treatment
R-001	AI credits are too expensive to build and test the AI scenario	Financial	3	3	9	Avoid	Use a self-hosted LLM, check if the BRH can provide AI services or if there are free plans for education
R-006	Scope is too large for bachelor thesis	Strategic	2	3	6	Mitigate	We have to plan and evaluate the goals carefully. The use of "User Story Maps" also helps to mitigate this risk.
R-007	Scenario or drill too complex for automation	Strategic	2	3	6	Mitigate	Create "User Story Maps" for epics, so that we have a good understanding of the scenarios and their drills.
R-002	Not enough time for documentation	Operational	2	3	6	Mitigate	Add a dedicated documentation issue with an according weight to each epic.
R-003	Not enough time for testing (drills, scenarios and environments)	Operational	3	2	6	Mitigate	Add a dedicated testing issue with an according weight to each epic.
R-004	Simulating or building network environments needs more time than planned	Operational	3	2	6	Mitigate	Add dedicated issues with according weight to each scenario epic.
R-005	User does not understand how to operate NSAK	Operational	3	2	6	Mitigate	Provide a Web GUI and/or user guides
R-012	Misaligned communication leads to wrong expectations	Strategic	2	2	4	Mitigate	Multiple channels and Bi-Weekly Meetings should keep the communication aligned.
R-010	Scenarios and drills need more time than planned	Operational	2	2	4	Mitigate	Use "User Story Maps" to get a good idea of the required steps to build the scenario, it's drills and estimate generously.
R-011	Not enough time for media artifacts (poster, presentations, film, book, etc.)	Operational	2	2	4	Mitigate	Include the creation of the media artifacts as milestones/epics in the project planning.
R-009	Complex network environments need more hardware than available	Financial	2	2	4	Avoid	We can simulate network environments instead of real hardware.
R-008	The AI bubble bursts and we need to switch AI provider	External	2	2	4	Avoid	Use vendor agnostic tools and protocolls, so that we can switch provider easily.
R-014	Regressions: tested scenarios stop working because a drill was changed	Operational	3	1	3	Mitigate	Add automated test cases for detecting regressions early.
R-015	The NSAK framework/device itself is vulnerable	Operational	3	1	3	Transfer	Clearly state that the NSAK framework/device is still a POC and not ready for production use.
R-013	One of the students gets injured or sick	External	1	3	3	Accept	We accept this risk.
R-019	An existing framework already does everything we have planned	Strategic	2	1	2	Accept	We accept the risk, but will compare well known frameworks with NSAK and highlight our USP (HW and AI).
R-017	Drills not compatible between scenarios	Operational	2	1	2	Mitigate	The implementation of the configuration management should provide a way that small deviations can be branched.
R-018	The AI might go rogue in the AI scenario	Operational	2	1	2	Mitigate	Besides the fact that we have physical access to the device we add a killswitch to the AI scenario.
R-016	The NSAK hardware devices stop working	Financial	1	2	2	Accept	The fact that we have two devices already reduces the impact, further we could also use other HW.
R-021	Lack of resources for attack scenarios	Operational	1	1	1	Mitigate	Get familiar and use the MITRE ATT&CK framework as reference and source.
R-022	Lack of resources / consensus for defense scenarios	Operational	1	1	1	Mitigate	Get familiar and use well known sources such as NIST as source.
R-020	NSAK gets abused by hackers	External	1	1	1	Transfer	Clearly state that the NSAK framework/device is built only for educational and white hate purposes.

Figure A.17.: Risk Assessment

A.8. Meeting Notes

A.8.1. Checkpoint Meeting 1: 26.02.26

Add slides if we can reproduce them

Participants	Hj. Wenger (Instructor), Frank Gauss, Lukas von Allmen
Agenda	- Goals presentation via slides - Clarification whether there is a deadline or submission required for the presentation (besides adding it to the addendum). - Clarification whether the defense presentation should be an enhanced version of the public presentation or a separate, more technical presentation. - Proposal to establish a recurring meeting series every Thursday at 15:00 during the semester.
Tasks	- Hand in a set of success criteria for the project goals: We thought we can delivery the success criteria and the project goals as milestones in gitlab, we corrected this later on. - Conduct a risk assessment: A.7.8 - Contact expert via e-mail: Contacted at 21.02.26, response 03.03.26, invited to next checkpoint meeting 05.03.26, the expert takes part at the meeting every two weeks

Table A.16.: Meeting Notes: Checkpoint 1

A.8.2. Checkpoint Meeting 2: 05.03.26

Add slides if we can reproduce them

Participants	Hj. Wenger (Instructor), Urs Keller (Expert), Frank Gauss, Lukas von Allmen
Agenda	- Review of the project goals as milestones and project planning. - Review of risk assessment. - Discussion on the investigation and analysis of network communication between end devices and external networks (RED scenario: sniffing, filtering, and potential data exfiltration).
Decisions	- Variant 1 was selected for further development. - Open questions remain regarding the use and management of AI tokens.
Notes	- We misunderstood how to deliver the project goals and success criteria: Instead of defining the goals as milestone we added a goal section in the thesis. - Discussion on the rationale behind selecting Variant 1. Consideration of whether a user interface is required when MCP functionality is available - Open Research Question. - NSAK integrates AI capabilities. - The development of a Web GUI is currently considered lower priority - suggestion of Urs Keller.
Tasks	- Send the slides of the final presentation and documentation of “Project 2” to Urs Keller (Expert): “Done” (via E-Mail). - Delivery the projects goals with a detailed description as part of the thesis by Saturday, 07.03.26: “Done” 1.3 - Finalize the project planning: “Done” A.6 - Write the first few chapters of the thesis: “Done”

Table A.17.: Meeting Notes: Checkpoint 2

A.8.3. Checkpoint Meeting 3: 19.03.26

Participants	Hj. Wenger (Instructor), Urs Keller (Expert), Frank Gauss, Lukas von Allmen
Agenda	- Review Project Management and Roadmap - Review of the active milestones (Framework Comparison, AI-Research, Configuration Management) - Discuss Variant Decisions: AI-Integration - Ask about AI token usage - Ask about repository access
Decisions	- We will proceed using NSAK as the basis for our thesis. - Open questions remain regarding the use and management of AI tokens.
Notes	- -
Tasks	- Document decision that we will continue to work with the NSAK framework: 2.2.3 - Expand on variant decision for AI-Integration: - Split diagram into 2 variants: - Expand in detail how the variants are working (e.g., sequence diagram): A.21 and A.23 - Configuration management: Move metadata and others below the device node: “Done” 5.1.2 - BFH TI: Has a cluster with studio macs for AI usage -> Ask if we can use them (https://mlmp.ti.bfh.ch/ via VPN)

Table A.18.: Meeting Notes: Checkpoint 3

A.8.4. Checkpoint Meeting 4: 02.04.26

Participants	Hj. Wenger (Instructor), Frank Gauss, Lukas von Allmen
Agenda	- Overview Roadmap and active milestones - Reproducible Test Environment: Introduce ContainerLab, Show diagram, Physical Implementation (could) - AI Scenario: Demo AI-agent connection to self-hosted, LangChain Research LLM - Review MCP Sequence Diagrams - Discuss what needs to be added to the main document and what to the appendix - Ask if it's possible to lend more HW for physical test setup - Ask for access to BFH AI Cluster
Decisions	- -
Notes	- - Alternative to container LAB: Cisco - DNS3 (not open source) - It would be interesting to add more services to the DMZ in the reproducible test environment - Tutor: The current structure is preferred, we should also ask the expert -
Tasks	- Ask for access to BFH AI: https://mlmp.ti.bfh.ch (Requires VPN) - If we are not allowed, ask tutor for guidance - Variant decisions: Add pros/cons and elaborate why we choose a variant - AI-agent: Design and implement logging - Ask the expert about the preferred document structure: Thesis vs Appendix - Create a HW list for tutor, date and configuration (must be planned, device configuration could be done by assistants) -

Table A.19.: Meeting Notes: Checkpoint 4

A.8.5. Checkpoint Meeting 5: 16.04.26

Participants	Hj. Wenger (Instructor), Urs Keller (Expert), Frank Gauss, Lukas von Allmen
Agenda	- Demo: Reproducible test environment - Demo: AI Agent - Show current state of the project goals and roadmap - Fix the date for bachelor thesis defense: Ideally 17ör 18.04.2026
Decisions	- Defense Date: 18.06.2026, 13:30 - 15:30
Tasks	- Add a system prompt to tell the LLM to create a human-readable report of what it did, parallel to the logs: - Export logs to a remote server like Zabbix or Grafana:

Table A.20.: Meeting Notes: Checkpoint 5

A.8.6. Checkpoint Meeting 6: 30.04.26

Participants	Hj. Wenger (Instructor), Frank Gauss, Lukas von Allmen
Agenda	- Demo: Reconnaissance scenario - Demo: Grafana / Loki (Authentication Issue) - Show Roadmap and next two weeks - How can we compare and evaluate the AI with the Reconnaissance scenario?
Notes	- Take a look at nmap plugins - To compare and evaluate the AI with the Reconnaissance scenario, we could run both scenarios in the BFH LAB (physical or virtual). For a physical test run we would need to fix a date with HJ. Wenger. - Interesting metrics are: Speed, Correctness, Initial knowledge (pre-conditions), Creativity (3 - 5 interesting criteria) - Only configure Grafana / Loki to the point where we can easily analyze the steps and tools calls.
Decisions	-
Tasks	-

Table A.21.: Meeting Notes: Checkpoint 6

A.8.7. Checkpoint Meeting 7: 14.05.26

A.8.8. Checkpoint Meeting 8: 28.05.26

A.8.9. Checkpoint Meeting 9: 11.06.26

Variant 1: AI Purple Team

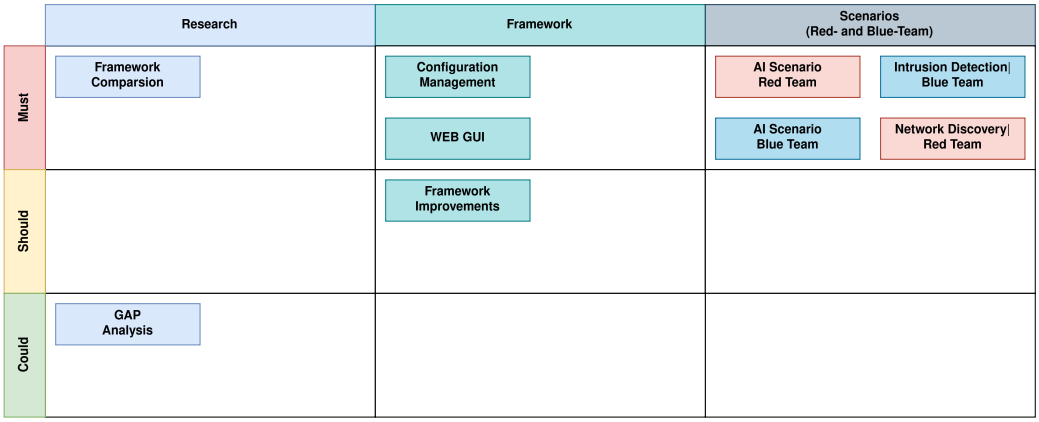


Figure A.18.: Variant 1: AI vs Manually Built Scenarios

A.9. Variant Decision: Thesis Goals

During the first sprint of the project, which was focused on project management and planning, we identified the very promising hypothesis that red and blue team activities could be automated and/or enhanced by AI. Pursuing this hypothesis would change the projects focus and goals and thus requires the agreement and approval of the stakeholder. To communicate this circumstance efficiently and leave the option to go with the status quo, we prepared two options to conduct a "Variant Decision". In the following two subsections the two variants presented in the second checkpoint meeting A.17 are described and illustrated. It must be noted, that the variants and their visualizations A.18 A.19 are momentary snapshots and will not be updated for traceability reasons. For example, the Web GUI was already deprioritized during the discussion of the variants, which is reflected in the project goals 1.3 but not here.

A.9.1. Variant 1: AI vs Manually Built Scenarios

Figure A.18 shows the project goals adjusted for the AI focused hypothesis, which is favored variant of the project team. In this option most of the evaluated scenarios will not be realized in favor of AI based scenarios, which also requires additional research 1.3.1. The basic idea is to implement scenarios for network discovery 1.3.3, intrusion detection 1.3.3 and an AI based scenario with the same capabilities 1.3.3. In the evaluation of the thesis we can then assess the quality of the AI based scenarios itself 1.3.4 and in comparison to the manually built scenarios 1.3.4.

Variant 2

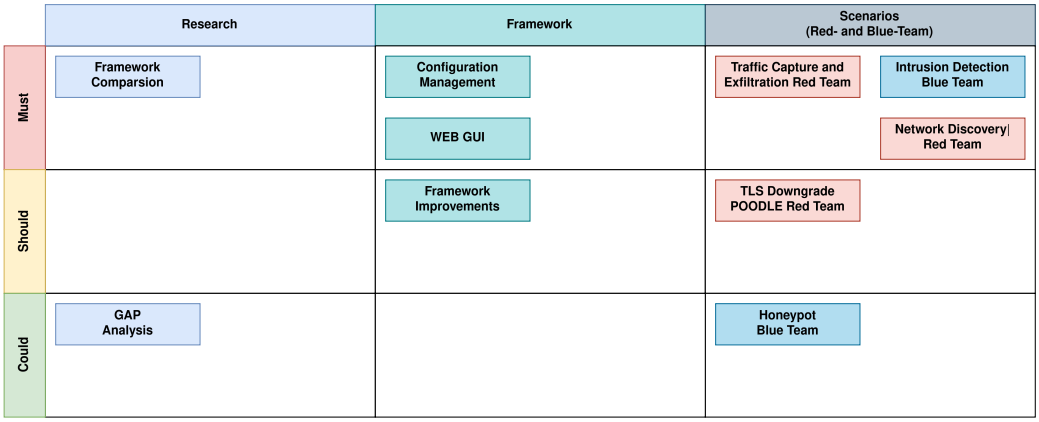


Figure A.19.: Variant 2: Multiple Blue and Red Team Scenarios

A.9.2. Variant 2: Multiple Blue and Red Team Scenarios (Status Quo)

This is the fallback if the other option A.9.1 is not approved by the stakeholders. Even if the project team does not favor this more conservative variant, it still represents a valid way to conduct an interesting project. As illustrated below A.19, in this variant we propose to proceed with the previously planned scenarios for which we already conducted "User Story Mappings" documented under the workshops sectionA.11. The focus would lie in more conventional scenarios, overall framework improvements and an in depth comparison with another framework in form of a GAP Analysis.

A.9.3. Decision

The stakeholders and project team decided in favor of **Variant 1: AI vs Manually Built Scenarios** A.9.1 in the checkpoint meeting A.17.

A.10. Variant Decision: AI Integration

During the AI-research 2.2 we identified two interesting ways to allow NSAK to interact with AI based technologies. As illustrated in A.20, the first variant purposes to implement an AI-agent directly into the framework and use it in scenarios and drills. In the second variant shown in A.22, the capabilities of NSAK and other relevant tools on the NSAK device would be exposed via MCP.

A.10.1. Variant 1: AI-agent in NSAK

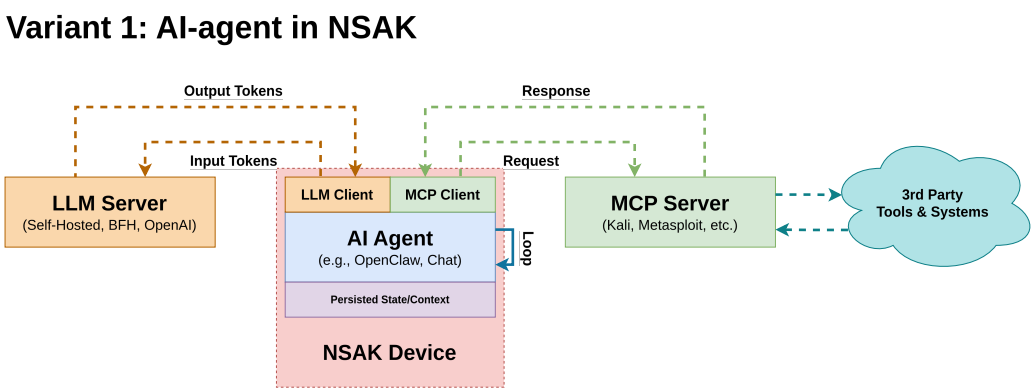


Figure A.20.: AI Integration Variant 1: AI-agent in NSAK

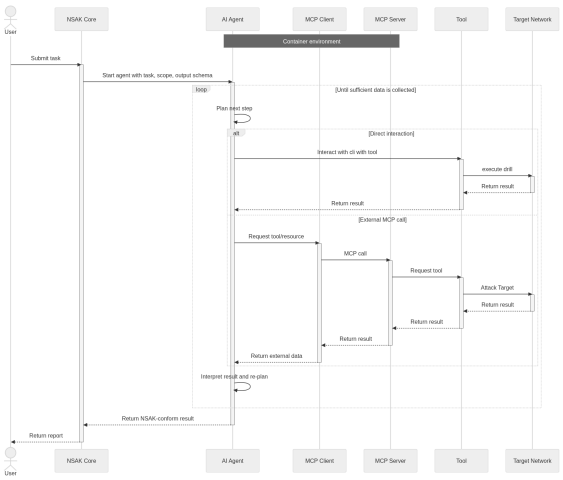


Figure A.21.: Variant 1, AI Agent in NSAK

In this architecture, the AI agent is embedded within the NSAK device. The NSAK core establishes the operational guardrails and schema, and provides the initial

prompt to the AI Agent. The agent uses a local or remote LLM, which is accessible over the management network. It operates within an iterative loop, in which it may either invoke external tools via a command-line interface (e.g., nmap, Metasploit) or interact with them through an MCP server interface. The results of these operations are subsequently processed and returned to the NSAK core in a structured schema-compliant format.

A.10.2. Variant 2: NSAK as MCP Server

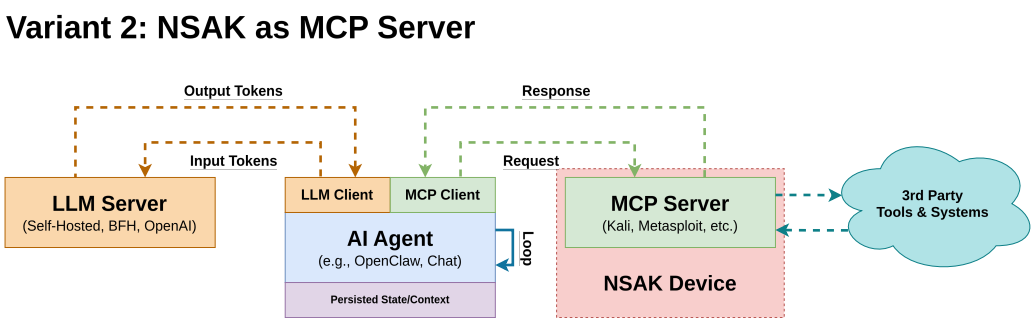


Figure A.22.: AI Integration Variant 2: NSAK as MCP server

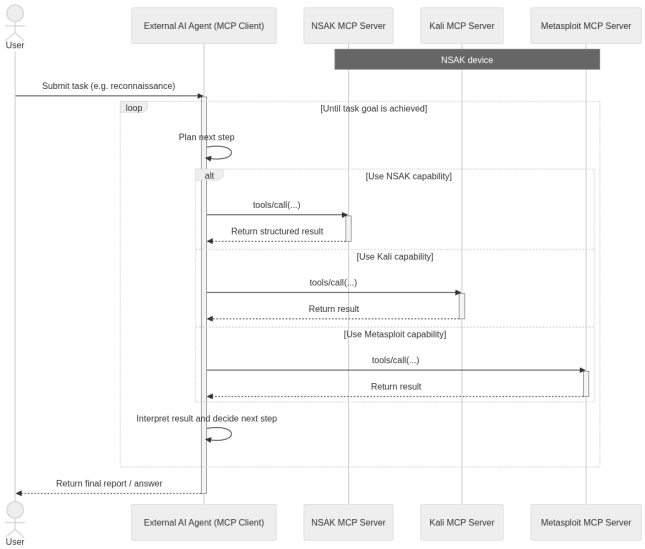


Figure A.23.: Variant 2 NSAK MCP Server

Variant 1, in which the Agentic-AI is embedded within the NSAK device, enables the definition and enforcement of schemas and security policies within a controlled environment. Furthermore, it allows direct interaction with tools via the

command-line interface and integration with MCP servers. This results in a tightly coupled architecture with high control and consistency.

In contrast, Variant 2 relies on an external agent that accesses the NSAK device through an exposed MCP server interface. While this approach provides increased modularity, it also introduces limitations in flexibility, as the agent is restricted to the set of tools exposed by the NSAK device.

From a holistic and multi-layered architectural perspective, Variant 1 is preferred. It enables deeper system integration, higher security, and a wider range of executable tools.

A.10.3. Decision

The project team decided in favor of **Variant 1: AI-agent in NSAK** A.10.1, as it promises to greatly increase the capabilities of NSAK regarding autonomy and automation.

enhance the decision with technical background

This is not correct anymore: we use LangChain Tools instead of MCP for tool calling, which is their own abstraction over vendor specific APIs

A.11. Workshops

In the Project Management phase, several workshops were conducted, including a brainstorming session using Post-it notes and a User Story Mapping exercise on a whiteboard. The goal of these workshops was to clarify the scenarios a network edge device should support.

Following these workshops, the insights gathered were used to derive issues and goals, which were then mapped to detailed milestones in GitLab. All deliverables were categorized using the MoSCoW prioritization method into must, should, and could requirements. An example of this prioritization and scenario mapping is illustrated in Figure A.26. All Milestones links are provided in the URL at the top of the section.

A.11.1. Brainstorm Scenario Selection

In this workshop, participants brainstorm and cluster potential red and blue team scenarios for practical cybersecurity exercises.

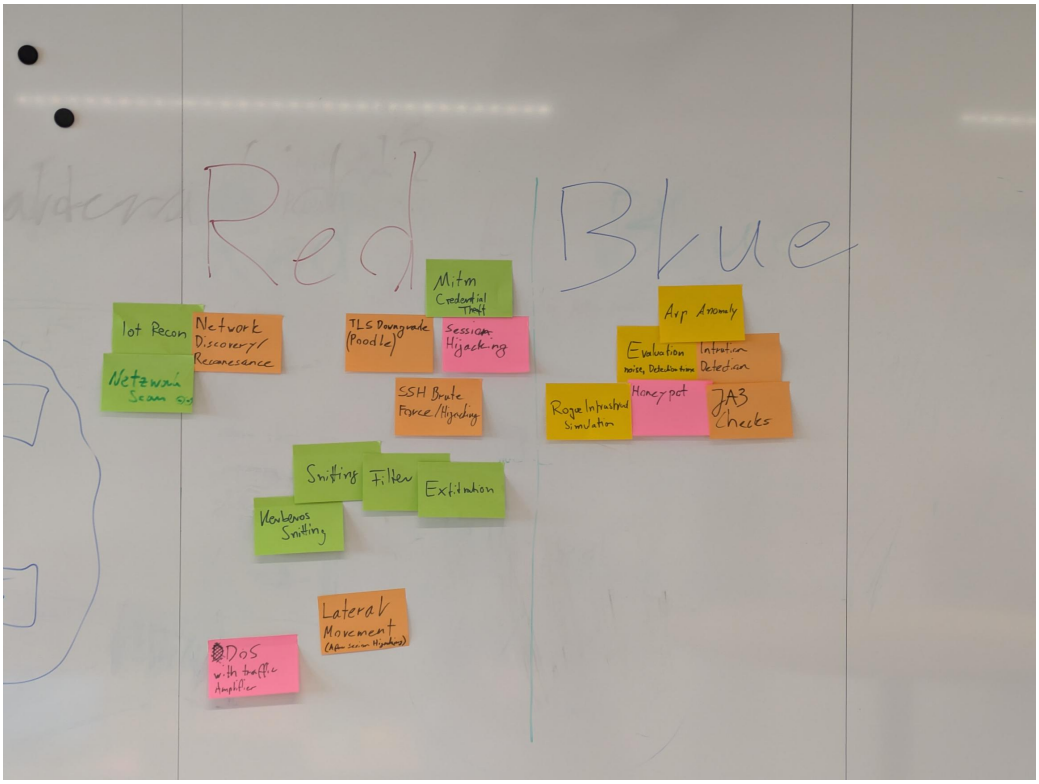


Figure A.24.: Brainstorm Scenarios

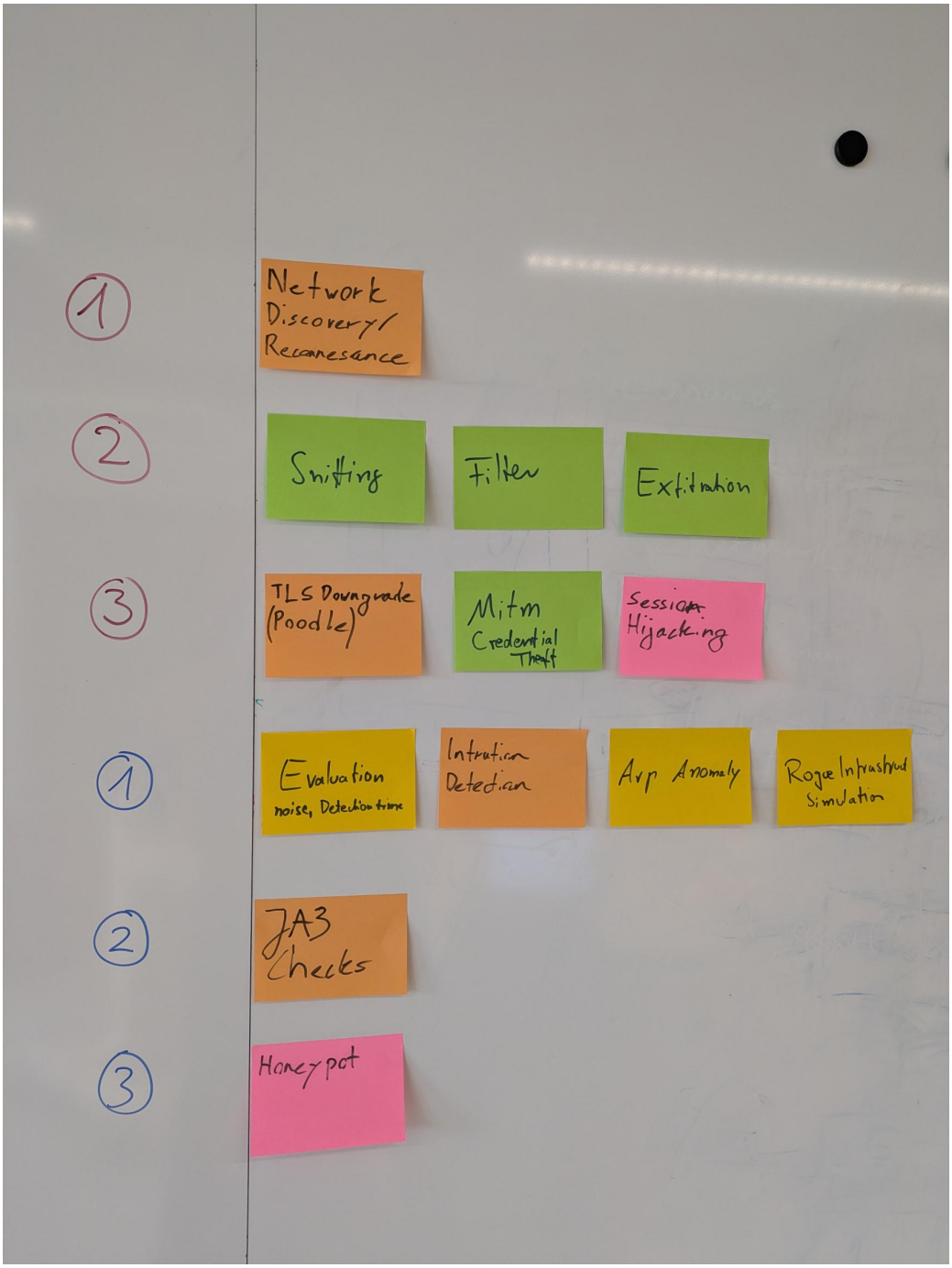


Figure A.25.: Clustering possible Scenarios

The Brainstorm session is used to derive further red/blue team scenarios and led to the pivot of AI-enabled Nsak scenario.

A.11.2. NSAK Scenario: Network Discovery - Red Team

Network Discovery Scenario

Objectives

This workshop aims to evaluate the different steps of the network discovery process and to help participants understand how attackers identify and map potential targets within a network.

Building on this objective, Red Teams consider network discovery one of the most essential activities during the reconnaissance phase, as it helps them identify hosts and services and evaluate potential targets. Once a network map is created, the team can use it in later phases. For example, they might execute an ARP spoofing attack, which allows them to intercept network traffic more efficiently.

In relation to these activities, this scenario addresses the following point from the thesis proposal:
'Unterstützung typischer Vorgehensweisen aus dem Red-Team- und Penetration-Testing-Bereich, etwa durch Scan- oder Angriffssimulationen.'

The workshop led to the milestone *NSAK Scenario: Network Discovery – Red Team (must)*, in which each step for discovering and mapping a network is documented and structured for later implementation.

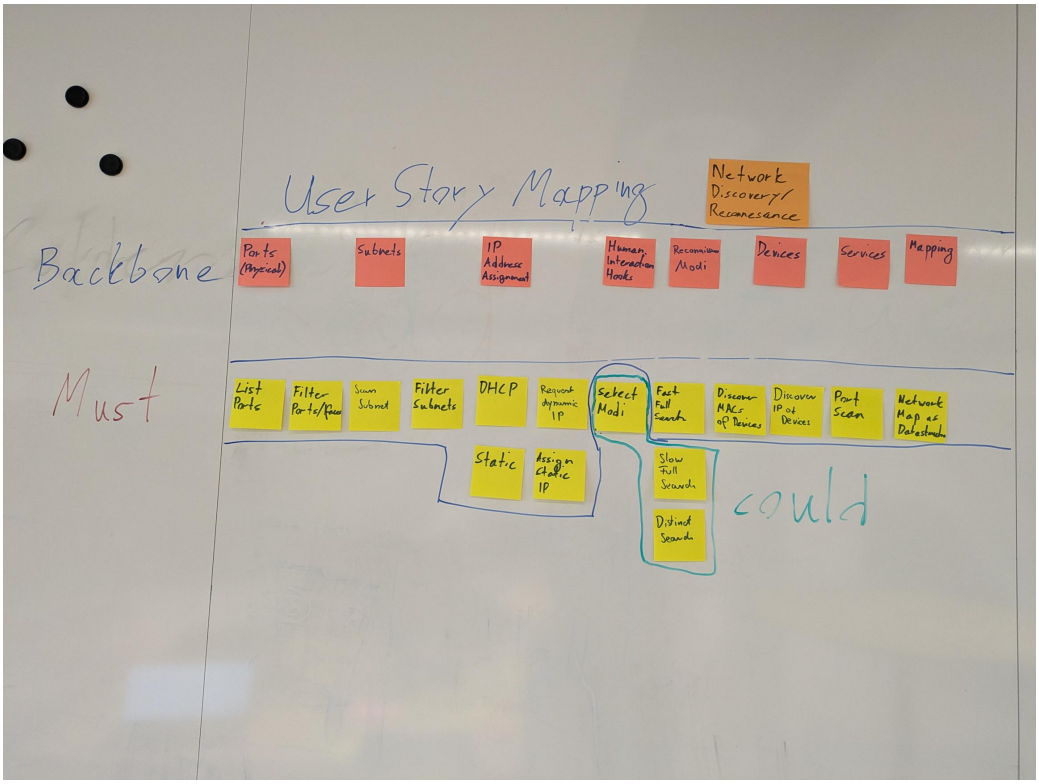


Figure A.26.: NSAK Scenario: Network Discovery - Red Team

A.11.3. NSAK Scenario: TLS Downgrade Poodle - Red Team

TLS Downgrade Poodle Milestone

Objectives

This workshop demonstrates how attackers can exploit TLS downgrade vulnerabilities, such as POODLE, to intercept encrypted communication.

In this scenario, the infamous POODLE attack is implemented, in which a malicious actor tries to convince a client and a server to use the vulnerable TLS version 1.1 instead of 1.2 or 1.3 to encrypt HTTP traffic. This attack requires the attacker to sit between the client and the server (a man-in-the-middle, or MITM), allowing them to intercept and control the traffic. When the client initiates a TLS handshake, the attacker drops packets to the server and sends TCP reset packets to the client, eventually trying older TLS versions. Because TLS 1.1 is vulnerable to a CBC Padding attack, it is possible to create a padding oracle that gradually decrypts the cookie and hijacks the user's session.

Building on this, the scenario directly addresses the following aspect from the thesis proposal:
‘Unterstützung typischer Vorgehensweisen aus dem Red-Team- und Penetration-Testing-Bereich, etwa durch Scan- oder Angriffssimulationen’

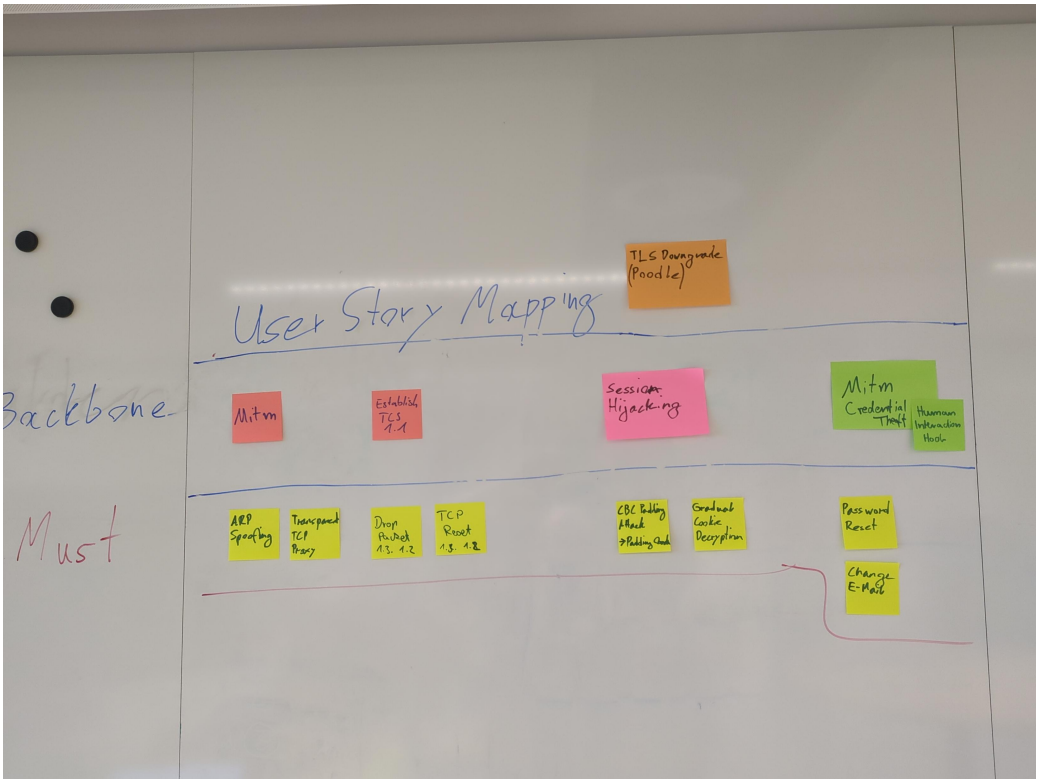


Figure A.27.: NSAK Scenario: TLS Downgrade Poodle - Red Team

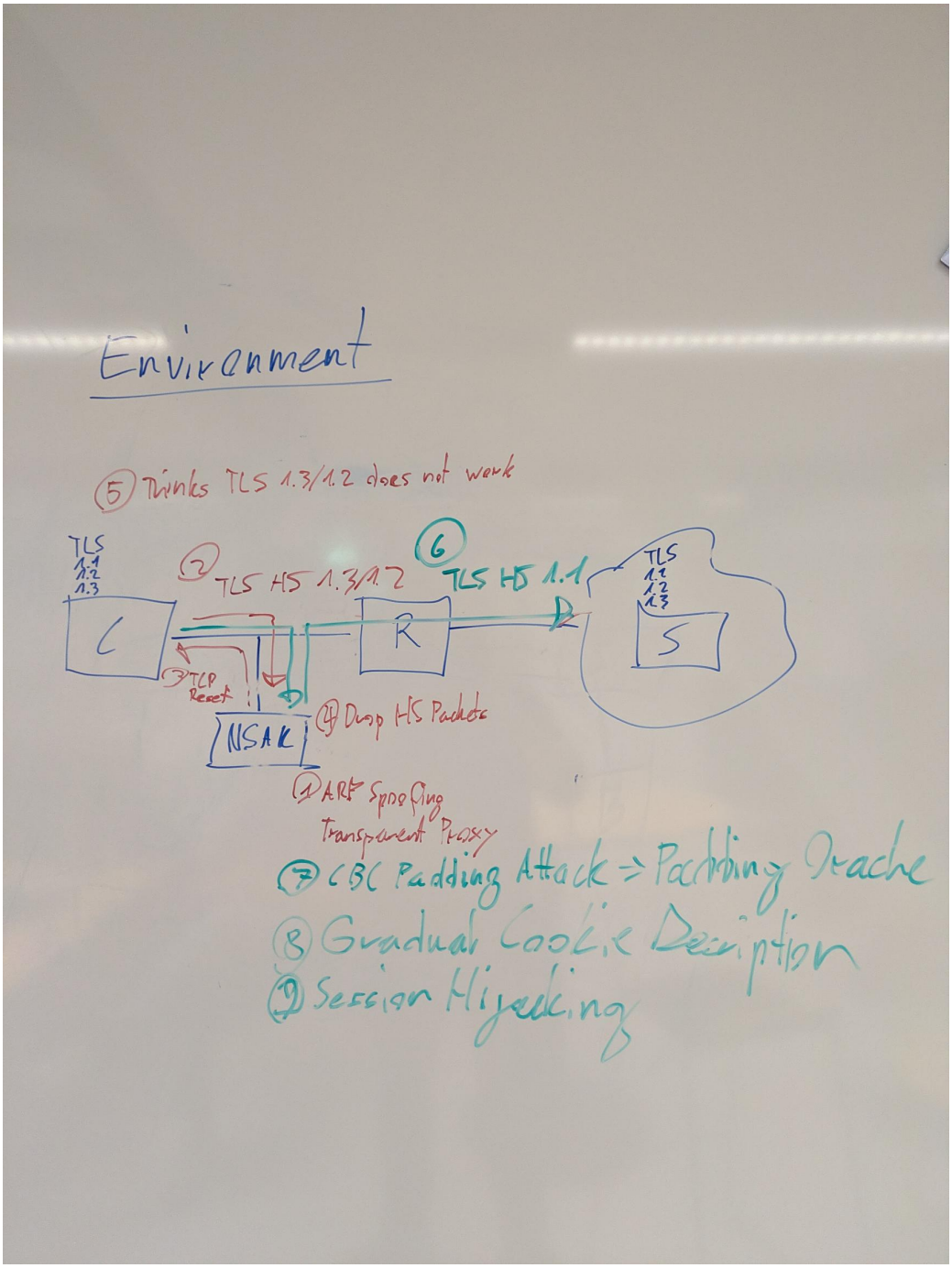


Figure A.28.: NSAK Scenario: TLS Downgrade Poodle -Topology

A.11.4. NSAK Scenario: Honeypot - Blue Team

Honeypot Milestone

Honeypots can serve as an additional form of intrusion detection and help Blue Teams detect attackers in a network. The system tries to attract malicious actors by providing an attack surface as large as possible; a simple way to achieve this is to open ports. After an attacker tries to connect to any of the open ports, the system notifies an operator and may close all open ports to prevent further attempts to compromise the Honeypot. The operator can then manually intervene to identify and isolate the attacker and analyze the captured payload.

This scenario covers the following bullet point from the thesis proposal: ‘Bereitstellung von Funktionen zur Erkennung, Protokollierung und Analyse verdächtiger Aktivitäten als Teil defensiver Massnahmen’

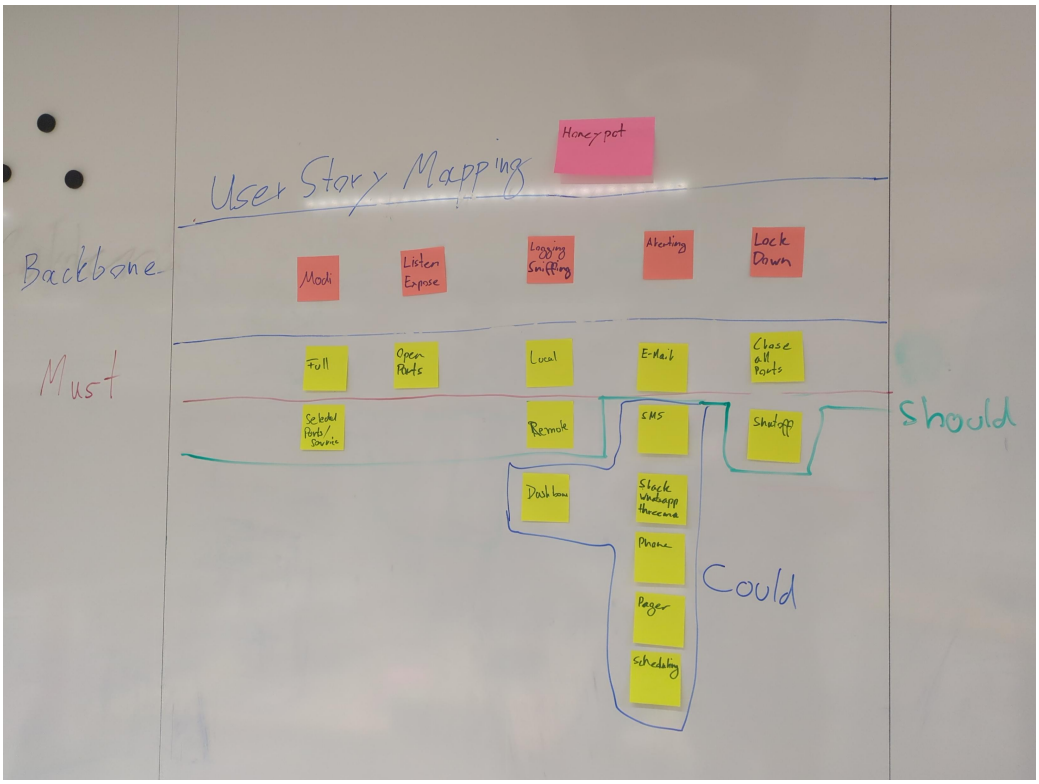


Figure A.29.: NSAK Scenario: Honeypot - Blue Team

A.11.5. NSAK Scenario: Traffic Capture - Red Team

Traffic Capture Milestone

Capturing, exfiltrating, and analyzing traffic from a targeted network is a key Red Team activity and crucial for initial reconnaissance and planning the next phases of the attack. Based on the gathered intelligence, network participants and services can be identified, and possible attack vectors may be evaluated. If unencrypted traffic is present on the network, session data and credentials could already have been extracted.

This scenario addresses the following bullet point from the thesis proposal: 'Untersuchung und Auswertung der Netzwerkkommunikation zwischen Endgeräten und externen Netzen.'

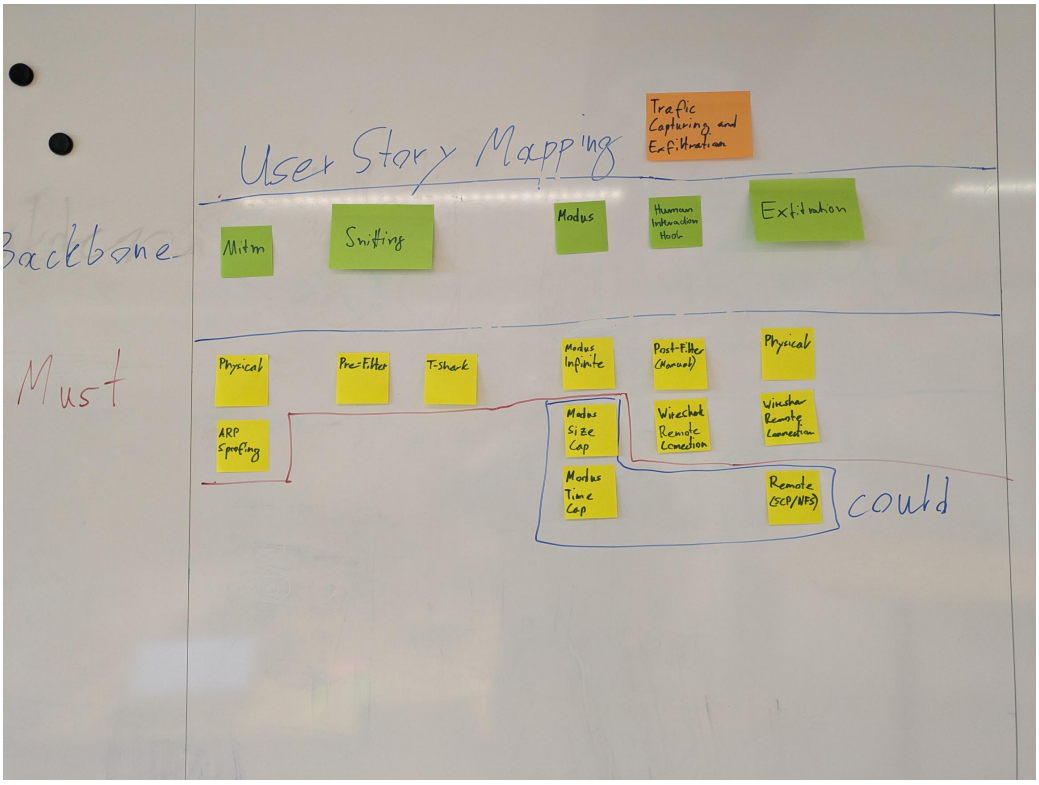


Figure A.30.: NSAK Scenario: Traffic Capture - Red Team

A.11.6. NSAK Scenario: Intrusion Detection - Blue Team

Intrusion Detection Milestone

Intrusion detection is one of the core Blue Team activities and aims to identify malicious actors and misconfigured clients in production network environments. This can be achieved by introducing a system at a traffic mirroring port or as a central point in a network. The intrusion detection system can leverage a wide array of security policies, reference traffic, and other markers to effectively iden-

tify anomalies. As soon as suspicious activity is detected or a policy is violated, automated measures may be initiated, and security engineers will be automatically alerted for manual intervention.

This scenario covers the following bullet point from the thesis proposal: “Bereitstellung von Funktionen zur Erkennung, Protokollierung und Analyze verdächtiger Aktivitäten als Teil defensiver Massnahmen”

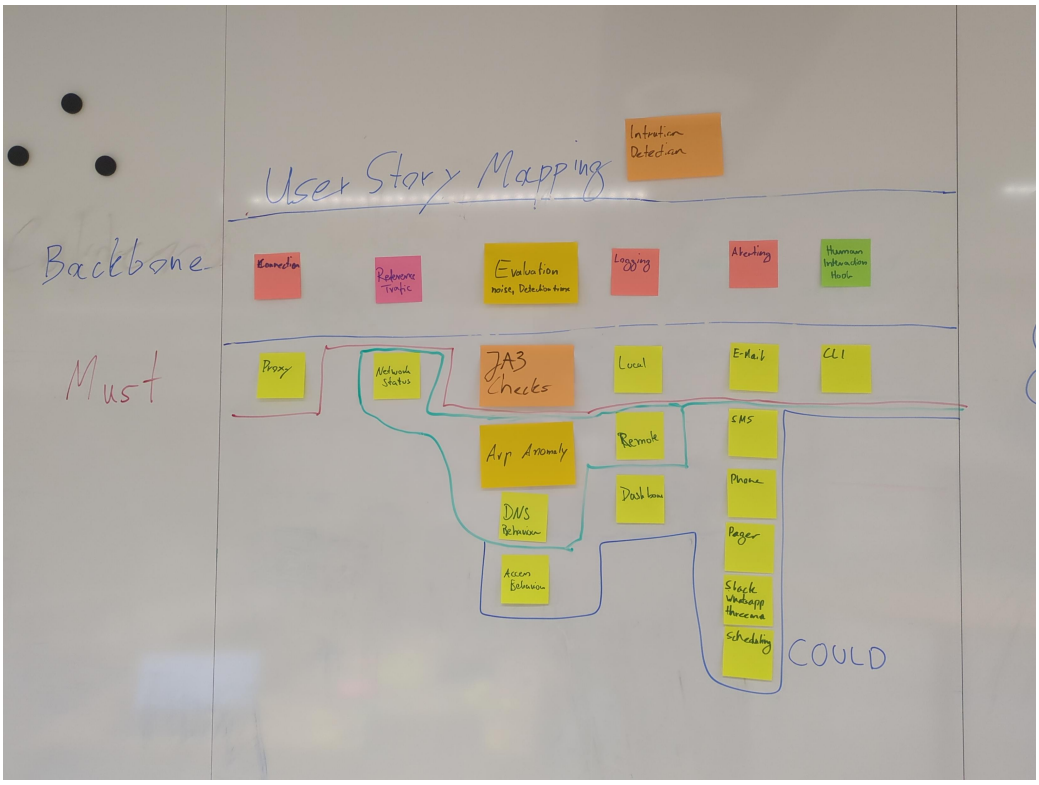


Figure A.31.: NSAK Scenario: Intrusion Detection - Blue Team

A.11.7. NSAK Scenario: AI - Purple Team

AI Purple Team Milestone

Even tough we were not quite hooked by the current hype around AI during the predecessor project 1.2, we now see a huge potential in integrating it into the NSAK-framework, especially in because of the tool-calling capabilities of agentic AI. In our point of view, the possibility that common blue and red team activities, which require a lot of know how and time, can be simplified or speed up is really promising. This scenarios would explore if it’s possible that intrusion detection or network reconnaissance tasks can be assisted or maybe even fully automated by an AI-agent.

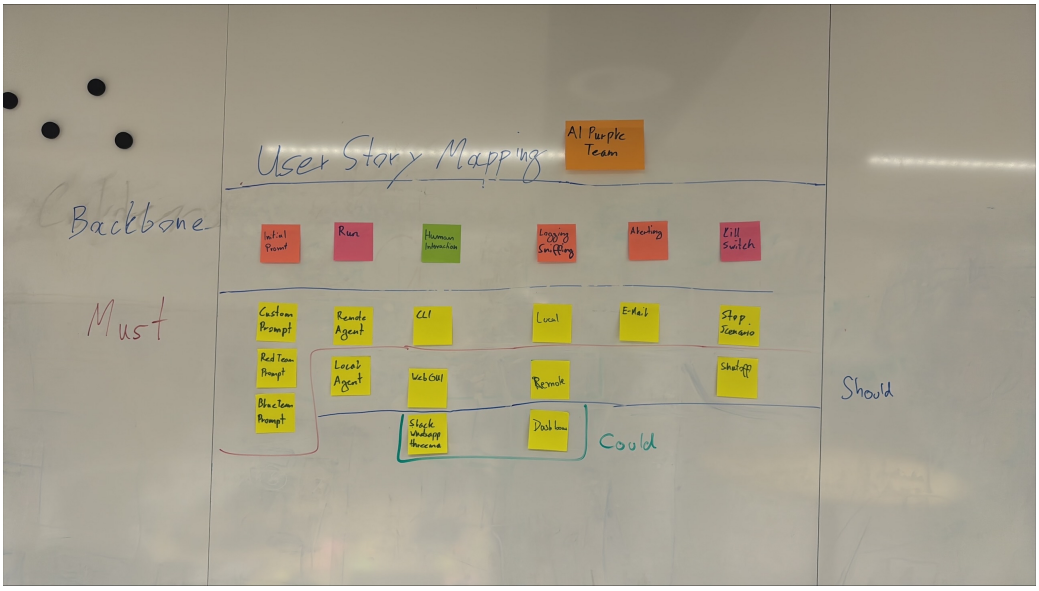


Figure A.32.: NSAK Scenario: AI - Purple Team

A.12. Self-Hosted LLM

During the risk assessment A.7.8 we identified that the AI tokens required to use cloud based LLMs might be very expensive, and it's really hard to estimate the costs. One of the possibilities to avoid this risk was the usage of self-hosted LLMs, and luckily we already had existing hardware at home which is suitable for running smaller LLMs. Besides eliminating the costs for AI tokens, at least during the development phase, it also allows the quick evaluation of different AI Models. Depending on the quality and performance of the smaller models running in the self-hosted setup we can consider not using a cloud based LLM at all.

Refactor the section: Move all information in their respective subsections, etc.

A.12.1. Hardware Setup

Component	Specification
CPU	AMD Ryzen 5 3600 (6 cores / 12 threads, up to 4.2 GHz)
GPU	NVIDIA GeForce RTX 3070 (8 GB VRAM)
System Memory	32 GB RAM

Table A.22.: Hardware configuration of the self-hosted LLM inference server.

The 8 GB VRAM of the RTX 3070 allows us to run models with up to 7 billion parameters, depending on the specific model [24].

A.12.2. System Setup

The system setup described in **??** will configure and expose the Ollama API and OpenWeb UI via NGINX, which provided a chat GUI to interact with Ollama. In contrast to Ollama, which runs directly on the host, the other services are run in containers managed by rootles podman and defined the docker-compose.yml **??**. The ?? default.conf **??** configures the web server to terminate the SSL connection and act as reverse proxy for the "Open WebUI" and the Ollama API under the directory `/llm/`.

Services:

- Ollama: LLM manager, exposing an API for prompts and other interactions (host).
- Swag: ?? web server with Certbot for automated SSL certificate generation (containerized).
- Open WebUI: Web based chat interface for LLM managers (containerized).

System Hardening

It is recommended to put a proper firewall before the services we want to expose to the internet, but for our purposes we can just disable the port forwarding when we don't use the AI. It still makes sense to perform a minimal system hardening as shown in the following listing 33, before exposing the services to the internet.

```
# Install UFW
sudo pacman -Syu ufw
sudo ufw status

# Allow SSH for remote access
sudo ufw allow ssh

# Allow HTTP for certbot HTTP challange
sudo ufw allow http

# Allow HTTPS for Open WebUI
sudo ufw allow https

# Allow default Ollama port
sudo ufw allow 11434/tcp

# Make sure to run this command after adding the "allow" rules!
sudo ufw enable

sudo ufw status
```

Listing 33: UFW configuration

Expose Open WebUI and Ollama

For exposing the services from the home network via HTTPS to the internet, we need a domain name which points to the home router. To achieve this we can use DynDNS, which is provided for free by Infomaniak where the domain "hiube.ch" was registered, and is supported by the FRITZ!Box used in the home network. The screenshots A.34 and A.33 are showing the respective configurations.

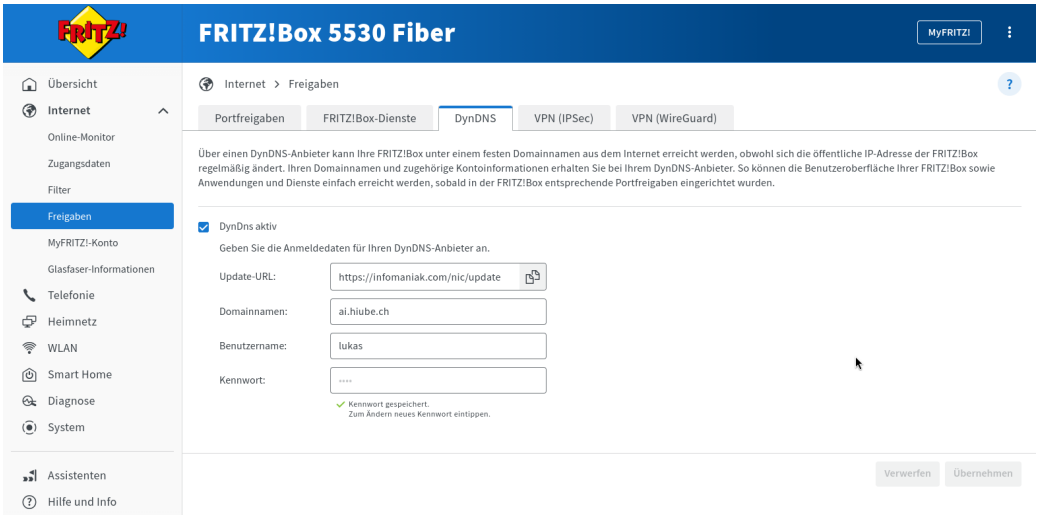


Figure A.33.: FRITZ!Box DynDNS Setup

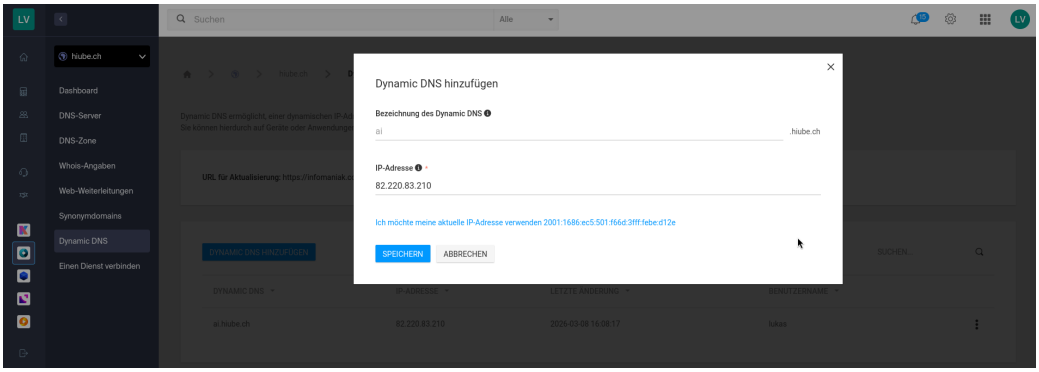


Figure A.34.: Infomaniak DynDNS Setup

Now that the domain name “ai.hiube.ch” points to the router in the home network, we need to redirect the HTTP(S) traffic to the LLM server. This can be achieved with port forwarding rules on the FRITZ!Box, as shown in the screenshot A.35.

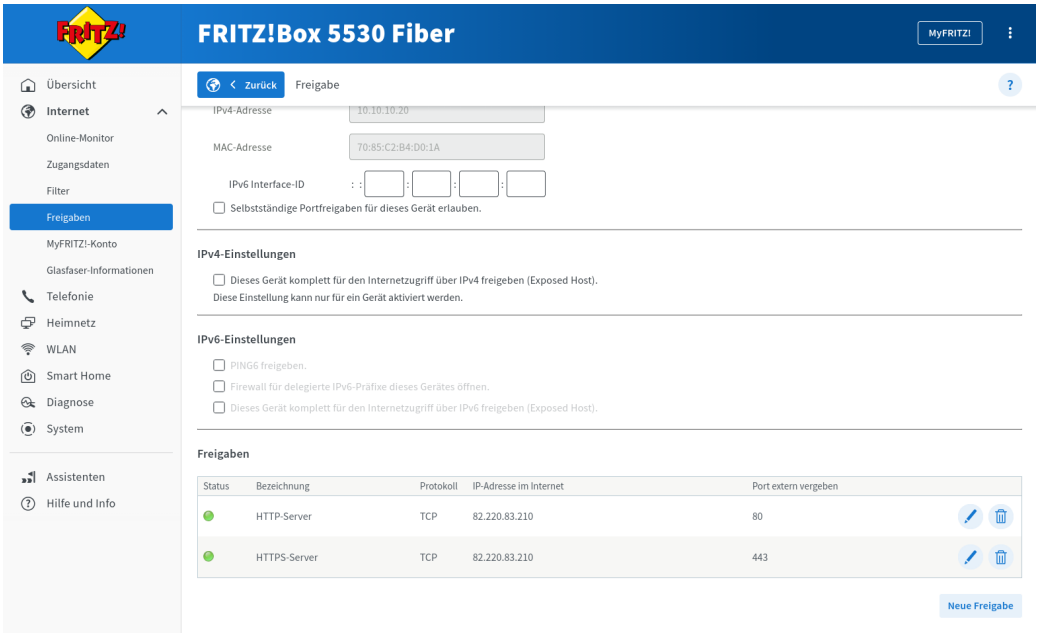


Figure A.35.: FRITZ!Box DynDNS Setup

The Open WebUI and Ollama API should already be reachable via “ai.hiube.ch”, but we will receive a security warning because there is no valid SSL certificate. The easiest way to generate the certificate, is to trigger certbot by restarting the “swag” service as shown in the listing below 34.

```
# Login as user ai and restart the swag service
podman-compose restart swag
```

Listing 34: Restart SWAG Container

A.12.3. Evaluation

For quickly evaluate models and assess their capabilities we can now use the Open WebUI served under “ai.hiube.ch”. The screenshot A.36 shows a quick test with “qwen2.5-coder:7B”.

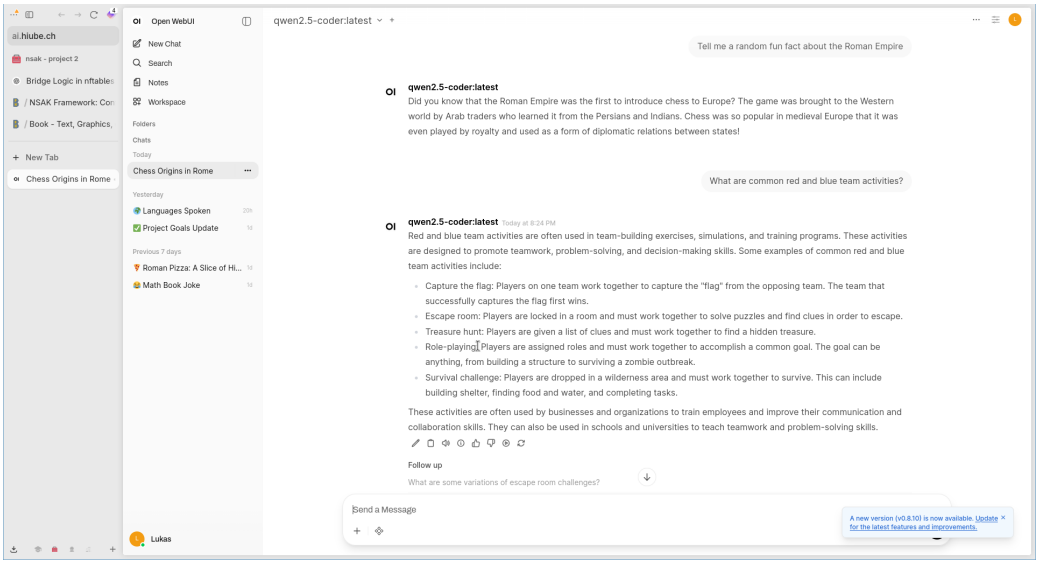


Figure A.36.: FRITZ!Box DynDNS Setup

Later on we can use the Ollama API served under `/llm/` as a backend for AI agents.

A.12.4. Self-Hosted Base Setup

```
# Login as ai user
ssh ai@10.10.10.20

# Create a docker compose file and add contents from
↪ lst:nginx-docker-compose-yml
vim docker-compose.yml

# Start the services, so that they create the volume mappings
↪ etc.
# The containers will most likely fail, but that's fine for now.
podman-compose up -d
podman-compose down
```

Listing 35: Self-Hosted Base Setup

```
# Login as ai user
ssh ai@10.10.10.20

# Create a docker compose file and add contents from lst:nginx-docker-compose-yml
vim docker-compose.yml

# Start the services, so that they create the volume mappings etc.
# The containers will most likely fail, but that's fine for now.
podman-compose up -d
podman-compose down
```

Listing 36: Nginx & Let's Encrypt Instructions

A.12.5. Nginx & Let's Encrypt - Reverse Proxy with SSL Termination

```yaml
services:
  swag:
    image: lscr.io/linuxserver/swag:latest
    container_name: swag
    cap_add:
      - NET_ADMIN
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=Europe/Zurich
      - URL=hiube.ch
      - VALIDATION=http
      - SUBDOMAINS=ai,
      - EMAIL=<my-email>
      - ONLY_SUBDOMAINS=true
      - EXTRA_DOMAINS= #optional
      - STAGING=false #optional
      - DISABLE_F2B= #optional
      - SWAG_AUTORELOAD= #optional
      - SWAG_AUTORELOAD_WATCHLIST= #optional
    volumes:
      - /home/ai/swag:/config
    ports:
      - 443:443
      - 80:80
      - 443:443/udp
    restart: unless-stopped
```

Listing 37: Nginx & Let's Encrypt Compose File

A.12.6. Ollama - LLM Server

```
ssh ai@10.10.10.20

# Check nvidia driver is loaded and the GPU was correctly detected, otherwise install the according drivers
nvidia-smi

# Install Ollama and enable systemd service
sudo pacman -Syu ollama
sudo systemctl enable --now ollama
sudo systemctl status ollama

# Download and run a qwen coder 7b model for testing
# Later used models:
# - `qwen2.5:7b-instruct` (tool calling)
# - `gemma4:e2b` (was hyped, but is not that good in tool calling)
ollama pull qwen2.5-coder:7b
ollama run qwen2.5-coder

# Open the nginx defalult.conf and add the contents from lst:ollama-nginx-conf
vim swag/nginx/site-confs/default.conf

# Start the services, this time they should run successfully.
# For a local setup, certbot will fail to perform the http challenge for issuing the TLS certificate,
# as long as DynDNS and the port forwarding are not set up properly.
podman-compose up -d
```

Listing 38: Ollama Instructions

```
# Redirect all traffic to HTTPS
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name ai.hiube.ch;

    location / {
        return 301 https://$host$request_uri;
    }
}

# Using upstream enables us to restart ollama without restarting the nginx
upstream ollama {
    server host.docker.internal:11434;
}

server {
    listen 43434 ssl;
    listen [::]:43434 ssl;

    server_name ai.hiube.ch;

    include /config/nginx/ssl.conf;

    location / {
        auth_basic "Restricted";
        auth_basic_user_file /config/nginx/.htpasswd;

        proxy_pass http://ollama;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        chunked_transfer_encoding off;
        proxy_buffering off;
        proxy_cache off;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 600s;
        proxy_send_timeout 600s;
    }
}
```


Listing 39: Ollama Nginx Configuration

```
# Enable Ollama debug logs

During the development of the AI intrusion detection scenario, Ollama suddenly sent an empty response terminating the agent.
For investigation, I enabled the debug logs on the AI server.

```bash
sudo systemctl edit --full ollama.service

# Add the following line
Environment="OLLAMA_DEBUG=2"

# Restart the service
sudo systemctl restart ollama.service

# The logs can now be listed with:
journalctl -f -b -u ollama
```

Container logs of the empty Ollama response:

-> The last log indicates a container restart (scenario was terminated by empty response)

```bash
❯ sudo podman logs -f simple_tcp_client_server_ai_intrusion_detection_1
WARNING:nsak.core.scenario.scenario_manager:EXEC SCENARIO: AI Intrusion Detection
INFO:root:Starting scenario: AI Intrusion Detection
WARNING:nsak.core.drill.drill_manager:EXEC DRILL: Discover Hosts
DEBUG:httpcore.connection:connect_tcp.started host='ai.hiube.ch' port=43434 local_address=None timeout=None socket_options=None
DEBUG:httpcore.connection:connect_tcp.complete return_value=<httpcore._backends.sync.SyncStream object at 0x7f6434e86f90>
DEBUG:httpcore.connection:start_tls.started ssl_context=<ssl.SSLContext object at 0x7f6434eff4d0> server_hostname='ai.hiube.ch' timeout=None
DEBUG:httpcore.connection:start_tls.complete return_value=<httpcore._backends.sync.SyncStream object at 0x7f6434eab110>
DEBUG:httpcore.http11:send_request_headers.started request=<Request [b'POST']>
DEBUG:httpcore.http11:send_request_headers.complete
DEBUG:httpcore.http11:send_request_body.started request=<Request [b'POST']>
DEBUG:httpcore.http11:send_request_body.complete
DEBUG:httpcore.http11:receive_response_headers.started request=<Request [b'POST']>
DEBUG:httpcore.http11:receive_response_headers.complete
```
```

A.12.7. Open WebUI - Web Chat for LLM Servers

```
ssh ai@10.10.10.20

# Open the nginx defalult.conf and add the contents from
↪  lst:open-webui-nginx-conf
vim swag/nginx/site-confs/default.conf

# Start the services
podman-compose up -d
```

Listing 41: Open WebUI Instructions

```yaml
services:
  # ...

  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    container_name: open-webui
    volumes:
      - /home/ai/open_webui:/app/backend/data # Use a named
      ↪  volume for persistence
    environment:
      OLLAMA_BASE_URL: http://host.docker.internal:11434
    restart: always
```

Listing 42: Open WebUI Compose File

```
# Open WebUI
upstream open_webui {
    server open-webui:8080;
}

server {
    listen 443 ssl default_server;
    listen [::]:443 ssl default_server;

    server_name ai.hiube.ch;

    include /config/nginx/ssl.conf;

    location / {
        proxy_pass http://open_webui;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
        ↪  $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_read_timeout 900s;
        proxy_send_timeout 900s;
    }
}
```

Listing 43: Open WebUI Nginx Configuration

A.12.8. Drawio MCP - Diagram Tool

```
ssh ai@10.10.10.20

# https://www.drawio.com/blog/diagrams-docker-app
echo "unqualified-search-registries = [\"docker.io\"]" >> /etc/containers/registries.conf

# Add drawio.hiube.ch CNAME entry to ai.hiube.ch
# Add drawio.hiube.ch to linuxserver.io swag config for letsencrypt and add the drawio service from below
vim docker-compose.yaml

# Update nginx conifg
sudo vim swag/nginx/site-confs/default.conf

# Download drawio-mcp
git clone https://github.com/lgazo/drawio-mcp-server.git

# Restart services
podman-compose restart swag
```

Listing 44: Drawio Instructions

```yaml
services:
  # ...

  drawio-mcp-server:
    build: ./drawio-mcp-server
    container_name: drawio-mcp-server
    restart: unless-stopped
    command:
      - "node"
      - "packages/drawio-mcp-server/build/index.js"
      - "--transport"
      - "http"
      - "--editor"
      - "--http-port"
      - "3000"
```

Listing 45: Drawio Compose File

```
# Draw.io
upstream drawio {
    server drawio:8080;
}

upstream drawio_mcp {
    server drawio-mcp-server:3000;
}

upstream drawio_ws {
    server drawio-mcp-server:3333;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;

    server_name drawio.hiube.ch;

    include /config/nginx/ssl.conf;

    location / {
        auth_basic "Restricted";
        auth_basic_user_file /config/nginx/.htpasswd;

        proxy_pass http://drawio;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
        ↪  $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }

    location /mcp {
        auth_basic "Restricted";
        auth_basic_user_file /config/nginx/.htpasswd;

        proxy_pass http://drawio_mcp/mcp;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
        ↪  $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_buffering off;
        proxy_cache off;
        proxy_read_timeout 3600s;
        proxy_send_timeout 3600s;
        chunked_transfer_encoding on;
```

A.12.9. Netbox - Network Inventory System

```
ssh ai@10.10.10.20

#
 ↪  https://learn.srlinux.dev/blog/2024/creating-a-network-digital-twin-with-co
# https://github.com/netbox-community/netbox-docker
git clone -b release
 ↪  https://github.com/netbox-community/netbox-docker.git
cd netbox-docker/

cp docker-compose.override.yml.example
 ↪  docker-compose.override.yml

podman-compose up -d

podman-compose exec netbox /opt/netbox/netbox/manage.py
 ↪  createsuperuser

cd ..

# Add netbox.hiube.ch CNAME entry to ai.hiube.ch
# Add netbox.hiube.ch to linuxserver.io swag config for
 ↪  letsencrypt
vim docker-compose.yaml

# Update nginx conifg
sudo vim swag/nginx/site-confs/default.conf

# Restart services
podman-compose restart swag
```

Listing 47: Netbox Instructions

```
1
```

Listing 48: Netbox Compose File

```
# Netbox
upstream netbox {
    server host.docker.internal:8000;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;

    server_name netbox.hiube.ch;

    include /config/nginx/ssl.conf;

    location / {
        proxy_pass http://netbox;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
        ↪  $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Listing 49: Netbox Nginx Configuration

A.12.10. Grafana / Loki - Logging System

```
ssh ai@10.10.10.20

#
 ↪  https://grafana.com/docs/loki/latest/setup/install/docker/#install-with-doc
mkdir loki
wget
 ↪  https://raw.githubusercontent.com/grafana/loki/v3.7.0/examples/getting-star
 ↪  -O loki/docker-compose.yaml
wget
 ↪  https://raw.githubusercontent.com/grafana/loki/v3.7.0/examples/getting-star
 ↪  -O loki/alloy-local-config.yaml
wget
 ↪  https://raw.githubusercontent.com/grafana/loki/v3.7.0/examples/getting-star
 ↪  -O loki/loki-config.yaml

# Adjustments needed for rootles podman
systemctl --user enable --now podman.socket

# Adjust the alloy service described in the loki `compose.yaml`
vim loki/docker-compose.yaml

# Add grafana.hiube.ch CNAME entry to ai.hiube.ch
# Add grafana.hiube.ch to linuxserver.io swag config for
 ↪  letsencrypt
vim docker-compose.yaml

# Update nginx conifg
sudo vim swag/nginx/site-confs/default.conf

# Restart services
podman-compose up -d
```

Listing 50: Grafana / Loki Instructions

```yaml
# ~/loki/docker-compose.yaml
networks:
  loki:

services:
  read:
    image: grafana/loki:latest
    command: "-config.file=/etc/loki/config.yaml -target=read"
    ports:
      - 127.0.0.1:3101:3100
      - 7946
      - 9095
    volumes:
      - ./loki/loki-config.yaml:/etc/loki/config.yaml
    depends_on:
      - minio
    healthcheck:
      test: [ "CMD", "/usr/bin/loki", "-health" ]
      start_period: 30s
      interval: 10s
      timeout: 5s
      retries: 5
    networks: &loki-dns
      loki:
        aliases:
          - loki

  write:
    image: grafana/loki:latest
    command: "-config.file=/etc/loki/config.yaml -target=write"
    ports:
      - 127.0.0.1:3102:3100
      - 7946
      - 9095
    volumes:
      - ./loki/loki-config.yaml:/etc/loki/config.yaml
    healthcheck:
      test: [ "CMD", "/usr/bin/loki", "-health" ]
      start_period: 30s
      interval: 10s
      timeout: 5s
      retries: 5
    depends_on:
      - minio
    networks:
      <<: *loki-dns

  alloy:
    image: grafana/alloy:latest
    volumes:
      -
      ↪  ./loki/alloy-local-config.yaml:/etc/alloy/config.alloy:ro
      - /run/user/1001/podman/podman.sock:/var/run/docker.sock
    command:  run --server.http.listen-addr=0.0.0.0:12345
```

```
# Grafana / Loki
upstream grafana {
    server grafana:3000;
}
upstream loki-read {
    server read:3100;
}
upstream loki-write {
    server write:3100;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;

    server_name grafana.hiube.ch;

    include /config/nginx/ssl.conf;

    auth_basic "Restricted";
    auth_basic_user_file /config/nginx/.htpasswd;

    location / {
        auth_basic off;

        proxy_pass http://grafana;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
        ↪  $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        proxy_read_timeout 900s;
        proxy_send_timeout 900s;
    }

    location = /api/prom/push {
        proxy_pass       http://loki-write$request_uri;
    }

    location = /api/prom/tail {
        proxy_pass       http://loki-read$request_uri;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
    }

    location ~ /api/prom/.* {
        proxy_pass       http://loki-read$request_uri;
    }

    location = /loki/api/v1/push {
        proxy_pass       http://loki-write$request_uri;
```

```yaml
# ~/loki/loki-config.yaml
auth_enabled: false

server:
  http_listen_address: 0.0.0.0
  http_listen_port: 3100

memberlist:
  join_members: ["read", "write", "backend"]
  dead_node_reclaim_time: 30s
  gossip_to_dead_nodes_time: 15s
  left_ingesters_timeout: 30s
  bind_addr: ['0.0.0.0']
  bind_port: 7946
  gossip_interval: 2s

schema_config:
  configs:
    - from: 2023-01-01
      store: tsdb
      object_store: s3
      schema: v13
      index:
        prefix: index_
        period: 24h
common:
  path_prefix: /loki
  replication_factor: 1
  compactor_address: http://backend:3100
  storage:
    s3:
      endpoint: minio:9000
      insecure: true
      bucketnames: loki-data
      access_key_id: loki
      secret_access_key: supersecret
      s3forcepathstyle: true
  ring:
    kvstore:
      store: memberlist
ruler:
  storage:
    s3:
      bucketnames: loki-ruler
```



```yaml
compactor:
  working_directory: /tmp/compactor
  retention_enabled: true
  delete_request_store: s3

limits_config:
  retention_period: 8760h # 1 year
  retention_stream:
    - selector: '{job="container_logs"}'
```

```yaml
# ~/loki/loki-config.yaml
auth_enabled: false

server:
  http_listen_address: 0.0.0.0
  http_listen_port: 3100

memberlist:
  join_members: ["read", "write", "backend"]
  dead_node_reclaim_time: 30s
  gossip_to_dead_nodes_time: 15s
  left_ingesters_timeout: 30s
  bind_addr: ['0.0.0.0']
  bind_port: 7946
  gossip_interval: 2s

schema_config:
  configs:
    - from: 2023-01-01
      store: tsdb
      object_store: s3
      schema: v13
      index:
        prefix: index_
        period: 24h
common:
  path_prefix: /loki
  replication_factor: 1
  compactor_address: http://backend:3100
  storage:
    s3:
      endpoint: minio:9000
      insecure: true
      bucketnames: loki-data
      access_key_id: loki
      secret_access_key: supersecret
      s3forcepathstyle: true
  ring:
    kvstore:
      store: memberlist
ruler:
  storage:
    s3:
      bucketnames: loki-ruler
```



```yaml
compactor:
  working_directory: /tmp/compactor
  retention_enabled: true
  delete_request_store: s3

limits_config:
  retention_period: 8760h # 1 year
  retention_stream:
    - selector: '{job="container_logs"}'
```

A.13. Writing Quality

Introduced from Professor David Stuckler the PEER-System: Simple and clear academic writing.

The structure is:

- P 1 POINT just one
- E Evidence and Example 1-n
- E Explanation
- R Repeat linking Links and leads to next topic

«««< Updated upstream

A.14. Test-Environment-Code

A.14.1. container-lab

```
  # ── Firewall (Alpine + nftables) ──────────────────────────────────────
  # Tri-homed router/firewall: eth1=WAN, eth2=DMZ, eth3=LAN.
  # All inter-zone traffic is filtered here.
  # ip_forward enables kernel-level packet routing between interfaces.
  firewall:
    kind: linux
    image: alpine:3.19
    cmd: tail -f /dev/null
    cap-add:
      - NET_ADMIN
    sysctls:
      net.ipv4.ip_forward: 1  # required to forward packets between interfaces
    exec:
      - apk add --no-cache nftables iproute2
      - ip addr add 10.0.1.1/24 dev eth1
      - ip addr add 172.16.1.1/24 dev eth2
      - ip addr add 192.168.10.1/24 dev eth3
      - ip route replace default via 10.0.1.254
      # Default FORWARD policy: drop everything not explicitly allowed
      - nft add table inet filter
      - sh -c "nft add chain inet filter forward '{ type filter hook forward priority 0; policy drop; }'"
      # Allow return traffic for already-established connections (stateful firewall)
      - nft add rule inet filter forward ct state established,related accept
      # WAN → DMZ: only HTTP/HTTPS and DNS to dmz-server
      - sh -c "nft add rule inet filter forward iifname eth1 oifname eth2 ip daddr 172.16.1.10 tcp dport
      ↪  '{ 80, 443 }' accept"
      - nft add rule inet filter forward iifname eth1 oifname eth2 ip daddr 172.16.1.10 udp dport 53
      ↪  accept
      - nft add rule inet filter forward iifname eth1 oifname eth2 ip daddr 172.16.1.10 tcp dport 53
      ↪  accept
      # WAN → LAN: completely blocked (no rules = default DROP applies)
      # DMZ → LAN: completely blocked
      - nft add rule inet filter forward iifname eth2 oifname eth3 drop
      # LAN → DMZ: allowed (internal users browse DMZ services)
      - nft add rule inet filter forward iifname eth3 oifname eth2 accept
      # LAN → WAN: HTTP/HTTPS only, with source NAT (masquerade)
      - sh -c "nft add rule inet filter forward iifname eth3 oifname eth1 tcp dport '{ 80, 443 }' accept"
      # NAT: source masquerade for LAN and DMZ egress to WAN
      - nft add table ip nat
      - sh -c "nft add chain ip nat postrouting '{ type nat hook postrouting priority 100; }'"
      - nft add rule ip nat postrouting ip saddr 192.168.10.0/24 oifname eth1 masquerade
      - nft add rule ip nat postrouting ip saddr 172.16.1.0/24 oifname eth1 masquerade
      # DMZ → WAN: DNS forwarding (dmz-server forwards unknown queries to 8.8.8.8)
      - nft add rule inet filter forward iifname eth2 oifname eth1 udp dport 53 accept
      - nft add rule inet filter forward iifname eth2 oifname eth1 tcp dport 53 accept
```

Listing 55: Firewall

Listing 55 shows the nf-tabels rules set for segmentation of wan, dmz and lan.

```
# ──────────────────────────────────────────────────────────────────────
#  DMZ Nodes  (172.16.1.0/24)
# ──────────────────────────────────────────────────────────────────────

# ── DMZ: Combined Web + DNS Server ──────────────────────────────────
# Single DMZ node running both nginx (web) and BIND9 (DNS).
# Merging saves a device slot while covering two distinct attack surfaces.
#
# Web (nginx):
#   80/tcp   HTTP  — corporate landing page, /robots.txt, /api/v1/status
#   443/tcp  HTTPS — self-signed cert: CN=dmz-server.lab.local
#   Recon: Server header leaks version; /robots.txt disallows /admin /backup
#
# DNS (BIND9):
#   53/udp+tcp  queries and zone transfer
#   AXFR intentionally open (misconfiguration) — reveals all internal IPs
#   Recon: dig @172.16.1.10 lab.local AXFR  → full zone dump
dmz-server:
  kind: linux
  image: alpine:3.19
  cmd: tail -f /dev/null
  cap-add:
    - NET_ADMIN
  binds:
    - scripts/dmz-web-server:/dmz-web-server:ro
  exec:
    - apk add --no-cache nginx bind bind-tools iproute2 openssl python3
    - ip addr add 172.16.1.10/24 dev eth1
    - ip route replace default via 172.16.1.1
    - sh /dmz-web-server/setup.sh
```

Listing 56: Web-Server and DNS

Listing 56 shows a web server and DNS server node in the Containerlab YAML configuration. This node is located in the demilitarized zone (DMZ) of the network.

```
# ──────────────────────────────────────────────────────────────
#  LAN Nodes  (192.168.10.0/24)
# ──────────────────────────────────────────────────────────────

# ── LAN: Internal Server (Samba + LDAP + SSH) ──────────────────
# Combines Domain Controller and File Server into one node.
# Samba provides SMB shares; OpenLDAP provides a lightweight directory.
# No full AD provisioning — covers the same recon/attack surface at less cost.
#
#   22/tcp   SSH   (admin / Password123!)
#   139/tcp  NetBIOS (Samba)
#   389/tcp  LDAP  — anonymous base read: DC=lab,DC=local (user list exposed)
#   445/tcp  SMB   — shares: public (anon read), finance (auth), it (auth)
#
# Recon:
#   smbclient -L //192.168.10.5 -N         → lists all shares
#   smbmap -H 192.168.10.5                 → share names + permissions
#   ldapsearch -x -H ldap://192.168.10.5   → user list, group memberships
ctrl-server:
  kind: linux
  image: alpine:3.19
  cmd: tail -f /dev/null
  cap-add:
    - NET_ADMIN
    - SYS_ADMIN  # needed for Samba AD provisioning
  binds:
    - scripts/ctrl-file-server:/ctrl-file-server:ro
  exec:
    - apk add --no-cache samba openldap openldap-back-mdb openldap-clients openssh iproute2
    - ip addr add 192.168.10.5/24 dev eth1
    - ip route replace default via 192.168.10.1
    - sh -c 'echo "nameserver 172.16.1.10" > /etc/resolv.conf'
    - sh /ctrl-file-server/server_setup.sh
```

Listing 57: File Server and Domain Control

The LAN segment consists of four nodes, as shown in the following code snippets.

```
  # ── LAN: Workstation alice (Finance dept, user: asmith) ──────────────────
  # Simulates a Finance department workstation.
  # Sensitive documents on disk; drive mappings reveal share paths.
  #
  #   22/tcp   SSH  (asmith / Password123!)
  #   445/tcp  SMB  (Samba standalone)
  #
  # Recon:
  #   SSH banner leaks org name and policy
  #   /home/asmith/Documents/Q3_Finance.txt — confidential data (post-exploit)
  #   /home/asmith/Desktop/drive_mappings.txt — reveals \\ctrl-server\finance path
  alice:
    kind: linux
    image: alpine:3.19
    cmd: tail -f /dev/null
    cap-add:
      - NET_ADMIN
    exec:
      - apk add --no-cache openssh samba iproute2
      - ip addr add 192.168.10.100/24 dev eth1
      - ip route replace default via 192.168.10.1
      - sh -c 'echo "nameserver 172.16.1.10" > /etc/resolv.conf'
      - ssh-keygen -A
      - sh -c 'echo "PasswordAuthentication yes" >> /etc/ssh/sshd_config'
      - sh -c 'printf "NSAK-Enterprise - Authorized Access Only\nThis system is monitored.\n" >
      ↪  /etc/ssh/banner.txt'
      - sh -c 'echo "Banner /etc/ssh/banner.txt" >> /etc/ssh/sshd_config'
      - adduser -D -s /bin/sh asmith
      - sh -c 'echo "asmith:Password123!" | chpasswd'
      - mkdir -p /home/asmith/Documents /home/asmith/Desktop
      - sh -c 'echo "Q3 2024 Finance Report - CONFIDENTIAL" > /home/asmith/Documents/Q3_Finance.txt'
      - sh -c 'printf "DOMAIN=LAB\nUSER=asmith\nSHARE=\\\\\\\\ctrl-server\\\\finance\n" >
      ↪  /home/asmith/Desktop/drive_mappings.txt'
      - /usr/sbin/sshd

  # ── LAN: Workstation bob (IT dept, user: bjones) ─────────────────────────
  # Simulates an IT department workstation.
  # bjones has elevated rights — scripts directory reveals full internal topology.
  #
  #   22/tcp   SSH  (bjones / Password123!)
  #   445/tcp  SMB  (Samba standalone)
  #
  # Recon:
  #   /home/bjones/Scripts/hosts.txt  — lists all internal IPs and roles
  #   /home/bjones/Scripts/README.txt — hints at admin access
  bob:
    kind: linux
    image: alpine:3.19
    cmd: tail -f /dev/null
    cap-add:
      - NET_ADMIN
    exec:
      - apk add --no-cache openssh samba iproute2
      - ip addr add 192.168.10.101/24 dev eth1
      - ip route replace default via 192.168.10.1
      - sh -c 'echo "nameserver 172.16.1.10" > /etc/resolv.conf'
      - ssh-keygen -A
      - sh -c 'echo "PasswordAuthentication yes" >> /etc/ssh/sshd_config'
      - sh -c 'printf "Acme Corp AG - Authorized Access Only\nThis system is monitored.\n" >
      ↪  /etc/ssh/banner.txt'
      - sh -c 'echo "Banner /etc/ssh/banner.txt" >> /etc/ssh/sshd_config'
      - adduser -D -s /bin/sh bjones
      - sh -c 'echo "bjones:Password123!" | chpasswd'
      - mkdir -p /home/bjones/Documents /home/bjones/Scripts
      - sh -c 'printf "ctrl-server=192.168.10.5\nalice=192.168.10.100\nprinter=192.168.10.50\n" >
      ↪  /home/bjones/Scripts/hosts.txt'
      - sh -c 'echo "# admin tooling — see ctrl-server at 192.168.10.5" >
      ↪  /home/bjones/Scripts/README.txt'
      - /usr/sbin/sshd
```

Listing 58: Work-Stations Bob and Alice

```
  # ── LAN: Network Printer (HP-like, intentionally misconfigured) ──────────────
  # Simulates a typical enterprise network printer — the "forgotten device".
  # Rarely patched, often misconfigured, rich source of OSINT and lateral movement.
  #
  #   80/tcp    HTTP  — unauthenticated web admin interface
  #   161/udp   SNMP  — community string "public" (v1/v2c, full device info)
  #   631/tcp   IPP   — Internet Printing Protocol (job history, no auth)
  #
  # Recon:
  #   snmpwalk -v2c -c public 192.168.10.50  → model, serial, toner, job list, usernames
  #   curl http://192.168.10.50              → unauthenticated admin page, firmware version
  #   curl http://192.168.10.50:631/jobs     → print job history reveals internal usernames
  printer:
    kind: linux
    image: alpine:3.19
    cmd: tail -f /dev/null
    cap-add:
      - NET_ADMIN
      - NET_BIND_SERVICE  # needed to bind to privileged ports 80 and 631
    binds:
      # printer_sim.py serves the web admin UI (port 80) and IPP (port 631)
      - scripts/printer_sim.py:/printer_sim.py:ro
    exec:
      - apk add --no-cache python3 net-snmp iproute2
      - ip addr add 192.168.10.50/24 dev eth1
      - ip route replace default via 192.168.10.1
      - sh -c 'echo "nameserver 172.16.1.10" > /etc/resolv.conf'
      # SNMP with open public community string — intentional misconfiguration
      - sh -c 'printf "rocommunity public\nsysName HP-LaserJet-M428fdw\nsysLocation Server Room
      ↪  B2\nsysContact it@lab.local\n" > /etc/snmp/snmpd.conf'
      - sh -c 'snmpd -f -Lo &'
      # Web admin UI (80) and IPP job listener (631)
      - sh -c 'python3 /printer_sim.py > /var/log/printer_sim.log 2>&1 &'
```

Listing 59: Printer

Listing ?? illustrates two workstations, Alice and Bob, the printer shown in Listing 59, and the domain controller with Samba file server shown in Listing 57.

```
      - ip addr add 172.16.1.200/24   dev eth2
      - ip route replace default via 172.20.20.1 dev eth0
      - sh -c 'echo "nameserver 8.8.8.8" > /etc/resolv.conf'
    mgmt-ipv4: 172.20.20.10

links:
  # WAN segment (WAN 10.0.1.0/24)
  - endpoints: [ "firewall:eth1", "wan-br:fw1" ]
  - endpoints: [ "external:eth1" , "wan-br:ext" ]
  # DMZ segment (172.16.1.0/24)
  - endpoints: [ "firewall:eth2", "dmz-br:fw2" ]
  - endpoints: [ "dmz-server:eth1", "dmz-br:dmz-serv" ]
  - endpoints: [ "nsak:eth2", "dmz-br:nsak-dmz" ]
  # LAN segment (192.168.10.0/24)
  - endpoints: [ "firewall:eth3", "lan-br:fw3" ]
```

Listing 60: Links

Listing 60 shows the connection between endpoints over the defined interfaces.

A.14.2. Flaws and Vulnerabilities

```
DMZ --- dmz-server (172.16.1.10)

DNS / BIND9
- allow-transfer { any; } --- AXFR (zone transfer) open to everyone
- dnssec-validation no --- no DNSSEC validation
- Zone contains internal LAN addresses (ctrl, alice, bob, printer)

Web / nginx
- /secret.txt accessible without auth -> returns DB Password:
↪  {flag:exposed-SOS}
- Self-signed certificate (CN=dmz-server.vlab.local)
---
LAN --- ctrl-server (192.168.10.5)

LDAP / OpenLDAP
- by anonymous read --- anonymous read access to all entries -> user list,
↪  emails, departments
- rootpw Password123! stored in plaintext in config
- User passwords stored in LDIF: userPassword: Password123!
ldapsearch -x -H ldap://192.168.10.5 -b "dc=lab,dc=local"

SMB / Samba
- ntlm auth = yes + lanman auth = yes --- outdated, weak auth protocols
- Share public anonymously readable (no auth) -> README.txt hints at bjones
- Share finance contains salary and budget data (asmith / Password123!)
- Share it contains dc01_build_sheet.txt with full infrastructure details
- All passwords: Password123!
---
LAN --- alice (192.168.10.100) \& bob (192.168.10.101)

- SSH password auth enabled, weak password Password123!
- SSH banner leaks organisation name
- alice: /home/asmith/Documents/Q3_Finance.txt (confidential),
↪  /home/asmith/Desktop/drive_mappings.txt (share paths)
- bob: /home/bjones/Scripts/hosts.txt (all internal IPs + roles), README.txt
↪  hints at admin access
---
LAN --- printer (192.168.10.50)

- Port 80: unauthenticated web admin interface, leaks firmware version
↪  (HP-WebServer/2.6.5), serial number, location, contact
- Port 631 /jobs: print job history without auth -> reveals usernames
↪  (asmith, bjones, admin) and document names
(server_credentials.txt, payroll)
- SNMP community string public (v1/v2c) -> snmpwalk exposes full device info
---
Firewall policy (intentional gaps)

- LAN -> DMZ fully open (no port restriction) --- LAN users can reach all
↪  DMZ ports, not just 80/443
- DMZ -> WAN: DNS forwarding to 8.8.8.8 allowed -> DNS exfiltration possible
```

Listing 61: Introduced Vulnerabilities in the Virtual Enterprise Network

=======

A.15. Reconnaissance Scenario: Source Listings

```
"""
Scenario entrypoint for Reconnaissance .
"""
from scapy.arch import get_if_addr

from nsak.core import DrillManager
from nsak.core.network import NetworkDiscoveryResultMap


def _print_enumeration_findings(findings: dict[str, list[str]]) -> None:
    if not findings:
        print("\n### Service Enumeration: no findings ###\n")
        return
    print("\n### Service Enumeration Findings: ###\n")
    for endpoint, lines in findings.items():
        print(f"  {endpoint}:")
        for line in lines:
            print(f"    {line}")
    print("\n### ------------------------------ ###\n")


def run(interface: str | None = None, subnet: str | None = None) -> None:
    """
    Scenario, which runs Reconnaissance attack.
    1. If no interface is specified, scan all active physical interfaces
    2. If no interface is specified, scan all active physical interfaces
    3. Discover network on ifc and subnet
    :return: None
    """
    if interface is None:
        discovered = DrillManager.execute("discover_network_interfaces")
        interfaces_to_scan = [iface.name for iface in discovered]
    else:
        interfaces_to_scan = [interface]

    all_results = {}
    for iface_name in interfaces_to_scan:
        if get_if_addr(iface_name) in ("0.0.0.0", ""):
            DrillManager.execute("dhcp_request", interface=iface_name)

        result: NetworkDiscoveryResultMap = DrillManager.execute(
            "discover_hosts",
            interface=iface_name,
```

```yaml
drills:
  discover_hosts:
    name: Discover Hosts
    author: vonal3@bfh.ch
    repository:
     ↪  https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffe

    dependencies:
      system:
        - arp-scan
        - nmap
      python: [ ]

    interface:
      arguments:
        interface:
          type: str
        subnet:
          type: str
          default: null
      return_type: NetworkDiscoveryResultMap
```

Listing 63: Discover Hosts Drill: Resource Configuration

```python
import logging
import dataclasses
import subprocess
from ipaddress import IPv4Address, IPv6Address

from nsak.core import IPAddress
from nsak.core import config
from nsak.core.network import NetworkDiscoveryResultMap, NetworkDiscoveryResult, NetworkService, NetworkServiceEndpoint
from nsak.core.network.configuration import EthernetConfiguration

logger = logging.getLogger(__name__)


@dataclasses.dataclass(frozen=True, kw_only=True)
class ARPScanResult:
    ip: IPAddress
    mac: str
    vendor: str


def parse_arp_scan_result(arp_scan_result_output: str) -> list[ARPScanResult]:
    """

    :param arp_scan_result_output:
    :return:
    """

    entries = []
    for line in arp_scan_result_output.splitlines():
        parts = line.split(None, 2)
        if len(parts) < 2:
            continue

        raw_ip, mac = parts[:2]
        try:
            ip = IPv4Address(raw_ip)
        except ValueError:
            ip = IPv6Address(raw_ip)
        vendor = parts[2] if len(parts) == 3 else ""
        entries.append(ARPScanResult(ip=ip, mac=mac, vendor=vendor))

    return entries


def parse_nmap_result(nmap_output: str) -> list[ARPScanResult]:
    """
    Parses the output of `nmap -sn` into a list of ARPScanResults.
```

```python
import logging
import re
import subprocess
from typing import Literal

from nsak.core.network import NetworkDiscoveryResultMap, NetworkService, NetworkServiceEndpoint

logger = logging.getLogger(__name__)

# matches: "22/tcp open  ssh     OpenSSH 8.9p1"
_PORT_LINE = re.compile(r"^(\d+)/(tcp|udp)\s+open\s+(\S+)\s*(.*)") #compile regex once in obj (p,prot,name,version,


def parse_nmap(stdout: str) -> list[tuple[int, Literal["tcp", "udp"], str, str]]:
    """Extract open ports from nmap stdout as (port, protocol, name, version) tuples.

    :param stdout: Raw nmap output text.
    :return: List of (port, protocol, service name, version) tuples for open ports.
    """
    results = []
    # compare the output with the regex and skip if output does not match | group(0)=whole line, group(1)=port...)
    for line in stdout.splitlines():
        match = _PORT_LINE.match(line.strip())
        if match:
            port = int(match.group(1))
            protocol = match.group(2)
            name = match.group(3)
            version = match.group(4).strip()
            results.append((port, protocol, name, version))
    return results


def run(discovery_result: NetworkDiscoveryResultMap) -> NetworkDiscoveryResultMap:
    """Run nmap service scan on all target IPs and append open ports to the discovery result.

    :param discovery_result: Existing host discovery result containing target IPs per interface.
    :return: The same result map, with open port/service information.
    """
    for iface_name, result in discovery_result.results.items():
        mac_by_ip = {
            endpoint.ip: endpoint.mac
            for service in result.network_services
```

```
import logging
import re
import subprocess

from nsak.core.network import NetworkDiscoveryResultMap

logger = logging.getLogger(__name__)

_NSE_LINE = re.compile(r"^\|[_ ]?\s*(.*)")  # nmap NSE output: "| text" or "|_ text"

# Known services get targeted scripts; everything else falls back to "banner"
# (banner grabs the initial TCP response — gives software name + version for any unknown service)
_SERVICE_SCRIPTS: dict[str, list[str]] = {
    "http":          ["http-title", "http-headers", "http-robots.txt"],
    "https":         ["http-title", "http-headers", "http-robots.txt"],
    "http-alt":      ["http-title", "http-headers", "http-robots.txt"],
    "domain":        ["dns-zone-transfer", "dns-brute"],
    "smtp":          ["smtp-commands", "smtp-enum-users"],
    "smtps":         ["smtp-commands"],
    "ftp":           ["ftp-anon", "ftp-ls"],
    "netbios-ssn":   ["smb-security-mode", "smb2-security-mode"],
    "microsoft-ds": ["smb-security-mode", "smb2-security-mode"],
    "ldap":          ["ldap-rootdse"],
    "ldapssl":       ["ldap-rootdse"],
}


def _parse_nse_output(stdout: str) -> list[str]:
    """Extract script result lines from nmap stdout.

    :param stdout: Raw nmap output.
    :return: Parsed output lines with pipe prefixes stripped.
    """
    findings = []
    for line in stdout.splitlines():
        m = _NSE_LINE.match(line.strip())
        if m:
            text = m.group(1).strip()
            if text:
                findings.append(text)
    return findings


def run(discovery_result: NetworkDiscoveryResultMap) -> dict[str, list[str]]:
    """Run service-specific nmap NSE scripts on all discovered services.
```

»»»»> Stashed changes